



Specification and Verification of Multi-Clock Systems Using a Temporal Logic with Clock Constraints

YUANRUI ZHANG, College of Computer Science and Technology/College of Software, Nanjing University of Aeronautics and Astronautics, Nanjing, China and Southwest University, Chongqing, China

FREDERIC MALLET, Université Côte d'Azur, Nice, France

MIN ZHANG, East China Normal University, Shanghai, China

ZHIMING LIU, Northwest University, Xi'an, China

The polychronous or multi-clock paradigm is adequate to model large distributed systems where achieving a full timed synchronization is not only very costly but also often not necessary. It concerns systems made of a set of components with loose synchronization constraints. We study an approach where those components are orchestrated using *logical clocks*, made popular by L. Lamport and synchronous languages. The temporal and causal specification of those systems is built by defining a set of *clock relations* that would constrain the instant when clocks can tick or must not tick, thus defining families of *valid schedules*. In this article, we propose a specification language, called $LTL_c/CCSL$, for specifying temporal properties of multi-clock systems. While traditional temporal logics (LTL, MTL, CTL*), whether linear or branching, rely on a global step, our language, $LTL_c/CCSL$, builds a partial order on logical clocks, thus allowing both a hierarchical approach based on refinement of clock hierarchies and compositionality, as what happens in one clock domain may remain largely independent of what may happen in other domains. This good property helps preserve the properties without requiring to perform the proofs again. An $LTL_c/CCSL$ specification consists of a clock temporal logic LTL_c , accompanied by a clock calculus called $CCSL$ for specifying clock relations. We build the syntax and semantics of LTL_c and link its semantics with $CCSL$. After that, we mainly focus on the verification aspect of $LTL_c/CCSL$ specifications using a model checking technique. We show how $LTL_c/CCSL$ can be used for specifying multi-clock systems with an example.

CCS Concepts: • **Software and its engineering** → *Software safety*; **Specification languages**; **Formal software verification**; • **Theory of computation** → **Linear logic**;

Additional Key Words and Phrases: Multi-clock systems, temporal logic, clock constraints, formal specification, model checking

This work was partially supported by the Youth Project of National Science Foundation of China (62102329), the Project of National Science Foundation of Chongqing (cstc2021jcyj-bshX0120), the Capacity Development Grant of Southwest University (SWU116007), the Project of National Science Foundation of China (62272397), and the Key Projects of National Science Foundation of China (61732019 and 62032019).

Author Current Affiliation: Zhiming Liu, School of Computer and Information Science, Southwest University, Chongqing, China and Northwest University, Xi'an, China.

Authors' Contact Information: Yuanrui Zhang, College of Computer Science and Technology/College of Software, Nanjing University of Aeronautics and Astronautics, Nanjing, Jiangsu, China and Southwest University, Chongqing, China; e-mail: zhangyrmath@126.com; Frederic Mallet, Université Côte d'Azur, Nice, Provence-Alpes-Côte d'Azur, France; e-mail: Frederic.Mallet@inria.fr; Min Zhang, East China Normal University, Shanghai, Shanghai, China; e-mail: zhangmin@sei.ecnu.edu.cn; Zhiming Liu (Corresponding author), Southwest University, Chongqing, China and Northwest University, Xi'an, Shaanxi, China; e-mail: zhimingliu88@swu.edu.cn.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from [permissions@acm.org](https://permissions.acm.org).

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 1433-299X/2024/06-ART13

<https://doi.org/10.1145/3670794>

ACM Reference Format:

Yuanrui Zhang, Frederic Mallet, Min Zhang, and Zhiming Liu. 2024. Specification and Verification of Multi-Clock Systems Using a Temporal Logic with Clock Constraints. *Form. Asp. Comput.* 36, 2, Article 13 (June 2024), 51 pages. <https://doi.org/10.1145/3670794>

1 Introduction

Large-scaled distributed real-time systems, such as cyber-physical systems [28] and Internet of Things [9], are integrations of heterogeneous subsystems that are interconnected through a (heterogeneous) network. These systems are often complex, with components loosely coupled and often designed using different **models of computation (MoCs)**. Specifying time constraints of this type of systems is a great challenge, because each component has its own notion of time (i.e., execution speed) depending on local requirements (e.g., sampling period of sensors) and there is no optimal global notion of time. For instance, some parts must react every millisecond, whereas other parts react every second and yet some other parts react only according to sporadic events. In such systems, *logical relations* between events play a dominant role, as what really matters is the relative ordering between events of different components rather than the physical times at which events actually occur [20, 30].

Figure 1 gives an example of an heterogeneous system to see this. A **power window (PW)** system of a vehicle consists of three components: a window-control switch (simply *switch*), a central controller (simply *controller*) and the window itself. Three components are assumed to be modelled by different MoCs, and their notions of time may vary (this is fully illustrated in Section 4). For instance, in the switch, a sequence of digital signals are generated through port U_s at a fixed frequency, say, every 4 ms, when a user keeps pressing the button. However, in the controller, there are no minimal timescales. The controller only samples the signals from the switch through port U_1 in a discrete manner and sends a corresponding signal to the window after an inner computation. Here, the two behaviours through ports U_s and U_1 , respectively named event U_s and event U_1 , follow a *synchronous* relation. It is a logical relation in the sense that it has nothing to do with explicit physical times at which U_s and U_1 occur; what only matters is that “some signal of U_s is immediately received by U_1 without buffering it”. The occurrence of U_s and its corresponding occurrence of U_1 are thought to be “at the same *instant*”—an assumption indicating that their before-after order (in a physical time sense) is irrelevant and can be ignored.

In this example, and in the article in general, we use the word *clock* instead of ‘event’, as *logical clocks* [26] ultimately capture remarkable discrete events of the system. The *ticks* of a clock represent the sequence of occurrences of its corresponding event. The logical relation seen previously, also called a *clock relation*, coordinates the behaviours of different components so that they act accurately and timely to their local notions of time. The idea of abstracting explicit physical time and only considering relative relations of occurrences of events has been deeply embedded into the design paradigms of now popular modelling languages for real-time distributed systems, like Esterel [12], Signal [11], Simulink [1], Ptolemy [39] and UML/MARTE [38], or even SCADE [17], whose certified compiler is largely used for the design of safety critical systems like aircrafts or trains.

A system model in which there does not exist a global clock to synchronize all discrete events is called a *multi-clocked* (or *polychronous* in some works (e.g., [27])) system. In such a system, clocks may remain independent from each other without necessarily having a (known or at least explicit) relation between them. This happens when either designers do not have the mechanisms to compare all of the events or they do not want to pay the cost of being able to do so. In the controller of Figure 1, for example, due to the uncertain inner computations, the precise time

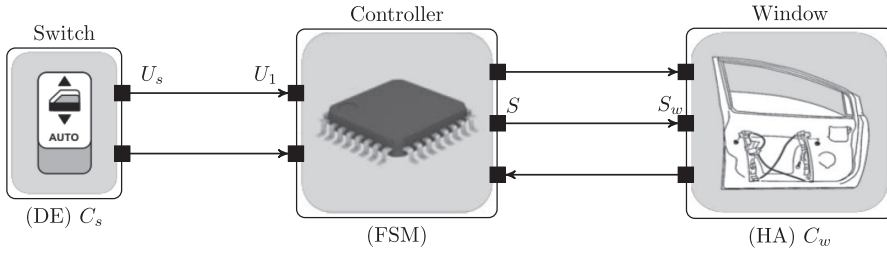


Fig. 1. An example of multi-clock systems.

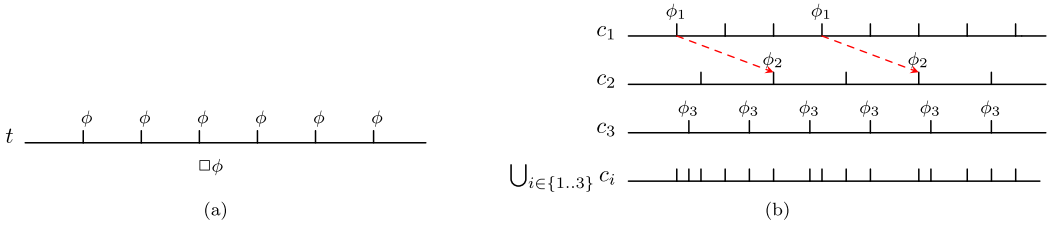


Fig. 2. A global timeline (left) vs. multiple local timelines (right).

interval between successive occurrences of event S in the controller cannot be guaranteed (see the example in the work of Lee and Lohstroh [29]). However, event S_w , which receives messages generated by event S , is triggered precisely every 32 ms. In this case, there is no way to make sure that the two events S and S_w are synchronized by some third clock and thus they are incomparable.

Since there might exist no global notion of time, for a multi-clock system, temporal properties are, and should be, only related to the local clocks of each component. This is because it makes little sense to describe a property for all time/steps of the model. Instead of a temporal property like “property ϕ holds at all time/steps” described by a traditional **linear temporal logic (LTL)** formula $\Box\phi$ (Figure 2(a)), temporal properties related to local clocks are preferred, such as follows:

- (1) “at all steps where clock c_3 ticks, property ϕ_3 holds” (see Figure 2(b)), or
- (2) “whenever property ϕ_1 holds at some step where clock c_1 ticks, there will be a (future) step where clock c_2 ticks and property ϕ_2 holds” (indicated by dashed arrows in Figure 2(b)).

These *local temporal properties*, as we call them, are built upon “multiple timelines” indicated by local clocks. They capture the time structure of a multi-clock system in a clear way. If we had to reconcile the three timelines into a single one (see the bottom timeline), the expected property would be much harder to express. Moreover, as we discuss later, such local properties are more robust to changes within the system models, since the time properties may remain completely independent from the new clocks or events that might be introduced later on, or from other clock domains.

In this article, we propose a specification language to handle explicitly logical clocks and clock relations, so as to specify the temporal properties of multi-clock systems as discussed earlier. The language is a temporal logic called *clock temporal logic* (LTL_c for short), which is accompanied with a clock calculus, namely **Clock Constraint Specification Language (CCSL)** [35], for specifying logical relations among clocks. An $LTL_c/CCSL$ specification $\langle \phi, \Phi \rangle$, where ϕ is an LTL_c formula and Φ is a set of CCSL constraints, captures a different set of temporal properties from traditional temporal logics. It is proved that CCSL has a different expressive power from LTL (cf. [21]). Unlike temporal logics such as LTL, **metric temporal logic (MTL)** [6] or **metric interval temporal**

logic (MITL) [5], which only focus on “a property on a set of global states (steps)”, LTL_c binds each atomic proposition and each temporal operator with a clock, like $\Box_c$, where the clock (here c) plays the role of a filter that ‘extracts’ the states at which c ticks. For example, formula $\Box_{c_3}\phi_3$, which specifies the preceding property (1), expresses that at every state *where clock c_3 ticks*, the formula ϕ_3 holds. Clocks appearing in LTL_c formulae form a partially ordered time model where multiple timelines co-exist. This is more general than branching time, as we show in the article. Indeed, timelines may also represent alternative executions (branch) and not only concurrent clock domains of the same execution. CCSL expresses pure logical relations between clocks. For instance, the relation between U_s and U_1 mentioned previously as a kind of synchronous relations can be expressed as a clock relation $U_1 \subseteq U_s$ in CCSL. CCSL provides a kernel set of clock expressions that are expressive enough to capture most interesting clock relations [36], and it has been proven as a powerful formalism for specifying distributed real-time systems [8, 22, 25, 40, 41]. $LTL_c/CCSL$ enriches CCSL specifications with the ability to express quantitative properties concerning values rather than pure clock relations.

In an $LTL_c/CCSL$ specification, clocks are *first-class citizens* in the logic, providing a hierarchical model of time. With clocks, an $LTL_c/CCSL$ specification syntactically isolates temporal properties from the local clocks to which these properties are only related. CCSL clock constraints stand for capturing the temporal and time interrelations among components of the system. Such a hierarchical structure supports a convenient refinement process of time constraints by simply adding new clock constraints into the existing set Φ of an $LTL_c/CCSL$ specification $\langle\phi, \Phi\rangle$, while keeping the form of the LTL_c formula ϕ unchanged.

As an example, consider an $LTL_c/CCSL$ specification

$$sp = \langle\Box_{c_1}(\phi_1 \rightarrow \Diamond_{c_2}\phi_2), \{c_1 < c_2\}\rangle,$$

where formula $\Box_{c_1}(\phi_1 \rightarrow \Diamond_{c_2}\phi_2)$ actually captures Property (2) mentioned earlier, which is based on the local clocks c_1 and c_2 . Clock constraint $c_1 < c_2$ describes a *precedence relation* between c_1 and c_2 , meaning “every tick of c_2 is dependent on a previous tick of c_1 ”. Intuitively, formula $\Box_{c_1}(\phi_1 \rightarrow \Diamond_{c_2}\phi_2)$ is independent from the constraint $c_1 < c_2$, since its meaning has nothing to do with how the two clocks relate to each other. Assuming in a refined system model a local clock c_3 that has a relation with c_2 as $c_3 \leq c_2$, the new specification becomes

$$sp' = \langle\Box_{c_1}(\phi_1 \rightarrow \Diamond_{c_2}\phi_2), \{c_1 < c_2, c_3 \leq c_2\}\rangle,$$

where the temporal property $\Box_{c_1}(\phi_1 \rightarrow \Diamond_{c_2}\phi_2)$ still holds.

In this article, after establishing $LTL_c/CCSL$, we mainly focus on the verification of $LTL_c/CCSL$ specifications. We propose a model checking approach for verifying an $LTL_c/CCSL$ specification given a **clock transition system (CTS)** for modelling the clock behaviours of a system. The main idea behind that is similar to the model checking approach for LTL, which relies on the transformation from logical formulae into transition systems [10]. Since the automata of $LTL_c/CCSL$ specifications might be infinite (as shown in Section 5.2), we only succeed in verifying a subset of $LTL_c/CCSL$ specifications whose state space is bounded.

We begin by first building CTSs as an automata semantics for clock behaviours and **CTSs with accepting states (CTSAs)** as an automata semantics for LTL_c formulae and CCSL constraints (Section 5.1, 5.2). Then we analyze the composition of CTSAs (Section 5.3) and show how the automata semantics of CCSL connects to CCSL expressions (Section 5.4). With these as foundations, the verification of $LTL_c/CCSL$ specifications can be carried out by transforming both LTL_c formulae and CCSL constraints into CTSAs, and make reachability analysis on the composition of these CTSAs and the CTS of a multi-clock system. The transformation from LTL_c formulae into CTSAs relies on the transformation from LTL formulae into Büchi automata, so we also

analyze the expressiveness of LTL_c and show how to encode an LTL_c formula into an LTL formula (Section 2.2).

In summary, the main contributions of this article include the following:

- (1) We propose a novel specification language $LTL_c/CCSL$, which is a clock temporal logic LTL_c accompanied with an existing clock calculus $CCSL$, to specify temporal properties of multi-clock systems. These properties are related to logical clocks and clock constraints and cannot be simply expressed by LTL.
- (2) We propose a model checking technique for verifying a set of bounded $LTL_c/CCSL$ specifications. To support it, we propose the observation theory of CCSL and analyze how the behaviour of a CTSA of a set of $CCSL$ constraints relates to its semantics.

The rest of the article is organized as follows. In Section 2, we propose LTL_c and compare its expressiveness with LTL. In Section 3, we give the semantics of $LTL_c/CCSL$ specifications. Section 4 analyzes a case study and shows how $LTL_c/CCSL$ specifications can be used to specify properties of multi-clock systems. In Section 5, we propose an automata semantics for $LTL_c/CCSL$ and an observation theory of CCSL based on the automata semantics. Based on Section 5, Section 6 mainly introduces a method for verifying $LTL_c/CCSL$ specifications. We discuss related work in Section 7. Section 8 concludes this article and discusses possible future work.

2 Clock Temporal Logic (LTL_c)

In this section, we propose LTL_c ¹, to specify temporal properties based on multiple local clocks. We first introduce the syntax and semantics of LTL_c in Section 2.1, then analyze its expressiveness in Section 2.2.

2.1 Syntax and Semantics of LTL_c

Before we define the syntax of LTL_c , we introduce the notion of logical clocks. In a distributed system, a clock captures a remarkable discrete events, the ticks of a clock represent the sequence of occurrences of its corresponding event.

Definition 2.1 (Logical Clocks). A logical clock c is an infinite sequence of ticks, denoted as

$$c = c(1)c(2) \dots c(i) \dots$$

Our goal is to design a logic that can naturally express temporal properties based on local times without requiring the notion of a global time. To this end, we extend LTL by assigning a clock to each atomic formula and temporal operator, to express that the atomic formula or temporal operator relates to a time recorded by that clock. An atomic formula must be associated with a clock, because when we say a proposition holds, we actually mean it holds at the states where the clock ticks. In the following, sometimes we also call a state an *instant*, and call a state where a clock ticks an *instant of that clock*.

Definition 2.2 (Syntax of LTL_c). The syntax of LTL_c is defined as follows in BNF form:

$$\phi ::= tt \mid p_c \mid \neg\phi \mid \phi \wedge \phi \mid X_c\phi \mid \phi U_c\phi.$$

tt is Boolean true. We use ff to represent Boolean false. p represents an atomic proposition. The temporal operators X , U are the *next* and *until* operators in LTL (see Appendix A for a brief introduction to LTL). c is a *clock*. Each atomic formula p is associated with a clock c , meaning that we only care for the truth of p at the instants of clock c . Each temporal operator (X , U) is associated with a clock c , meaning that it is only relevant at the instants where clock c ticks.

¹We use LTL_c instead of ‘CTL’ to distinguish it from that famous logic CTL by E. Clarke et al.

For a formula ϕ of the form $X_c\psi$ or $\psi_1U_c\psi_2$, we sometimes omit the associated clock of any atomic subformula of ϕ or of any temporal operator of a subformula of ϕ if it is c . For example, formula $X_c((p_1)_c \wedge X_c(p_2)_c)$ is often simply written as $X_c(p_1 \wedge Xp_2)$, where the associated clocks of p_1, p_2 and the operator X are omitted. Sometimes we use ϕ_c to denote a formula ϕ in which the associated clocks of all atomic formulae and all temporal operators are c . For example, formula $(p_1 \wedge Xp_2)_c$ just means $(p_1)_c \wedge X_c(p_2)_c$.

Given an LTL_c formula ϕ , we use $C(\phi)$ to denote the set of all clocks that appear in ϕ . For example, $C(X_c((p_1)_{c_1} \wedge X_c(p_2)_{c_2})) = \{c, c_1, c_2\}$.

Intuitively, LTL_c formulae capture the temporal properties relative to local times represented by logical clocks. The truth of an LTL_c formula not only depends on the truth of all of its atomic propositions but also all of its associated clocks. Clocks form a partially ordered time model in which the concept of a global time may not exist since clocks can be independent of each other.

For example, the formula $\Box_{c_1}(\phi_1 \rightarrow \Diamond_{c_2}\phi_2)$ discussed in Section 1 captures a temporal property (the property (2) in Section 1) relative to the logical clocks c_1 and c_2 . Its time model is shown in Figure 2(b), where there does not exist a minimum granularity of time to synchronize clocks c_1 and c_2 . The truth of this formula also depends on how clocks c_1 and c_2 tick at each instant.

The semantics of our proposed LTL_c is defined over a global configuration of clocks, in which whether a clock ticks or not at each instant is well defined. We first introduce the concept of traces and configurations based on which the semantics of LTL_c is defined in Definition 2.4.

Definition 2.3 (Trace and Configuration). A trace π is an infinite sequence:

$$\pi = q_0q_1 \dots q_i \dots,$$

where each q_i represents the system state at instant i .

Given a set of atomic propositions Π and a set of clocks Clk , let $S(\pi)$ be the set of states in the trace π , and a *configuration* is defined as a triple $\langle \pi, \theta, \xi \rangle$, where $\theta : S(\pi) \rightarrow 2^\Pi$ maps each state to a set of atomic propositions holding at the state, and $\xi : S(\pi) \rightarrow 2^{Clk}$ maps each state to a set of clocks that tick at the state.

For any position $i \geq 0$, we use $\pi[i]$, $\pi[0, i]$ and $\pi[i, \infty]$ to denote the $(i + 1)$ -th element q_i of π , the finite prefix $q_0q_1 \dots q_i$ of π and the suffix $q_iq_{i+1} \dots$ of π , respectively.

The configuration $\langle \pi, \theta, \xi \rangle$ describes an execution trace π with θ capturing its behaviour and ξ capturing its time model. ξ describes the schedule of all clocks at each instant.

Definition 2.4 (Semantics of LTL_c). Given an LTL_c formula ϕ , for any configuration $\langle \pi, \theta, \xi \rangle$, the trace π satisfies ϕ with respect to θ and ξ , denoted as $\pi \models_{\theta, \xi} \phi$, is defined inductively according to the syntactical structure of ϕ :

- (1) $\pi \models_{\theta, \xi} p_c$ iff $c \in \xi(\pi[0])$ and $p \in \theta(\pi[0])$.
- (2) $\pi \models_{\theta, \xi} \neg\phi$ iff $\pi \not\models_{\theta, \xi} \phi$.
- (3) $\pi \models_{\theta, \xi} \phi_1 \wedge \phi_2$ iff $\pi \models_{\theta, \xi} \phi_1$ and $\pi \models_{\theta, \xi} \phi_2$.
- (4) $\pi \models_{\theta, \xi} X_c\phi$ iff there exists $i > 0$ s.t. $c \in \xi(\pi[i])$ and $\pi[i, \infty] \models_{\theta, \xi} \phi$, and for any k with $0 < k < i$, $c \notin \xi(\pi[k])$.
- (5) $\pi \models_{\theta, \xi} \phi_1U_c\phi_2$ iff there exists $i \geq 0$ s.t. $c \in \xi(\pi[i])$ and $\pi[i, \infty] \models_{\theta, \xi} \phi_2$, and for any $0 \leq j < i$, if $c \in \xi(\pi[j])$ then $\pi[j, \infty] \models_{\theta, \xi} \phi_1$.

We simply write $\pi \models \phi$ if θ and ξ are clear or do not matter in the context.

The formula p_c expresses that at the current state, clock c must tick while the proposition p holds. The formula $X_c\phi$ means that at the next state at which clock c ticks, ϕ holds. The formula $\phi_1U_c\phi_2$ expresses that ϕ_1 holds at each state where clock c ticks until a state where ϕ_2 holds.

From Definition 2.4, it is easy to see that in LTL_c temporal formulae, a clock plays the role of a filter that can select those states at which it ticks within an execution trace. In other words, LTL_c formulae only concern states at which relevant clocks tick.

Like LTL, in LTL_c we introduce the *always* operator $\Box$ and the *eventually* operator $\Diamond$ for conveniently expressing safety and liveness properties, and they are defined as

$$\Diamond_c \phi = ttU_c \phi, \Box_c \phi = \neg \Diamond_c \neg \phi.$$

From Definition 2.4, we can easily prove that the meanings of $\Diamond_c \phi$ and $\Box_c \phi$ are as stated next:

- $\pi \models_{\theta, \xi} \Diamond_c \phi$ iff there exists $i \geq 0$ s.t. $c \in \xi(\pi[i])$ and $\pi[i, \infty] \models_{\theta, \xi} \phi$.
- $\pi \models_{\theta, \xi} \Box_c \phi$ iff for all $i \geq 0$, if $c \in \xi(\pi[i])$ then $\pi[i, \infty] \models_{\theta, \xi} \phi$.

Intuitively, the formula $\Box_c \phi$ means that at all states where clock c ticks, the formula ϕ holds. The formula $\Diamond_c \phi$ means that there exists a state where clock c ticks and ϕ holds.

2.2 Expressiveness of LTL_c and Its Relation to LTL

The definition of traces (Definition 2.3) and the semantics of LTL_c (Definition 2.4) have actually indicated that an LTL_c formula can be expressed as an LTL formula if we take the occurrences of clocks as a special type of atomic propositions. For any clock c , its occurrence at each state can be captured as a proposition, named P_c . So its absence at each state can be captured as $\neg P_c$. With this observation, according to Definition 2.4, LTL_c can be encoded into LTL as stated in the following definition.

Definition 2.5 (Encoding from LTL_c Formulae to LTL Formulae). Any LTL_c formula ϕ can be encoded into an LTL formula, denoted by $\phi^\Diamond$. The encoding is defined inductively as follows:

- (1) $p_c^\Diamond = P_c \wedge p$.
- (2) $(\neg \phi)^\Diamond = \neg \phi^\Diamond$.
- (3) $(\phi_1 \wedge \phi_2)^\Diamond = \phi_1^\Diamond \wedge \phi_2^\Diamond$.
- (4) $(X_c \phi)^\Diamond = X(\neg P_c U(P_c \wedge \phi^\Diamond))$.
- (5) $(\phi_1 U_c \phi_2)^\Diamond = (P_c \rightarrow \phi_1^\Diamond) U(P_c \wedge \phi_2^\Diamond)$.

For any clock c , P_c is the corresponding atomic proposition that indicates the occurrence of c .

By encoding clocks as special atomic propositions, an execution trace over a set of clocks can then be modelled as an execution trace without clocks. Formally, we have the following definition.

Definition 2.6 (Configuration vs. Ordinary Configuration). A configuration $\langle \pi, \theta, \xi \rangle$ can be encoded as an ordinary configuration (see Appendix A for its definition) denoted as $\langle \pi, \theta^\xi \rangle$ without mapping ξ . Mapping θ^ξ is defined as

$$\theta^\xi(q) = \theta(q) \cup \{P_c \mid c \in \xi(q)\}.$$

The semantics of LTL_c formulae and their encoded LTL formulae have the following relationship.

THEOREM 2.1. *Given an LTL_c formula ϕ , for any configuration $\langle \pi, \theta, \xi \rangle$, let $\phi^\Diamond$ be the encoded LTL formula of ϕ and $\langle \pi, \theta^\xi \rangle$ be the corresponding ordinary configuration of $\langle \pi, \theta, \xi \rangle$, then*

$$\pi \models_{\theta, \xi} \phi \text{ iff } \pi \models_{\theta^\xi} \phi^\Diamond,$$

where the satisfaction relation $\models_\theta$ for any θ is defined in Appendix A.

Theorem 2.1 can be proved inductively based on the syntax of LTL_c (Definition 2.4).

THE PROOF OF THEOREM 2.1. The proof is by induction on the syntax of LTL_c formula ϕ .

If $\phi = p_c$, we prove that $\pi \models_{\theta, \xi} p_c$ iff $\pi \models_{\theta^\xi} p_c$. By the semantics of p_c (Definition 2.4), the semantics of $p_c^\triangleright$ (Appendix A) and the relation between θ and θ^ξ in Definition 2.6, we have $\pi \models_{\theta, \xi} p_c$ iff $c \in \xi(\pi[0])$ and $p \in \theta(\pi[0])$ iff $\pi \models_{\theta^\xi} P_c$ and $\pi \models_{\theta^\xi} p$ iff $\pi \models_{\theta^\xi} P_c \wedge p$ iff $\pi \models_{\theta^\xi} p_c^\triangleright$.

For the inductive case, we only take $\phi = X_c \phi$ for an example, and other cases are similar. By the inductive hypothesis, we know that for any configuration $\langle \pi', \theta', \xi' \rangle$, $\pi' \models_{\theta', \xi'} \phi$ iff $\pi' \models_{\theta'^\xi} \phi^\triangleright$. Now we prove that $\pi \models_{\theta, \xi} X_c \phi$ iff $\pi \models_{\theta^\xi} (X_c \phi)^\triangleright$. Again, by the semantics of $X_c \phi$ in Definition 2.4, the semantics of $(X_c \phi)^\triangleright$ in Appendix A and the relation between θ and θ^ξ in Definition 2.6, we have $\pi \models_{\theta, \xi} X_c \phi$ iff there exists $i > 0$ s.t. $c \in \xi(\pi[i])$ and $\pi[i, \infty] \models_{\theta, \xi} \phi$ and for any k with $0 < k < i$, $c \notin \xi(\pi[k])$ iff there exists $i > 0$ s.t. $\pi[i, \infty] \models_{\theta^\xi} P_c$ and $\pi[i, \infty] \models_{\theta^\xi} \phi^\triangleright$ and for any k with $0 < k < i$, $\pi[k, \infty] \models \neg P_c$ iff $\pi \models_{\theta^\xi} X(\neg P_c U(P_c \wedge \phi^\triangleright))$ iff $\pi \models_{\theta^\xi} (X_c \phi)^\triangleright$. $\square$

Generally, LTL_c is less expressive than LTL, since in LTL_c each atomic and temporal formula must be associated with a clock while an LTL formula does not have to be related to an atomic proposition of a clock. However, imagining that we allow a fictitious global clock, say c_0 , to tick at any time (i.e., at any states of any traces), then in this sense any LTL formula can be expressed by an LTL_c formula, which is formed by simply assigning all of its atomic formulae and temporal operators to clock c_0 . Formally, suppose there exists a clock c_0 s.t. for any configuration $\langle \pi, \theta, \xi \rangle$ and any state $q \in S(\pi)$, $c_0 \in \xi(q)$ holds, then an LTL formula ϕ can be encoded into the LTL_c formula ϕ_{c_0} .

Notice that although the occurrences of clocks in LTL_c in fact can be encoded as atomic propositions in LTL in terms of expressiveness, the clock itself is an explicit notion in the model of practical systems and is independent from the notion of system states. One characteristic of clocks that is distinguished from general atomic propositions is that a clock is always an infinite sequence and ticks for infinitely many times in a reactive system model.

3 LTL_c With a Set of Clock Constraints

In Section 2, we have seen that in LTL_c , all temporal properties are based on multiple clocks and these clocks together form a partially ordered time model. In multi-clock systems, clocks are often constrained by some tense or loose logical relations that cannot be expressed by LTL_c formulae itself. In this section, we use a pure clock calculus called CCSL to capture these constraints. CCSL helps coordinate local clocks of different system components by describing how those clocks are related to each other using an expressive set of clock-oriented expressions. By associating CCSL constraints with LTL_c formulae, we introduce $LTL_c/CCSL$ specifications for specifying properties of multi-clock systems. In Section 4, we show how $LTL_c/CCSL$ specifications can be used to specify multi-clock systems in a hierarchical manner.

First, Section 3.1 introduces the syntax and semantics of CCSL in the context of the semantics of LTL_c . Then, Section 3.2 defines $LTL_c/CCSL$ specifications. Finally, Section 3.3 compares the expressiveness of $LTL_c/CCSL$ specifications with that of LTL.

3.1 Clock Constraints

CCSL [7, 35, 36] is a clock-oriented specification language for specifying causal and temporal constraints. It was initially proposed as a companion language for UML/MARTE [38]. In CCSL, clocks are taken as primitive elements and clock constraints are expressed as logical relations between clocks. As an independent clock calculus, CCSL is mainly used for coordinating different components, which may be modelled by different MoCs, of a distributed system.

The definition of CCSL proposed in this section is mainly based on previous works [7, 36] but is altered to adapt the semantics of LTL_c . We also introduce the concepts of dependence relation

between clocks (Definition 3.3) and well-defined clock constraints (Definition 3.3), which were not addressed previously. They are necessary for building a rigorous theory of CCSL in LTL_c .

A CCSL constraint (or a clock constraint) can either be a *clock relation* or a *clock definition*. Clock relations describe binary relationships between clocks, whereas clock definitions define new clocks by combining existing clocks using different operators.

The syntax of clock constraints is given in the following definition.

Definition 3.1 (Clock Constraint). A clock constraint Cn is given as follows in the BNF form:

$$Cn ::= Rel \mid c \triangleq e,$$

where Rel is called clock relation, defined as

$$Rel ::= c_1 < c_2 \mid c_1 \leq c_2 \mid c_1 \# c_2 \mid c_1 \subseteq c_2,$$

$c \triangleq e$ is called *clock definition*, where the expression e is defined as

$$e ::= c_1 \mid c_1 + c_2 \mid c_1 * c_2 \mid c_1 \blacktriangleright c_2 \mid c_1 \triangleright c_2 \mid c_1 \curvearrowright c_2 \mid c_1 \propto n \mid c_1 \$ n \mid c_1 \vee c_2 \mid c_1 \wedge c_2.$$

c must be a different clock from c_1 and c_2 , $n \geq 1$.

The intuitive meaning of each clock constraint is given in the following after the semantics of CCSL constraints is introduced (in Definition 3.5). A set of CCSL constraints is also called a *CCSL specification*, denoted by Φ . We write Φ^r and Φ^d to denote the set of clock relations and the set of clock definitions in Φ , respectively, and therefore $\Phi = \Phi^r \cup \Phi^d$.

A specification Φ should at least contain one clock relation; otherwise, the specification does not give any constraints between the clocks. Therefore, we stipulate that for any specification Φ , $\Phi^r \neq \emptyset$.

In a CCSL specification, if a clock, named c , is defined by some clock definition $c \triangleq e$, we call it a *defined clock*. We call a clock a *pure clock* if it is not defined by any clock definitions. For example, in specification $\{c_1 < c_2, c_1 \triangleq c_3 + c_4\}$, clocks c_2, c_3, c_4 are pure clocks, whereas clock c_1 is a defined clock.

The clock constraints defined in Definition 3.1 are the main focus in this article; however, we also need to introduce the negation of a clock relation Rel , denoted as $\sim Rel$, which is used in the verification of LTL_c /CCSL specifications in Section 6.1. We use Cn^+ to denote any clock constraint or the negation of any clock relation. Formally, we define

$$Cn^+ ::= Cn \mid \sim Rel.$$

For convenience, in the rest of the article, we also call the negation of a clock relation a *clock constraint*. So we also call Cn^+ a *clock constraint*.

Given a set of clock constraints Φ , we use $\sim \Phi$ to denote the set of clock constraints obtained by replacing every clock relation in Φ with its negation. Formally,

$$\sim \Phi = \{c \triangleq e \mid (c \triangleq e) \in \Phi^d\} \cup \{\sim Rel \mid Rel \in \Phi^r\}.$$

Given a clock constraint Cn^+ , we use $C(Cn^+)$ to denote the set of all clocks that appear in Cn^+ .

There exist dependence relations between clocks. Clocks appearing in one constraint may be defined in some clock definition of another constraint. For example, in specification $\{c_1 < c_2, c_1 \triangleq c_3 + c_4\}$, clock c_1 in the constraint $c_1 < c_2$ is defined in the clock definition $c_1 \triangleq c_3 + c_4$. The following definition defines the dependence relation between clocks.

Definition 3.2 (Dependence Relation between Clock Constraints). A clock constraint Cn_1^+ *directly depends on* a clock definition $Cn_2^+ = (c \triangleq e)$, denoted by $Cn_1^+ \rightsquigarrow Cn_2^+$, if c appears in Cn_1^+ . We use $\rightsquigarrow^+$ to denote the transitive closure of $\rightsquigarrow$. If $Cn_1^+ \rightsquigarrow^+ Cn_2^+$, we say Cn_1^+ *depends on* Cn_2^+ .

Table 1. Semantics of CCSL Relations

$\sigma \models c_1 \subseteq c_2, \mathbf{D}$ iff for any $i \geq 0$, if $\sigma[i] \models c_1, \mathbf{D}$, then $\sigma[i] \models c_2, \mathbf{D}$.	(Subclock)
$\sigma \models c_1 \# c_2, \mathbf{D}$ iff for any $i \geq 0$, $\sigma[i] \not\models c_1, \mathbf{D}$ or $\sigma[i] \not\models c_2, \mathbf{D}$ holds.	(Exclusion)
$\sigma \models c_1 < c_2, \mathbf{D}$ iff for any $i \geq 0$, $\delta^{\sigma, \mathbf{D}}(c_1, i) > \delta^{\sigma, \mathbf{D}}(c_2, i)$, or $\delta^{\sigma, \mathbf{D}}(c_1, i) = \delta^{\sigma, \mathbf{D}}(c_2, i)$ and $\sigma[i] \not\models c_2, \mathbf{D}$.	(Precedence)
$\sigma \models c_1 \leq c_2, \mathbf{D}$ iff for any $i \geq 0$, $\delta^{\sigma, \mathbf{D}}(c_1, i) \geq \delta^{\sigma, \mathbf{D}}(c_2, i)$ holds.	(Causality)

Apparently, one situation that needs to be avoided when defining a CCSL specification is that the clocks of two constraints come from the definitions of each other. One example is constraints $\{c_1 \triangleq c_2 + c_3, c_2 \triangleq c_1 * c_4\}$, where clock c_2 in the constraint $c_1 \triangleq c_2 + c_3$ comes from the definition $c_2 \triangleq c_1 * c_4$, whereas clock c_1 in the constraint $c_2 \triangleq c_1 * c_4$ comes from the definition $c_1 \triangleq c_2 + c_3$. To forbid ill-formed constraints like this, we introduce the notion of *well-defined constraints* in the following definition.

Definition 3.3 (Well-Defined Clock-Constraint Set). A set of clock constraints Φ is well defined iff in Φ any two clock constraints are not dependent on each other.

Note that Definition 3.3 also forbids the situation when one clock is defined in two different constraints at the same time—for example, constraints $\{c_1 \triangleq c_2 + c_3, c_1 \triangleq c_4 * c_5\}$.

In this work, all clock-constraint sets we consider are assumed to be well defined.

To associate CCSL constraints with LTL_c formulae, we give the semantics of CCSL constraints over the clock sequences of configurations, which is called the *clock schedules* of configurations.

Definition 3.4 (Clock Schedule). A clock schedule σ (or simply a “schedule”) is an infinite clock sequence:

$$\sigma = C_0 C_1 \dots C_i \dots$$

where each C_i is a set of clocks that tick at the instant i .

Given a configuration $\langle \pi, \theta, \xi \rangle$, we call

$$\xi(q_0)\xi(q_1)\dots\xi(q_i)\dots$$

the clock schedule of $\langle \pi, \theta, \xi \rangle$, denoted as $\sigma^{\langle \pi, \theta, \xi \rangle}$. When θ and ξ are clear and can be omitted in the context, we often call $\xi(q_0)\xi(q_1)\dots\xi(q_i)\dots$ the schedule of the trace π , denoted as σ^π .

The semantics of CCSL constraints given next mainly follows other works [7, 36]. The main difference is that to define that a clock schedule satisfies a set of clock constraints, we define the semantics of both clock relations and clock definitions with respect to a set of clock definitions.

Definition 3.5 (Semantics of CCSL Constraints). The semantics of CCSL constraints is given w.r.t. a set of clock definitions $\mathbf{D}$. They are shown in Tables 1 and 2, where the history of a clock schedule σ (w.r.t. $\mathbf{D}$): $\delta_{\mathbf{D}}^\sigma : Clk \rightarrow (\mathbb{N} \rightarrow \mathbb{N})$ is defined as follows:

$$\delta^{\sigma, \mathbf{D}}(c, i) = |\{j \mid 0 \leq j \leq i, \sigma[j] \models c, \mathbf{D}\}|.$$

The history of clock schedule $\delta_{\mathbf{D}}^\sigma$ records the number of counts each clock has ticked up to the current instant. The satisfaction relation $\sigma \models A, \mathbf{D}$ (in Tables 1 and 2) means that σ satisfies expression A under the set of clock definitions $\mathbf{D}$. We use $\mathbf{D}\{Cn\}$ to mean that Cn is a constraint appearing in $\mathbf{D}$ (i.e., $Cn \in \mathbf{D}$). We often write $\sigma \models Cn, \mathbf{D}$ simply as $\sigma \models Cn$ when $\mathbf{D} = \emptyset$.

Table 1 lists the semantics of four basic clock relations. Subclock and Exclusion describe a synchronous relation between two events. Subclock captures a constraint between c_1 and c_2 in which c_1 can only tick if c_2 ticks at the same instant; Exclusion captures a constraint between c_1 and c_2 in which c_1, c_2 cannot tick at the same instant. Precedence and Causality describe asynchronous

Table 2. Semantics of CCSL Definitions

$\sigma[i] \models c, D$ (c is pure)	iff $c \in \sigma[i]$.	
$\sigma[i] \models c, D\{c \triangleq c'\}$	iff $\sigma[i] \models c', D$.	(Identity)
$\sigma[i] \models c, D\{c \triangleq c_1 + c_2\}$	iff $\sigma[i] \models c_1, D$ or $\sigma[i] \models c_2, D$.	(Union)
$\sigma[i] \models c, D\{c \triangleq c_1 * c_2\}$	iff $\sigma[i] \models c_1, D$ and $\sigma[i] \models c_2, D$.	(Intersection)
$\sigma[i] \models c, D\{c \triangleq c_1 \blacktriangleright c_2\}$	iff $\sigma[i] \models c_2, D$ and there is a j s.t. (1) $0 \leq j < i$ and $\sigma[j] \models c_1, D$, and (2) $\sigma[k] \not\models c_2, D$ for all $j \leq k < i$.	(Strict Sample)
$\sigma[i] \models c, D\{c \triangleq c_1 \triangleright c_2\}$	iff $\sigma[i] \models c_2, D$ and there is a j s.t. (1) $0 \leq j \leq i$ and $\sigma[j] \models c_1, D$, and (2) $\sigma[k] \not\models c_2, D$ for all $j \leq k < i$.	(Sample)
$\sigma[i] \models c, D\{c \triangleq c_1 \curvearrowright c_2\}$	iff $\sigma[i] \models c_1, D$ and $\sigma[j] \not\models c_2, D$ for all $0 \leq j \leq i$.	(Interruption)
$\sigma[i] \models c, D\{c \triangleq c' \propto n\}$	iff $\sigma[i] \models c', D$ and $\delta^{\sigma, D}(c', i) = n \cdot \delta^{\sigma, D}(c, i)$.	(Periodicity)
$\sigma[i] \models c, D\{c \triangleq c' \$ n\}$	iff $\delta^{\sigma, D}(c, i) = \max(\delta^{\sigma, D}(c', i) - n, 0)$.	(Delay)
$\sigma[i] \models c, D\{c \triangleq c_1 \wedge c_2\}$	iff $\delta^{\sigma, D}(c, i) = \max(\delta^{\sigma, D}(c_1, i), \delta^{\sigma, D}(c_2, i))$.	(Infimum)
$\sigma[i] \models c, D\{c \triangleq c_1 \vee c_2\}$	iff $\delta^{\sigma, D}(c, i) = \min(\delta^{\sigma, D}(c_1, i), \delta^{\sigma, D}(c_2, i))$.	(Supremum)

dependence relations between two events. Causality captures a constraint between c_1 and c_2 in which each tick of clock c_2 is caused by a tick of clock c_1 . In other words, c_2 cannot tick before c_1 ticks. Precedence captures a similar constraint as Causality, but it does not allow two clocks ticking at the same instant. In other words, it expresses that c_1 ticks strictly faster than c_2 ticks.

Table 2 lists the semantics of 10 basic clock definitions and one item (the 1st item) for the case when c is pure. Identity defines a clock c that ticks just as clock c' ticks. Union defines a clock c that ticks at the instant when either c_1 or c_2 ticks. Intersection defines a clock c that ticks at the instant when both c_1 and c_2 tick. Sample defines a clock c that samples c_1 based on c_2 . c ticks at the instant when c_2 ticks for the first time after a tick of c_1 . Strict Sample is similar to Sample, but it does not allow clock c to tick at the instant when both c_2 and c_1 tick. Interruption defines a clock c that ticks at the instant when c_1 ticks until c_2 ticks for the first time. Periodicity defines a clock c that ticks every n ticks of clock c' . c ticks at the instant when c' ticks, and there are n ticks of c' (including the tick of c' at this instant) after the last instant when c ticks. Delay defines a clock c that ticks at the instant when c' ticks and before the instant c' has ticked for at least n times. Infimum defines the slowest clock that is at least as fast as both c_1 and c_2 . c ticks at the instant when the faster clock between c_1 and c_2 ticks. Supremum defines the fastest clock that is at least as slow as both c_1 and c_2 . c ticks at the instant when the slower clock between c_1 and c_2 ticks.

Besides the basic clock constraints defined previously, we introduce a clock relation called *synchronization*, denoted as $c_1 \approx c_2$. Two clocks are synchronized when they always tick at the same instant. Formally, $c_1 \approx c_2$ is defined such that

$$c_1 \approx c_2 \text{ iff } c_1 \leq c_2 \text{ and } c_2 \leq c_1.$$

Intuitively, it means that c_1 and c_2 tick simultaneously only when their ticks are caused by each other.

The semantics of the negations of clock relations is given as the following definition.

Definition 3.6 (Semantics of the Negations of Clock Relations). Given a clock relation Rel and a set of clock definitions D , the semantics of $\sim Rel$ is defined s.t. for any schedule σ ,

$$\sigma \models \sim Rel, D \text{ iff } \sigma \not\models Rel, D.$$

After giving the semantics of each clock expression, we define the semantics of a clock specification. A schedule satisfies a set of clock constraints iff it satisfies each clock relation under the set of all clock definitions of this set.

Definition 3.7 (Semantics of Clock Specifications). The semantics of a well-defined clock constraints Φ is given as follows: for any clock schedule σ ,

$$\sigma \models \Phi \text{ iff } \sigma \models Cn, \Phi^d \text{ for each } Cn \in \Phi^r,$$

where Φ^r and Φ^d are the sets of clock relations and clock definitions in Φ , respectively.

3.2 $LTL_c/CCSL$ Specifications

We consider the LTL_c formulae, whose partially ordered time model is coordinated by CCSL expressions. We call them $LTL_c/CCSL$ specifications, whose definition is introduced as follows.

Definition 3.8 ($LTL_c/CCSL$ Specification). An LTL_c formula accompanied with a set of CCSL constraints, called an $LTL_c/CCSL$ specification, is a pair $\langle \phi, \Phi \rangle$, where ϕ is an LTL_c formula and Φ is a set of CCSL constraints associated with the clocks appearing in ϕ .

The constraints Φ captures the time model of the LTL_c formula ϕ —that is, how clocks in ϕ behave and relate to each other. Notice that LTL_c formulae are in fact independent from the selection of clock calculi. Other clock calculi instead of CCSL could also be considered as a companion of LTL_c to describe its time model.

The semantics of $LTL_c/CCSL$ specifications is given as follows by linking the semantics of LTL_c and the semantics of CCSL with configurations.

Definition 3.9 (Semantics of $LTL_c/CCSL$ Specifications). Given a configuration $\langle \pi, \theta, \xi \rangle$ and an $LTL_c/CCSL$ specification $\langle \phi, \Phi \rangle$, trace π satisfies the specification $\langle \phi, \Phi \rangle$ with respect to θ and ξ , denoted by $\pi \models_{\theta, \xi} \langle \phi, \Phi \rangle$, and is defined as

$$\pi \models_{\theta, \xi} \langle \phi, \Phi \rangle \text{ iff } \pi \models_{\theta, \xi} \phi \text{ and } \sigma^\pi \models \Phi.$$

3.3 Expressiveness of $LTL_c/CCSL$ Specifications

From Section 2.2, we see that the expressiveness of LTL_c is actually the same as LTL. The expressiveness of $LTL_c/CCSL$ specifications, however, is more expressive than LTL. The reason is that CCSL and LTL have different expressiveness. Gascon et al. [21] give a comparison between CCSL and PSL [3]—a logic that has the same expressiveness as ω -regular language and is more expressive than LTL. On the one hand, some CCSL constraints such as $c_1 < c_2$ cannot be expressed by PSL/LTL formulae. According to the semantics of $c_1 < c_2$ (see Table 1), to encode $c_1 < c_2$, one has to maintain the difference $\Delta_i = \delta^{\sigma, D}(c_1, i) - \delta^{\sigma, D}(c_2, i)$ between the counters $\delta^{\sigma, D}(c_1, i)$ and $\delta^{\sigma, D}(c_2, i)$ of c_1 and c_2 , respectively, at each step i . Because Δ_i is unbounded, it is impossible to encode $c_1 < c_2$ into a finite Büchi automaton. So $c_1 < c_2$ cannot be encoded as a PSL formula. On the other hand, CCSL fails in expressing liveness properties, such as $F\phi$ (read as “finally ϕ ”), as expressed by PSL formulae, since all CCSL constraints, according to their semantics (Definition 3.5), are safety properties—properties stating that some properties hold at every step. One can refer to the work of Gascon et al. [21] for more details about their comparison.

4 Specification of Multi-Clock Systems using $LTL_c/CCSL$ Specifications

This section shows how to specify properties of multi-clock systems using $LTL_c/CCSL$ specifications through a simple example. As we shall see, $LTL_c/CCSL$ specifications are natural to specify multi-clock systems, with CCSL to specify logical constraints between the clocks of different components and LTL_c to specify value-related properties upon these clocks. We show how $LTL_c/CCSL$ specifications give rise to a hierarchical structure adequate to specify multi-clock systems, with CCSL providing a convenient way for refining the clock constraints.

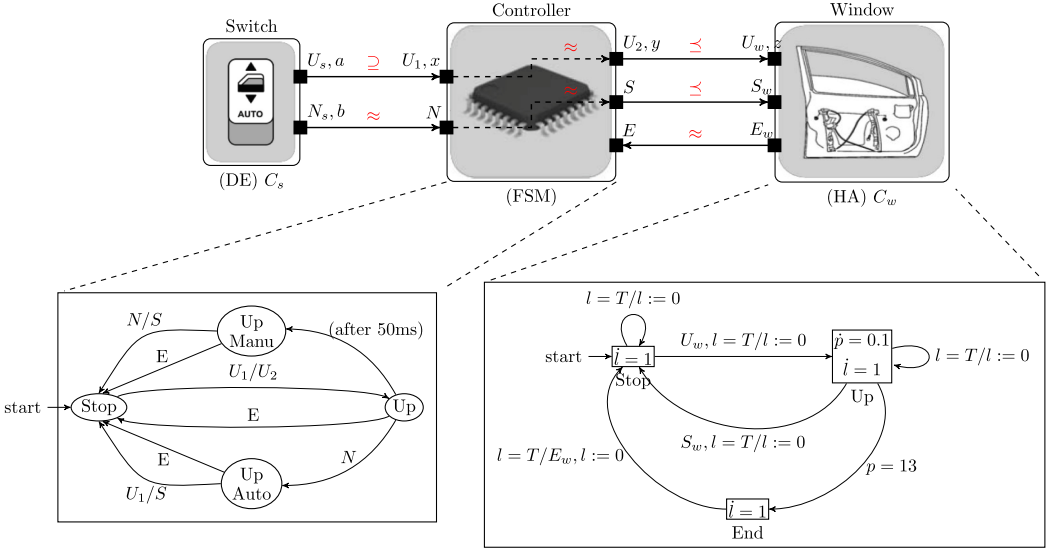


Fig. 3. PW system.

4.1 An Example of Multi-Clock Systems

As mentioned in Section 1, we introduce a simple multi-clock system—a PW system, which originally came from the work of Boulanger et al. [13]. As shown in Figure 3, the system consists of three components: a switch, a controller and a window.

The three components run in parallel and interact with each other by sending and receiving messages through different ports (i.e., U_s , N_s , U_1 , U_2 , N , S , E , U_w , S_w , E_w). Each port is associated with a clock (denoted by the same name) representing the occurrences of an event (denoted by the same name) on this port. For example, at every instant when clock U_s occurs, the switch sends a message from port U_s , whereas at every instant when clock U_1 occurs, the controller receives a message from port U_1 . We associate variables to some ports to store the messages sent or received from the ports. For example, we associate a variable a with the port U_s . A message sent from port U_s is stored in variable a , whereas a message received from port U_1 is stored in variable x .

In the PW system, we assume that three components are modelled by different MoCs: a discrete event (DE) model [31], a finite state machine (FSM) and a hybrid automaton (HA), respectively (see Figure 3). As shown in the following, the PW system is a multi-clock system since each component has its own notion of time and there does not exist a global clock that synchronizes all of the behaviours.

The behaviour of each component is illustrated as follows:

- The switch component is described as a discrete event model, where a behaviour is represented as a set of discrete events and each event is associated with a physical time value, called a *timestamp*. The switch continuously receives a user's inputs from the environment based on a local clock C_s with the period of 4 ms. Event U_s is triggered discretely when the user keeps pressing button Up . Event N_s is triggered once the user just releases button Up . The upper part of Figure 4 shows a possible behaviour of the user's inputs on the switch, where the occurrences of events U_s , N_s are bound with values of variables a , b , respectively. Note that for simplifying the model, we omit the discussion for the behaviour that the user presses or releases button *Down*. We also omit the corresponding behaviours in the controller and the window as shown in the following.

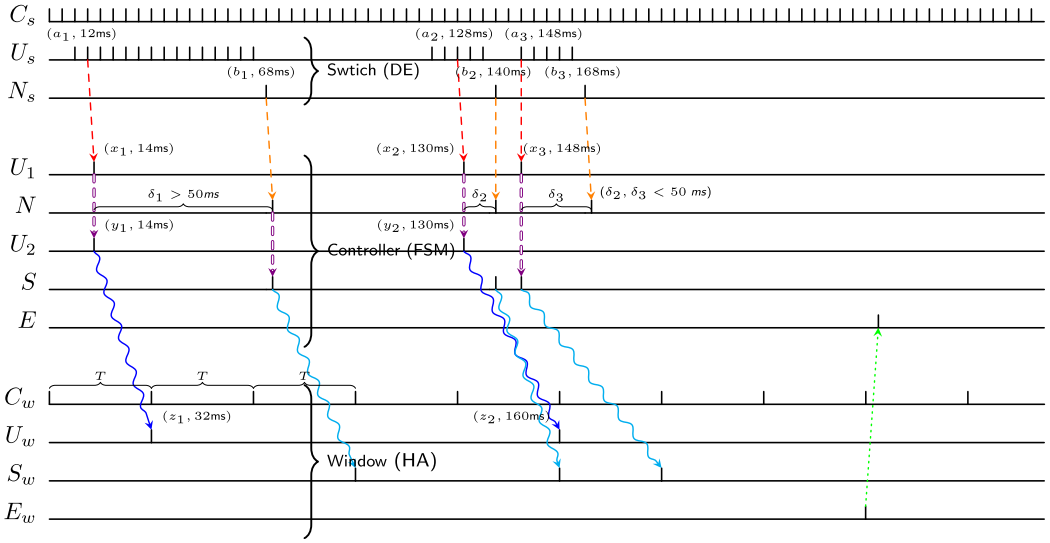


Fig. 4. A schedule of the PW system.

- The controller component interacts with both the switch and the window. It receives messages from the switch, and send messages to the window motor to move up and down the window. Event U_1 is triggered when receiving messages a from U_s and variable x is assigned to the value of a . Event N is triggered when receiving messages b from N_s . The controller is modelled by a Mealy-style timed finite state machine as shown in the lower part of Figure 3. It executes sporadically without being driven by a base clock. The controller offers two modes for moving up the window²:
 - The mode *Up Manual*, in which the window is moved up manually. If the user holds down button Up over 50ms or more, then the controller shall enter in this mode. The controller shall then send messages to the window motor to move the window up until the user releases button Up .
 - The mode *Up Automatic*, in which the window is moved up automatically. If the user releases button Up within less than 50ms, then the controller shall enter in this mode. The controller shall then send messages to the window motor to move the window up until the button is pressed again.

The transition of the timed finite state machine can be a timed transition, which indicates an elapse of time, or an event transition in a *guard/action* style, which means that once the *guard* is satisfied, the *action* is fired on transition without time consuming.

The behaviour of the controller is described as follows. At the state *Stop*, the controller receives a message from the switch that the button Up has been pressed through port U_1 and sends a message through port U_2 to the window motor to move up the window. This behaviour is captured by an event transition U_1/U_2 from state *Stop* to state *Up*. After reaching state *Up*, the controller returns to state *Stop* if it receives a message from the window through port E , meaning that the window is fully closed. If the controller receives a message from the switch through port N that means that button Up has been released within less than 50 ms, it shall move to state *Up Auto* (the *Up Automatic* mode), where

²Like in the switch, we omit the discussion of the behavior for moving down the window.

- (1) if the controller receives a message from the window through port E meaning that the window is fully closed, it shall move to state *Stop*;
- (2) if the controller receives a message from the switch through port U_1 meaning that button *Up* is pressed again, it shall send a message through port S to the window motor so as to stop moving the window.

On the other side, when in state *Up*, if the controller does not receive the message from the switch within 50ms, it shall move to state *Up Manu* (the *Up Manual* mode), where

- (1) if the controller receives a message from the window through port E meaning that the window is fully closed, it shall move to state *Stop*;
- (2) if the controller receives a message from the switch through port N meaning that button *Up* is released by the user, it shall send a message to the window motor so as to stop moving the window through port S .

The middle part of Figure 4 gives a possible schedule of the events in the finite state machine of the controller, where $\delta_1, \delta_2, \delta_3$ represent the time intervals between the receiving of the messages through events U_1 and N .

- The window component consists of a computational part, which we call *window controller*, and a physical part, called *motor*. The window controller behaves following a local clock with a period of T ms. It receives messages from ports U_w and S_w and drives the motor to physically move up or stop the window. When the window is moved to the top, the window controller stops the motor and sends a message back to the controller.

We model the window component as a Mealy-style hybrid automaton (shown in Figure 3), described as follows. Initially, at state *Stop*, the window controller listens to port U_w every T ms. Variable l represents the period of the local clock C_w . When event U_w occurs, the automaton jumps to state *Up*, where the motor starts moving the window up. Variable p indicates the position of the window, with $p = 0$ when the window is at the bottom and $p = 13$ when the window is at the top. During this process, the window controller prepares to receive a message from port S_w , and if it happens, the automaton jumps to state *Stop* at the tick of clock C_w . After the window reaches the top, the automaton transits to state *End*, waiting for sending a message through port E_w back to the controller. The lower part of Figure 4 describes a possible schedule of the discrete events in the hybrid automaton of the window.

The three components together form the PW system.

4.2 Specifying the PW System with $LTL_c/CCSL$ Specifications

In the PW system, as we see, each component has its own notion of time and there is no notion of “global time” for the model. So traditional LTL is not suitable to specify the model anymore.

In the following, we show that we can specify the PW system based on the local time of each of its three components using $LTL_c/CCSL$ specifications.

4.2.1 Local Time Properties in a Single Component. In the controller component, we consider the following property:

“the value received from the port U_1 is sent from the port U_2 immediately in the controller”.

This is a property that only relates to the local time, indicated by clocks U_1 and U_2 , in a component. We can specify the property as follows:

$$spec_{sp1} = \langle \Box_{U_1} (x = v \rightarrow ((y = v)_{U_2} \vee \mathbf{X}_{U_2} (y = v))), \Phi_{sp1} \rangle,$$

where $\Phi_{sp1} = \emptyset$. Note that for the controller component, the “immediate” moment to send the value from port U_2 is the right following tick of clock U_2 (U_2 might tick at the current instant together

with U_1 , which is actually our case shown in Figure 4). The specification means that whenever clock U_1 ticks, if the variable x equals to a value v , then the variable y is equal to the same value once clock U_2 ticks.

We see that the specification $spec_{sp1}$ is completely local to the controller component in the sense that when the local times of other components are changed, it does not affect the meaning of this specification.

4.2.2 Time Properties between Components. In the PW system, three components have their local times which are independent from each other. However, at some level of modelling, it is required that these component should be coordinated in some way so that the communication between them can work as expected. This is called *time adaption* [13] between different MoCs. $LTL_c/CCSL$ can be used either to specify or to enforce this type of system behaviour depending on different requirements.

For instance, we can claim that the communication between the controller and the switch should be synchronized in a way that the controller reacts sporadically to some of a user's inputs. This might be due to the limitation of the software/hardware capability of the controller. While the user's inputs (i.e., the digital signals converted from the behaviour of keeping pressing the Up button) are a consecutive sequence of discrete events with an interval of 4 ms between them, the controller only manage to respond to one of them.

To describe this communication, we build the following clock constraints:

$$U_1 \subseteq U_s, N_s \approx N.$$

The red and orange dashed arrows in Figure 4 describe these constraints. Note that synchrony does not necessarily mean "executing at the same physical time". In this example, we see that the occurrence (x_1 , 14ms) of U_1 is strictly after the occurrence (a_1 , 12ms) of U_s , whereas the occurrence (x_3 , 148ms) of U_1 is at the same physical time as the occurrence (a_3 , 148ms) of U_s . We build synchronous relations to guarantee that some (respectively, each) message of port U_s (respectively, port N_s) can be timely dealt with by port U_1 (respectively, port N) before the next message of port U_s (respectively, port N_s) was sent.

The communication from the controller to the window can be asynchronous. As indicated by the blue and light blue snake arrows in Figure 4, this is due to the fact that the period T is too large compared to the interval between the reactions of U_2 and S . So the messages of U_2 and S need to be stored in buffers before receiving by ports U_w and S_w of the window component. For example, the second and third occurrences of S are in one interval of clock C_w , thus the communication between S and S_w are asynchronous: the messages of S should be stored in a buffer before being received by S_w .

However, we specify the communication from the window (through port E_w) to the controller (through port E) to be synchronous, indicated by the green dashed arrow in Figure 4.

The following clock relations capture the clock constraints between the events of the two components:

$$U_2 \leq U_w, S \leq S_w, E_w \approx E.$$

To sum it up, the clock specification that describes the time adaption between the three components is as follows:

$$\Phi_{sp2} = \{U_1 \subseteq U_s, N_s \approx N, U_2 \leq U_w, S \leq S_w, E_w \approx E\}.$$

Except for time requirements, we can also consider data dependencies between the three components in $LTL_c/CCSL$. The process of specifying data constraints between different MoCs is called *data adaption* [13].

For example, we can consider the following property:

“The value sent through the port S will be eventually received from the port S_w ”.

It can be specified by an $LTL_c/CCSL$ specification given next:

$$spec_{sp2} = \langle \Box_S(y = v \rightarrow \Diamond_{S_w}(z = v)), \Phi_{sp2} \rangle,$$

which means that whenever clock S ticks, if the variable y equals to a value v , then the variable z will equal to the same value eventually when clock S_w ticks. Different from $spec_{sp1}$, this specification describes an asynchronous communication because $S \leq S_w$, which allows the switch to delay dealing with the message v .

From $spec_{sp2}$, we can see that formula $\Box_S(y = v \rightarrow \Diamond_{S_w}(z = v))$ actually is independent from the clock relations between clocks S and S_w . It is easy to change the time constraint between the two components without changing the local temporal property. In this example, if S and S_w are synchronized to each other, then we can change $spec_{sp2}$ by simply replacing the relation $S \leq S_w$ in Φ_2 with the relation $S \approx S_w$. In this case, formula $\Box_S(y = v \rightarrow \Diamond_{S_w}(z = v))$ is actually equivalent to $\Box_S(y = z)$, as S and S_w always tick at the same instant.

4.2.3 Refinement of $LTL_c/CCSL$ Specifications. As mentioned in Section 1, one of advantages of $LTL_c/CCSL$ is that it can well support the refinement of the time model of a multi-clock system.

For instance, in Section 4.2.1, we do not consider any time requirements inside the controller. From $spec_{sp1}$, we now consider a more refined time model by enriching the CCSL specification Φ_{sp1} with some clock constraints between the inputs and outputs of the controller. For example, we require that ports U_1, U_2 and ports N, S are synchronized, respectively, described as follows:

$$U_1 \approx U_2, N \approx S.$$

Their relations are shown by the violet dashed arrows in Figure 4.

The specification that refines $spec_{sp1}$ can thus be expressed as follows:

$$spec_{sp3} = \langle \Box_{U_1}(x = v \rightarrow ((y = v)_{U_2} \vee X_{U_2}(y = v))), \Phi_{sp3} \rangle,$$

where $\Phi_{sp3} = \{U_1 \approx U_2, N \approx S\} \supset \Phi_{sp1}$.

From the specification $spec_{sp2}$ in Section 4.2.2, we can consider a more refined time model between the three components, by introducing a more explicit connection between their local times.

For example, as shown in Figure 4, we define the new local clocks C_s and C_w to represent the minimum granularity of times of the switch component and the window component, respectively. Thus, all local clocks of these two components should be synchronized by C_s and C_w . We have the following clock constraints:

$$\Phi_0 = \{U_s \subseteq C_s, N_s \subseteq C_s, U_w \subseteq C_w, S_w \subseteq C_w, E_w \subseteq C_w\}.$$

With C_s and C_w , we make a further refinement of the time model by defining an explicit metric for T . Assume $T = 32$ ms. This can be realized by building a logical relation between C_s and C_w . We have that

$$\Phi_1 = \{C_w \approx u, u \triangleq C_s \alpha 8\},$$

where u is a newly defined clock. Since C_s ticks every 4 ms, C_w exactly ticks every eight ticks of C_s . Here, different from the previous specifications, synchrony specifies an exact coincidence between two physical time scales.

The specification after the two refinements Φ_0 and Φ_1 described earlier from $spec_{sp2}$ becomes

$$spec_{sp4} = \langle \Box_S(y = v \rightarrow \Diamond_{S_w}(z = v)), \Phi_{sp4} \rangle,$$

where $\Phi_{sp4} = \Phi_{sp3} \cup \Phi_0 \cup \Phi_1 \supset \Phi_{sp3}$.

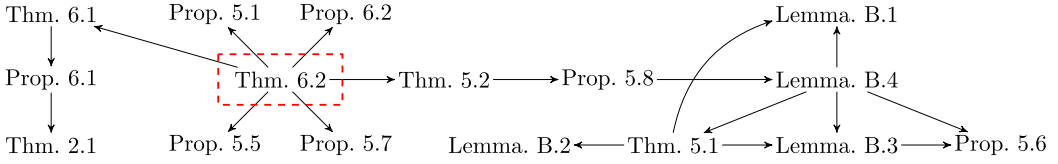


Fig. 5. Dependence relations between lemmas, propositions and theorems.

From this example, we see that such a connection of the time notions can be carried out at any modelling level of a system by simply adding new clock constraints into the old ones, without changing the form of the LTL_c formula itself. Such a separation of concerns of the properties and the time model is natural for specifying multi-clock systems.

5 Automata Semantics of $LTL_c/CCSL$

This section makes theoretical preparations for introducing a model checking approach for verification of $LTL_c/CCSL$ specifications in Section 6, where our final goal is to prove Theorem 6.2. The dependence relations between different lemmas, propositions and theorems in Section 5 and Section 6 are shown in Figure 5. In this section, we propose CTSs to capture the semantics of both LTL_c and $CCSL$ introduced in Section 2 and 3, and to facilitate the model checking process of $LTL_c/CCSL$. Unlike the model checking theory of LTL where the Büchi automaton of an LTL formula only accepts every behaviour of a system model, a CCSL constraint can itself generate behaviours. Thus, we need to show that the combination of several clock constraints is still an observer that can accept all behaviours of a system model. This is a step that is trivial in the theory of LTL model checking but not yet clear in CCSL, due to the difference of their calculi. Therefore, we propose an observation theory of CCSL and study the combination automata behaviours of clock constraints. This forms the foundation for analyzing how clock constraints relate to their automata behaviours in general and thus smooth the proposal of model checking technique further in Section 6.

In Section 5.1, we begin by introducing a special transition system—the CTS—as an automata semantics for the behaviors of clocks. Then, we introduce the CTSA, a more general CTS with accepting states, for capturing the semantics of LTL_c formulae and CCSL constraints. In Section 5.2, we introduce the CTSA semantics of CCSL constraints. After that, we propose an observation theory of CCSL in Section 5.3, which provides analysis bases for investigating the relationship between CCSL constraints and their corresponding CTSAs in Section 5.4.

5.1 Clock Transition System

We introduce the CTS, a transition system to capture the global configurations of clocks defined in Definition 2.3. The CTS enriches **ordinary transition systems (OTSs)** (cf. Appendix A) with states labelled by a set of clocks, which, as first-class citizens, indicate their occurrences at the current instant. Based on CTSs, we introduce a more general type of transition systems, called *CTSAs*, to capture the semantics of LTL_c formulae and CCSL constraints.

The idea of introducing clocks into transition systems was initially proposed in the work of Mallet and de Simone [36], where a **clock-labelled transition system (cLTS)** was defined as an automata semantics of CCSL. In cLTS, the occurrences of clocks were expressed as labels on transitions rather than labels on states. cLTSs are equivalent to our proposed CTS in terms of expressiveness, but they have a simpler form with fewer states. In this article, we use CTSs/CTSA as automata models of CCSL because they are more convenient in theoretical analysis of verification of $LTL_c/CCSL$ specifications.

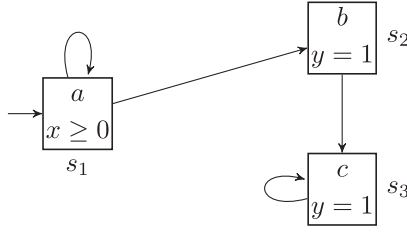


Fig. 6. An example of CTSs.

Definition 5.1 (Clock Transition System). A transition system with clocks is a 7-tuple $\mathcal{M} = \langle \Pi, Clk, Q, I, \theta, \xi, \mathcal{T} \rangle$ with the following components:

- (1) Π is a set of general atomic propositions.
- (2) Clk is a set of clocks.
- (3) Q is a (possibly infinite) set of states.
- (4) I is a finite set of initial states.
- (5) $\theta : Q \rightarrow 2^\Pi$ maps each state to the set of all propositions that are true in the state.
- (6) $\xi : Q \rightarrow 2^{Clk}$ maps each state to a set of clocks that tick in the state.
- (7) $\mathcal{T} \subseteq Q \times Q$ is a set of transitions.

We use p to represent a general atomic proposition. A transition $\langle q, q' \rangle \in \mathcal{T}$ means a computation that changes the state q to the state q' , and during the computation all clocks in $\xi(q')$ tick. $\xi(q)$ is often called the *clock behaviour* in a state q . We often use B to denote a clock behaviour.

Figure 6 shows a CTS $\mathcal{M} = \langle \Pi, Clk, Q, I, \theta, \xi, \mathcal{T} \rangle$, where $\Pi = \{x \geq 0, y = 1\}$, $Clk = \{a, b, c\}$, $Q = \{s_1, s_2, s_3\}$, $I = \{s_1\}$, and θ is defined s.t. $\theta(s_1) = \{x \geq 0\}$, $\theta(s_2) = \theta(s_3) = \{y = 1\}$. ξ is defined s.t. $\xi(s_1) = \{a\}$, $\xi(s_2) = \{b\}$ and $\xi(s_3) = \{c\}$. $\mathcal{T} = \{\langle s_1, s_1 \rangle, \langle s_1, s_2 \rangle, \langle s_2, s_3 \rangle, \langle s_3, s_3 \rangle\}$.

Given a CTS $\mathcal{M}$, we often use $P_{\mathcal{M}}$, where $P \in \{\Pi, Clk, Q, I, \theta, \xi, \mathcal{T}\}$, to represent the parameter P of $\mathcal{M}$. For example, $Clk_{\mathcal{M}}$ represents the set of clocks of $\mathcal{M}$.

Definition 5.2 (Trace, Configuration and Clock Schedule of a CTS). Let $\mathcal{M} = \langle \Pi, Clk, Q, I, \theta, \xi, \mathcal{T} \rangle$ be a CTS, $\pi = q_0 q_1 \dots q_i \dots$ is a trace of $\mathcal{M}$ defined s.t. $q_0 \in I$ and $\langle q_i, q_{i+1} \rangle \in \mathcal{T}$ for each $i \geq 0$. $\langle \pi, \theta, \xi \rangle$ is a configuration of $\mathcal{M}$, and σ^π is a clock schedule of $\mathcal{M}$.

A trace captures one possible behaviour of a CTS model.

We use $Trace(\mathcal{M})$ to denote the set of all traces accepted by $\mathcal{M}$.

We use $Sche(\mathcal{M})$ to denote the set of all schedules accepted by $\mathcal{M}$, and it is defined as

$$Sche(\mathcal{M}) = \{\sigma \mid \exists \pi. (\sigma^\pi = \sigma \wedge \pi \in Trace(\mathcal{M}))\}.$$

For a trace π (the same for a schedule σ^π) and a position $i \geq 0$, we use $\pi[i]$, $\pi[0, i]$ and $\pi[i, \infty]$ to denote the $(i + 1)$ -th element q_i of π , the finite prefix $q_0 q_1 \dots q_i$ of π and the suffix $q_i q_{i+1} \dots$ of π , respectively. Given a state $q \in Q$, we use $Trace(\mathcal{M}, q)$ (the same for $Sche(\mathcal{M}, q)$) to denote the set of all traces starting from q in $\mathcal{M}$: $Trace(\mathcal{M}, q) = \{\pi \mid \pi \in Trace(\mathcal{M}), \pi[0] = q\}$.

Definition 5.3 (Projection of Clock Schedules on CTSs). Given a schedule $\sigma = C_0 C_1 \dots C_i \dots$, the projection of σ on a CTS $\mathcal{M}$, denoted by $\sigma|_{\mathcal{M}}$, is defined as

$$\sigma|_{\mathcal{M}} = (C_0 \cap Clk_{\mathcal{M}})(C_1 \cap Clk_{\mathcal{M}}) \dots (C_i \cap Clk_{\mathcal{M}}) \dots$$

The parallel composition describes the behaviour when considering two CTSs run in parallel. It was first introduced in the work of Mallet and de Simone [36] for cLTS, and here we give a version for our proposed CTSs.

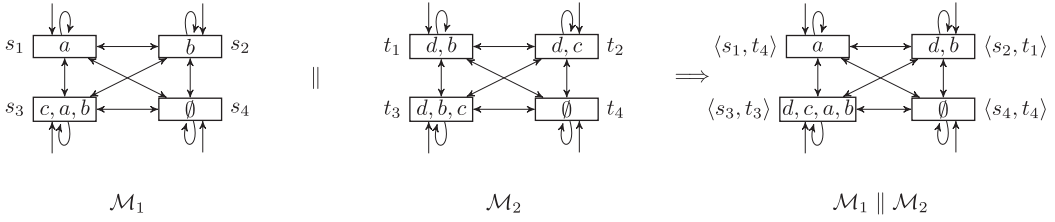


Fig. 7. An example of parallel composition.

Definition 5.4 (Parallel Composition of CTSs). Given two CTSs $\mathcal{M}_i = \langle \Pi_i, Clk_i, Q_i, I_i, \theta_i, \xi_i, \mathcal{T}_i \rangle$ ($i = 1, 2$), the parallel composition is defined as a CTS

$$\mathcal{M}_1 \parallel \mathcal{M}_2 = \langle \Pi, Clk, Q, I, \theta, \xi, \mathcal{T} \rangle,$$

where

- (1) $\Pi = \Pi_1 \cup \Pi_2$;
- (2) $Clk = Clk_1 \cup Clk_2$;
- (3) $Q = \{ \langle q_1, q_2 \rangle \mid q_1 \in Q_1, q_2 \in Q_2, Clk_1 \cap \xi_2(q_2) = Clk_2 \cap \xi_1(q_1), \Pi_1 \cap \theta(q_2) = \Pi_2 \cap \theta(q_1) \}$;
- (4) $I = \{ \langle q_1, q_2 \rangle \mid q_1 \in I_1, q_2 \in I_2, \langle q_1, q_2 \rangle \in Q \}$;
- (5) for each $\langle q_1, q_2 \rangle \in Q$, $\theta(\langle q_1, q_2 \rangle) = \theta_1(q_1) \cup \theta_2(q_2)$;
- (6) for each $\langle q_1, q_2 \rangle \in Q$, $\xi(\langle q_1, q_2 \rangle) = \xi_1(q_1) \cup \xi_2(q_2)$;
- (7) each transition $\langle \langle q_1, q_2 \rangle, \langle q'_1, q'_2 \rangle \rangle \in \mathcal{T}$ iff
 - (a) $\langle q_1, q_2 \rangle \in Q$ and $\langle q'_1, q'_2 \rangle \in Q$, and
 - (b) $\langle q_1, q'_1 \rangle \in \mathcal{T}_1$ and $\langle q_2, q'_2 \rangle \in \mathcal{T}_2$.

In the parallel composition of any two CTSs, any state $\langle q_1, q_2 \rangle$ must satisfy the conditions $Clk_1 \cap \xi(q_2) = Clk_2 \cap \xi(q_1)$ and $\Pi_1 \cap \theta(q_2) = \Pi_2 \cap \theta(q_1)$. The former means that at the state, two CTSs must agree on the behaviours of their common clocks (i.e., tick or not tick), whereas the latter means that at the state, two CTSs must agree on whether an atomic proposition is true or not.

Figure 7 shows that two CTSs $\mathcal{M}_i = \langle \Pi_i, Clk_i, Q_i, I_i, \theta_i, \xi_i, \mathcal{T}_i \rangle$ ($i = 1, 2$), where for simplicity we assume $\Pi_1 = \Pi_2 = \emptyset$, are composed together as $\mathcal{M}_1 \parallel \mathcal{M}_2$. $Clk_1 = \{a, b, c\}$, $Clk_2 = \{b, c, d\}$. Each rectangle node represents a state, and each arrow represents a transition. We use a state pointed by an arrow coming from outside to represent an initial state. To simplify the graphs, we use a bidirectional arrow between nodes A and B to represent both a transition from A to B and a transition from B to A . For example, in the CTS $\mathcal{M}_1$ of Figure 7, we use a bidirectional arrow between the state s_1 and the state s_2 to represent both transitions $\langle s_1, s_2 \rangle$ and $\langle s_2, s_1 \rangle$. Each state in $\mathcal{M}_1 \parallel \mathcal{M}_2$ must satisfy the conditions in item 3. For example, the state $\langle s_3, t_3 \rangle$ satisfies that $\xi_1(s_3) \cap Clk_2 = \{c, a, b\} \cap \{d, b, c\} = \{b, c\} = \{d, b, c\} \cap \{c, a, b\} = \xi_2(t_3) \cap Clk_1$.

The parallel composition is associative, as stated in the following proposition, which is proved in Appendix B.

PROPOSITION 5.1 (ASSOCIATIVITY OF PARALLEL COMPOSITION). Given three CTSs $\mathcal{M}_i = \langle \Pi_i, Clk_i, Q_i, I_i, \theta_i, \xi_i, \mathcal{T}_i \rangle$ ($i = 1, 2, 3$),

$$(\mathcal{M}_1 \parallel \mathcal{M}_2) \parallel \mathcal{M}_3 = \mathcal{M}_1 \parallel (\mathcal{M}_2 \parallel \mathcal{M}_3).$$

By Proposition 5.1, given n CTSs $\mathcal{M}_i$ ($1 \leq i \leq n$), the product $\mathcal{M}_1 \parallel (\mathcal{M}_2 \parallel \dots \parallel (\mathcal{M}_{n-1} \parallel \mathcal{M}_n) \dots)$ can be simply written as $\mathcal{M}_1 \parallel \mathcal{M}_2 \parallel \dots \parallel \mathcal{M}_n$, or $\|_{1 \leq i \leq n} \mathcal{M}_i$.

Definition 5.5 (Projection of States and Traces on CTSs). Given $\mathcal{M}_i = \langle \Pi_i, Clk_i, Q_i, I_i, \theta_i, \xi_i, \mathcal{T}_i \rangle$ ($1 \leq i \leq n$) and a product $\|_{1 \leq i \leq n} \mathcal{M}_i = \langle \Pi, Clk, Q, I, \theta, \xi, \mathcal{T} \rangle$, for any state $\langle q_1, \dots, q_n \rangle \in Q$, we

define

$$\langle q_1, \dots, q_n \rangle|_{\mathcal{M}_i} = q_i (1 \leq i \leq n)$$

as the projection of the state on the CTS $\mathcal{M}_i$.

Given a trace $\pi = q_0 q_1 \dots q_i \dots \in \text{Trace}(\|_{1 \leq i \leq n} \mathcal{M}_i)$, the projection of π on $\mathcal{M}_j$ ($1 \leq j \leq n$) is defined as

$$\pi|_{\mathcal{M}_j} = (q_0|_{\mathcal{M}_j})(q_1|_{\mathcal{M}_j})(q_i|_{\mathcal{M}_j}) \dots$$

For example, in the CTS $\mathcal{M}_1 \parallel \mathcal{M}_2$ in Figure 7, let $\pi = \langle s_1, t_4 \rangle \langle s_1, t_4 \rangle \langle s_1, t_4 \rangle \langle s_4, t_4 \rangle \langle s_3, t_3 \rangle \dots$ be a trace of $\mathcal{M}_1 \parallel \mathcal{M}_2$, then its projection on $\mathcal{M}_1$ is $\pi|_{\mathcal{M}_1} = s_1 s_1 s_1 s_4 s_3 \dots$

For a trace of the parallel composition of several CTSs, it is natural to think that the projection of the trace on one of these CTSs should also be one of its traces. The next proposition states this property.

PROPOSITION 5.2. *Given n CTSs $\mathcal{M}_i = \langle \Pi_i, \text{Clk}_i, Q_i, I_i, \theta_i, \xi_i, \mathcal{T}_i \rangle$ ($1 \leq i \leq n$), for any trace π ,*

$$\text{if } \pi \in \text{Trace}(\|_{1 \leq i \leq n} \mathcal{M}_i), \text{ then } \pi|_{\mathcal{M}_1} \in \text{Trace}(\mathcal{M}_1), \dots, \pi|_{\mathcal{M}_n} \in \text{Trace}(\mathcal{M}_n).$$

PROOF. Obviously from Definition 5.4. □

With a CTS, we can introduce the satisfaction relation between CTSs and LTL_c formulae.

Definition 5.6 (Satisfaction Relation between CTSs and LTL_c Formulae). Given a CTS $\mathcal{M} = \langle \Pi, \text{Clk}, Q, I, \theta, \xi, \mathcal{T} \rangle$ and an LTL_c formula ϕ , for any trace $\pi \in \text{Trace}(\mathcal{M})$, we define $\pi \models_{\mathcal{M}} \phi$ as

$$\pi \models_{\mathcal{M}} \phi \text{ iff } \pi \models_{\theta, \xi} \phi,$$

where θ and ξ appear as the parameters of $\mathcal{M}$.

For any state $q \in Q$, $q \models_{\mathcal{M}} \phi$ is defined s.t.

$$\text{for any trace } \pi \in \text{Trace}(\mathcal{M}, q), \pi \models_{\mathcal{M}} \phi.$$

In Definition 5.6, we define that a trace π satisfies an LTL_c formula ϕ in a CTS $\mathcal{M}$ as the satisfaction relation between the trace π , as the trace of the configuration $\langle \pi, \theta, \xi \rangle$, and the formula ϕ , which has been defined in Definition 2.4.

In verification of LTL_c /CCSL specifications, when introducing the automata semantics of CCSL constraints (in Section 5.2) and of LTL_c formulae (in Section 6.1.1), we need a type of more general CTSs with accepting states to allow CTSs to reject a certain set of traces. We introduce the CTSA as follows.

Definition 5.7 (Clock Transition Systems with Accepting States). A CTSA extends a CTS with a set of accepting states $\mathcal{E}$. It is defined as an 8-tuple

$$\mathcal{M}(\mathcal{E}) = \langle \Pi, \text{Clk}, Q, I, \theta, \xi, \mathcal{T}, \mathcal{E} \rangle, \text{ where } \mathcal{E} \subseteq Q.$$

A trace $\pi = q_0 q_1 \dots q_i \dots$ is accepted by $\mathcal{M}(\mathcal{E})$ iff

$$\text{there exists an infinite number of } i \geq 0 \text{ s.t. } q_i \in \mathcal{E}.$$

Similar to CTSs, we denote the set of all traces accepted by a CTSA $\mathcal{M}(\mathcal{E})$ as $\text{Trace}(\mathcal{M}(\mathcal{E}))$. Additionally, we define the set of all schedules accepted by $\mathcal{M}(\mathcal{E})$ as $\text{Sche}(\mathcal{M}(\mathcal{E})) = \{\sigma \mid \exists \pi. (\sigma^\pi = \sigma \wedge \pi \in \text{Trace}(\mathcal{M}(\mathcal{E})))\}$.

When no confusion occurs, sometimes we simply write a CTSA $\mathcal{M}(\mathcal{E})$ as $\mathcal{M}$ to hide $\mathcal{E}$.

Any CTS $\mathcal{M} = \langle \Pi, \text{Clk}, Q, I, \theta, \xi, \mathcal{T} \rangle$ can be taken as a special CTSA with all states being accepting states—that is, $\mathcal{M} = \mathcal{M}(Q) = \langle \Pi, \text{Clk}, Q, I, \theta, \xi, \mathcal{T}, Q \rangle$. Therefore, sometimes we do not distinguish CTSs and CSAAs and also call a CTS a CTSA.

Definition 5.8 (Or-Parallel-Composition of CTSA). Given two CTSAs $\mathcal{M}_i(\mathcal{E}_i) = \langle \Pi_i, \text{Clk}_i, Q_i, I_i, \theta_i, \xi_i, \mathcal{T}_i, \mathcal{E}_i \rangle$ ($i = 1, 2$), we define the product

$$\mathcal{M}(\mathcal{E}) = \mathcal{M}_1(\mathcal{E}_1) \parallel \mathcal{M}_2(\mathcal{E}_2)$$

just as in Definition 5.4, but with the accepting states $\mathcal{E}$ defined as

$$\mathcal{E} = \{ \langle q_1, q_2 \rangle \mid q_1 \in \mathcal{E}_1 \text{ or } q_2 \in \mathcal{E}_2 \}.$$

We abuse the symbol $\parallel$, which was used in Definition 5.4, for the composition between CTSA.

Definition 5.8 is a general version for Definition 5.4 since CTSA are special CTSA. It is called *Or-composition* because we only require that a trace of the composition passes the accepting states of one of its components. Thus, Proposition 5.2 does not hold for the composition between CTSA. Actually, it is easy to see that for any trace $\pi \in \text{Trace}(\mathcal{M}(\mathcal{E}))$, $\pi|_{\mathcal{M}_1(\mathcal{E}_1)} \in \text{Trace}(\mathcal{M}_1(\mathcal{E}_1))$ or $\pi|_{\mathcal{M}_2(\mathcal{E}_2)} \in \text{Trace}(\mathcal{M}_2(\mathcal{E}_2))$ holds.

5.2 CTSA Semantics of Clock Constraints

We introduce the CTSA semantics for clock constraints in CCSL. Each clock constraint corresponds to a CTSA which exactly captures its clock behaviour. An automata semantics of CCSL was given in the work of Mallet and de Simone [36], where it is based on cLTSs. Here we give an automata semantics of CCSL based on CTSA.

When discussing model checking of $LTL_c/CCSL$ specifications in Section 6.1, we also need to introduce the CTSA semantics for the negations of clock relations. The CTSA of the negation of a clock relation refuses all traces whose schedules satisfy this clock relation.

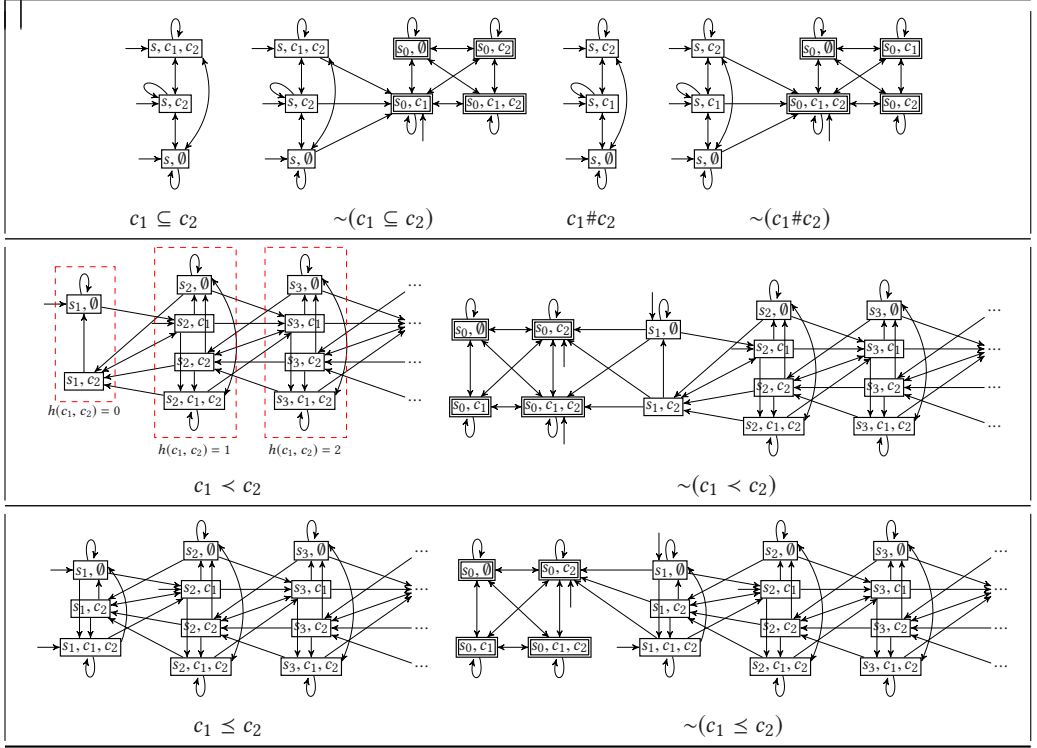
Definition 5.9 (CTSA Semantics of Clock Constraints). The CTSA semantics of CCSL constraints (including the negations of clock relations) is given in Table 3 and in Table 4 in Appendix C.

Given a clock constraint Cn^+ , we often denote its CTSA as $\mathcal{M}^{Cn^+}$.

Table 3 gives the CTSA semantics of each clock relation and its negation. We use a rectangle with a double border to represent an accepting state. For example, in the CTSA $\mathcal{M}^{\sim(c_1 \subseteq c_2)}$, the states $(s_0, \emptyset)$, (s_0, c_1) and (s_0, c_1, c_2) are accepting states, and the state (s_0, c_2) is both an initial state and an accepting state. As explained following Definition 5.4 in Section 5.1, for simplicity we use a bidirectional arrow between nodes A and B to represent both a transition from A to B and a transition from B to A . All states of any CTSA in Table 3 are only associated to sets of clocks, in other words, for any CTSA $\mathcal{M} = \langle \Pi, \text{Clk}, Q, I, \theta, \xi, \mathcal{T} \rangle$, $\Pi = \emptyset$. Each state is denoted in the form $(T, c_1, \dots, c_k)$ if the set of clocks associated to the state is $\{c_1, \dots, c_k\}$. T is a tag that classifies the states into different categories to make the CTSA easier to understand, as is explained in the following.

The automata of all clock relations are in fact CTSA, so we do not draw their accepting states. The difference between the automata of a clock relation and its negation is that those states of the CTSA of a clock relation are not accepting states in the CTSA of the negation of the clock relation. The latter contains some extra accepting states, tagged by s_0 , that the former does not have. A trace accepted by the CTSA of a clock relation is just rejected by the CTSA of the negation of the clock relation, and vice versa. For example, trace $\pi = (s, \emptyset)(s, c_2)(s, c_1, c_2)(s, c_2)(s, c_1, c_2)(s, c_2)(s, c_1, c_2) \dots$, which is accepted by the CTSA $\mathcal{M}^{c_1 \subseteq c_2}$, is rejected by the CTSA $\mathcal{M}^{\sim(c_1 \subseteq c_2)}$ since it never passes any accepting states of $\mathcal{M}^{\sim(c_1 \subseteq c_2)}$. However, trace $\pi' = (s, \emptyset)(s, c_1)(s, c_1, c_2)(s_0, c_2)(s_0, \emptyset)(s_0, \emptyset)(s_0, \emptyset) \dots$, which is accepted by the CTSA of the negation of $c_1 \subseteq c_2$, is rejected by the CTSA $\mathcal{M}^{c_1 \subseteq c_2}$, since once it passes one of the accepting states tagged by s_0 , it can never return to the states not tagged by s_0 .

Table 3. CTSA Semantics of Clock Relations and Their Negations



In the CTSA of some clock constraints, such as $c_1 < c_2$, $\sim(c_1 < c_2)$, $c_1 \leq c_2$ and $\sim(c_1 \leq c_2)$, the states tagged by different names, like $s_1, s_2, \dots$, have different indications on the differences between the numbers of the ticks of clocks. For example, in $\mathcal{M}^{c_1 < c_2}$, the states tagged by different names (drawn in different dashed red rectangles) indicate different differences between the numbers of the ticks of clocks c_1 and c_2 , denoted by $h(c_1, c_2)$. When a trace of $\mathcal{M}^{c_1 < c_2}$ passes any states with the same name, the values of $h(c_1, c_2)$ are always the same. For example, given a set of clock definitions D , let $\pi = (s_2, c_1)(s_3, c_1)(s_2, c_2)(s_3, c_1)(s_2, c_2)(s_3, c_1) \dots$ be a trace accepted by $\mathcal{M}^{c_1 < c_2}$, and each time the trace π passes the states tagged with s_2 (when $i = 0, 2, 4, 6, \dots$), we have $|\delta^{\sigma^\pi, D}(k, c_1) - \delta^{\sigma^\pi, D}(k, c_2)| = h(c_1, c_2) = 1$.

Table 4 (in Appendix C) gives the CTSA semantics of clock definitions. All automata in Table 4 are CTSSs, so we omit their accepting states. Their explanation is similar to that of the CTSA of clock relations and their negations given here, and we omit the details.

With the CTSA semantics for each constraint, we define the CTSA semantics for a set of constraints.

Definition 5.10 (CTSA Semantics of a Set of Clock Constraints). Given a set of clock constraints Φ , its CTSA semantics is defined as the parallel composition of the CTSA of all constraints in Φ —that is,

$$\parallel_{Cn^+ \in \Phi} \mathcal{M}^{Cn^+}.$$

Sometimes, we also denote the CTSA of a set of constraints Φ by $\mathcal{M}^\Phi$. For example, given $\mathcal{M}^{c_1 < c_2}$ and $\mathcal{M}^{c_1 \leq c_2}$ shown in Table 3, $\mathcal{M}^{c_1 < c_2} \parallel \mathcal{M}^{c_1 \leq c_2}$ is the CTS of the clock constraints $\{c_1 < c_2, c_1 \leq c_2\}$, denoted by $\mathcal{M}^{\{c_1 < c_2, c_1 \leq c_2\}}$.

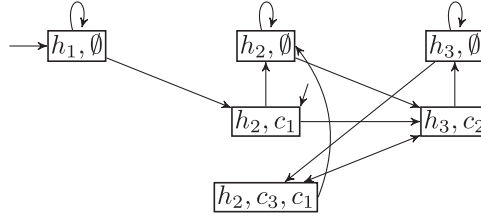


Fig. 8. The CTS of clock constraints $\{c_1 < c_2, c_2 < c_3, c_3 = c_1 \$ 1\}$.

From Tables 3 and 4, we can see that the CTSA of some clock constraints (e.g., $c_1 < c_2$) have an infinite number of states. Following Mallet et al. [37], we call a constraint (respectively, a set of constraints) whose corresponding CTSA has a finite number of states as a *bounded constraint* (respectively, a set of *bounded constraints*). Even though the CTSA of some constraints may be infinite, the parallel compositions of infinite CTSA are not necessarily infinite. Thus, a set of constraints might be bounded even some of the constraints are unbounded. One typical example is that $\mathcal{M}^{\{c_1 < c_2, c_3 \triangleq c_1 \$ 1, c_2 < c_3\}}$ (shown in Figure 8) is finite while $\mathcal{M}^{c_1 < c_2}$ and $\mathcal{M}^{c_2 < c_3}$ are infinite (see Table 3). More details about bounded CCSL constraints can be found in the work of Mallet et al. [37].

It is natural to think that there exists a correspondence relationship between the CTSA semantics of clock constraints given in this section and the schedule-based semantics given in Section 3.1. Intuitively, as indicated in the preceding analysis, for clock relations, when $D = \emptyset$, we can see that the schedules of a clock relation or its negation exactly correspond to the traces of its CTSA. The situation when $D \neq \emptyset$ is more complex. More details about their relation are fully analyzed in Section 5.4.

5.3 Observation Theory of CCSL

An *observer* “observes” any behaviour of its environment and behaves accordingly. In the traditional model checking theory of LTL [10], to verify that a system model $\mathcal{M}$ satisfies an LTL formula ϕ , reachability analysis is made on the product $\mathcal{M} \parallel \mathcal{M}^{\neg\phi}$ of $\mathcal{M}$ and the model $\mathcal{M}^{\neg\phi}$ of the LTL formula $\neg\phi$. In $\mathcal{M} \parallel \mathcal{M}^{\neg\phi}$, actually there is an implicit relation between $\mathcal{M}$ and $\mathcal{M}^{\neg\phi}$, in which any behaviour of $\mathcal{M}$ must be observed by $\mathcal{M}^{\neg\phi}$ in the sense that $\mathcal{M}^{\neg\phi}$ can proceed given any trace of $\mathcal{M}$. We call this relation an *observation relation* in our context in terms of CTSA, which is formally stated as follows.

Definition 5.11 (Observation Relation). Given two automata $\mathcal{M}_1$ and $\mathcal{M}_2$, we say $\mathcal{M}_2$ *observes* $\mathcal{M}_1$, abbreviated as $\mathcal{M}_2 \text{ obs } \mathcal{M}_1$, if for any trace $\pi \in \text{Trace}(\mathcal{M}_1)$,

there exists a trace $\pi' \in \text{Trace}(\mathcal{M}_1 \parallel \mathcal{M}_2)$ s.t. $\pi'|_{\mathcal{M}_1} = \pi$.

Note that the automata we mean in Definition 5.11 is general and can be either OTSA/Büchi automata or CTSA/CTSA.

In the model checking theory of LTL, the observation relation $\mathcal{M}^{\neg\phi} \text{ obs } \mathcal{M}$ is always guaranteed by the transformation from the LTL formula $\neg\phi$ into the Büchi automaton $\mathcal{M}^{\neg\phi}$. However, since the transformation from CCSL constraints into CTSA is not based on that of LTL (and neither CCSL and LTL have the same expressiveness), we need to prove that the observation relation also holds between a system model and the CTSA of a set of CCSL constraints.

To this end, in this subsection, we propose an observation theory for CCSL. Our goal is to study how the CTSA of a clock constraint can be an observer, how a composed observer is formed and under what conditions the observation relation between CTSA can be defined. The proposed theory

is necessary in analysis of the relation between clock constraints and their CTSA in Section 5.4, and of the proof of the model checking theory of $LTL_c/CCSL$ specifications in Section 6.1.

While studying the observation relation in CCSL in the following, we only focus on the clock behaviours of CTSA, because in the CTSA of a clock constraint, the set of atomic propositions is always empty so the parallel composition of two CTSA is only determined by their clock behaviours.

We begin by first defining *observer*. Intuitively, in CCSL, an observer over a set of clocks A is a special CTS/CTSA such that at any instant, no matter what a clock behaviour $B \subseteq A$ is offered by the environment, it can always proceed by agreeing on B .

Definition 5.12 (Observer). Let $\mathcal{M} = \langle \Pi, Clk, Q, I, \theta, \xi, \mathcal{T}, \mathcal{E} \rangle$ be a CTSA with $\Pi = \emptyset$. We call $\mathcal{M}$ an observer of a given clock behaviour $F \subseteq Clk$ if for any clock behaviour $X \subseteq F$ ($X \neq \emptyset$),

- (1) there exists an initial state $i \in I$ s.t. $\xi(i) \cap F = X$, and
- (2) for any $q \in Q$, there exists a transition $\langle q, q' \rangle \in \mathcal{T}$ s.t. $\xi(q') \cap F = X$.

An observer is denoted as $\mathcal{M}[F, C]$, with $C = Clk - F$.

We often call F the set of *free clocks*, and we call C the set of *constrained clocks* of an observer $\mathcal{M}[F, C]$.

Sometimes we use $F_{\mathcal{M}}, C_{\mathcal{M}}$ to respectively represent the sets of free clocks and constrained clocks of an observer $\mathcal{M}$.

In Definition 5.12, the observation made by an observer is reflected by the inclusion relation $X \subseteq \xi(i)$ (respectively, $X \subseteq \xi(q')$). Intuitively, it means that for any clock behaviour $X \subseteq F$ offered by the environment, the observer $\mathcal{M}$ agrees on it by offering $\xi(i)$ (respectively, $\xi(q')$). It is possible that $X \subset \xi(i)$ (respectively, $X \subset \xi(q')$), which means that set $\xi(i)$ (respectively, $\xi(q')$) might include other clocks not in X (and F).

As an example, $\mathcal{M}^{(c_1 < c_2)}$ (see Table 3) is an observer of the clock behaviour $\{c_1, c_2\}$. At any state, if a clock behaviour, such as $X = \{c_1\} \subseteq \{c_1, c_2\}$, is offered, $\mathcal{M}^{(c_1 < c_2)}$ can always proceed by offering $\{c_1\}$ or $\{c_1, c_2\}$. In a state of $\mathcal{M}^{(c_1 < c_2)}$, such as $(s_1, \emptyset)$, there exists a transition $\langle (s_1, \emptyset), (s_2, \{c_1\}) \rangle$ with $\xi((s_2, \{c_1\})) \cap \{c_1, c_2\} = \{c_1\} \cap \{c_1, c_2\} = \{c_1\} = X$. The CTSA $\mathcal{M}^{c \triangleleft c_1 + c_2}$ (see Table 4 in Appendix C) is an observer with the set of free clocks $\{c_1, c_2\}$ and the set of constrained clocks $\{c\}$. At any state, it observes any subset of $\{c_1, c_2\}$ and triggers clock c accordingly. For example, in the state $(s, \{c, c_1\})$, for the clock behaviour X , there exists a transition $\langle (s, \{c, c_1\}), (s, \{c, c_1\}) \rangle$ with $\xi((s, \{c, c_1\})) \cap \{c_1, c_2\} = \{c, c_1\} \cap \{c_1, c_2\} = \{c_1\} = X$.

PROPOSITION 5.3. *For the negation of any clock relation Rel , its CTSA $\mathcal{M}^{Rel}$ is an observer of the set of free clocks $C(Rel)$ (with the set of constrained clocks $\emptyset$). For any clock definition $c \triangleleft e$, its CTSA $\mathcal{M}^{c \triangleleft e}$ is an observer of the set of free clocks $C(e)$ (with the set of constrained clocks $\{c\}$).*

Proposition 5.3 is straightforward according to the CTSA semantics of clock constraints given in Section 5.2. We have seen the preceding examples $\mathcal{M}^{(c_1 < c_2)}$ and $\mathcal{M}^{c \triangleleft c_1 + c_2}$ as two observers.

Note that for any clock relation Rel , $\mathcal{M}^{Rel}[C(Rel), \emptyset]$ is not an observer. For example, in $\mathcal{M}^{c_1 < c_2}$ (see Table 3), for the state $(s_1, \emptyset)$ and the clock behaviour $X = \{c_2\}$, there exists no transition $\langle (s_1, \emptyset), q \rangle$ such that $\xi(q) \cap C(c_1 < c_2) = \{c_2\} = X$.

Following the dependence relation between clock constraints defined in Definition 3.2 in Section 3.1, we can also define a similar dependence relation between observers.

Definition 5.13 (Dependence Relation between Observers). Given two observers $\mathcal{M}_i[F_i, C_i] = \langle \Pi_i, Clk_i, Q_i, I_i, \theta_i, \xi_i, \mathcal{T}_i, \mathcal{E} \rangle$ ($i = 1, 2$), we say $\mathcal{M}_1$ *directly depends on* $\mathcal{M}_2$, denoted as $\mathcal{M}_1 \rightsquigarrow \mathcal{M}_2$, iff $C_2 \cap Clk_1 \neq \emptyset$. Similarly in Definition 3.2, we use $\rightsquigarrow^+$ to denote the transitive closure of $\rightsquigarrow$. We say $\mathcal{M}_1$ *depends on* $\mathcal{M}_2$ if $\mathcal{M}_1 \rightsquigarrow^+ \mathcal{M}_2$.

A set of observers A is *well defined* if in A there are no two observers that are dependent on each other.

The next proposition shows that for a subset of clock constraints and their corresponding observers, the dependence relation defined on them (in Definition 3.2 and Definition 5.13, respectively) is equivalent.

PROPOSITION 5.4. (i) For any clock constraints Cn_1^+ and Cn_2^+ , which are either the negations of some clock relations or clock definitions, $Cn_1^+ \rightsquigarrow Cn_2^+$ iff $\mathcal{M}^{Cn_1^+} \rightsquigarrow \mathcal{M}^{Cn_2^+}$.

(ii) A set of clock constraint Φ is well defined (cf. Definition 3.3) iff $\bigcup_{Cn^+ \in \sim_\Phi} \{\mathcal{M}^{Cn^+}\}$ is well defined.

We omit the proof of Proposition 5.4. Proposition 5.4 (i) can be easily proved by analyzing all CCSL constraints defined in Definition 3.1. Proposition 5.4 (ii) is a direct result of Proposition 5.4 (i) by Definition 3.3 and Definition 5.13.

Definition 5.12 defines an observer that observes a certain set of clocks from its environment. Now considering environment as a CTSA, we describe the relation that an observer can observe any clock behaviour of the CTSA.

Definition 5.14 (Observation Relation between an Observer and a CTSA). Let $\mathcal{M} = \langle \Pi, Clk, Q, I, \theta, \xi, \mathcal{T}, \mathcal{E} \rangle$ be a CTSA and $\mathcal{M}'[F', C'] = \langle \Pi', Clk', Q', I', \theta', \xi', \mathcal{T}', \mathcal{E}' \rangle$ be an observer, $\mathcal{M}'$ observes $\mathcal{M}$ through F' , abbreviated as $\mathcal{M}' \text{ obst}_{F'} \mathcal{M}$ (or simply $\mathcal{M}' \text{ obst } \mathcal{M}$), is defined s.t.

$$Clk \cap Clk' \subseteq F'.$$

The condition $Clk \cap Clk' \subseteq F'$ means that the common clock behaviours of $\mathcal{M}$ and $\mathcal{M}'$ must be included in the set of the free clocks of $\mathcal{M}'$ —that is, F' . Intuitively, when this condition is satisfied, we can guarantee that $\mathcal{M}'$ can observe any clock behaviour of $\mathcal{M}$ in the sense of Definition 5.12.

For example, given two observers $\mathcal{M}_1[F_1, C_1]$ and $\mathcal{M}_2[F_2, C_2]$ of the definitions $c \triangleq a * b$ and $d \triangleq b * c$, respectively, as shown in Figure 7, we have $F_1 = \{a, b\}$, $C_1 = \{c\}$, $F_2 = \{b, c\}$ and $C_2 = \{d\}$. Since $Clk_1 \cap Clk_2 = \{c, a, b\} \cap \{d, b, c\} = \{b, c\} \subseteq F_2$, therefore $\mathcal{M}_2 \text{ obst}_{F_2} \mathcal{M}_1$. As shown in Figure 7, at any state of $\mathcal{M}_2$, no matter what the clock behaviour of $\mathcal{M}_1$ is, one of the two conditions in Definition 5.12 holds. At the state t_2 of $\mathcal{M}_2$, for instance, for a clock behaviour $\{c, a, b\}$ of $\mathcal{M}_1$, there is a transition $\langle t_2, t_3 \rangle$ s.t. $\xi_2(t_3) \cap F_2 = \{c, a, b\} \cap F_2$ holds.

Since for any observers its set of atomic propositions is empty, the parallel composition of an observer and a CTSA is totally determined by their clock behaviours. Therefore, it is not hard to see that the relation defined in Definition 5.14 actually implies the observation relation defined in Definition 5.11. We have the following proposition.

PROPOSITION 5.5. Let $\mathcal{M} = \langle \Pi, Clk, Q, I, \theta, \xi, \mathcal{T}, \mathcal{E} \rangle$ be a CTSA and $\mathcal{M}'[F', C'] = \langle \Pi', Clk', Q', I', \theta', \xi', \mathcal{T}', \mathcal{E}' \rangle$ be an observer. Thus,

$$\text{if } \mathcal{M}' \text{ obst } \mathcal{M}, \text{ then } \mathcal{M}' \text{ obs } \mathcal{M}.$$

For example, for the two observers $\mathcal{M}_1$ and $\mathcal{M}_2$ shown in Figure 7, we have $\mathcal{M}_2 \text{ obs } \mathcal{M}_1$, so $\mathcal{M}_2$ can proceed given any trace of $\mathcal{M}_1$. For example, for a trace $\pi = s_1 s_4 s_2 s_3 s_3 \dots$ of $\mathcal{M}_1$, we have a trace $\pi_0 = \langle s_1, t_4 \rangle \langle s_4, t_4 \rangle \langle s_2, t_1 \rangle \langle s_3, t_3 \rangle \langle s_3, t_3 \rangle \langle s_3, t_3 \rangle \dots$ s.t. $\pi_0|_{\mathcal{M}_1} = \pi$.

PROOF OF PROPOSITION 5.5. To prove $\mathcal{M}' \text{ obs } \mathcal{M}$, it is enough to show that along any trace $\pi \in \text{Trace}(\mathcal{M})$,

- (1) for $\pi[0]$, there exists an initial state x_0 of $\mathcal{M}'$ s.t. $\langle \pi[0], x_0 \rangle$ is an initial state of $\mathcal{M} \parallel \mathcal{M}'$;
- (2) for any transition $\langle q, q' \rangle$ and any state x of $\mathcal{M}'$ with $\langle q, x \rangle$ being a state of $\mathcal{M} \parallel \mathcal{M}'$, there exists a transition $\langle \langle q, x \rangle, s \rangle$ of $\mathcal{M} \parallel \mathcal{M}'$.

(1): Let $X_0 = \xi(\pi[0]) \cap \text{Clk}'$, and since $\text{Clk} \cap \text{Clk}' \subseteq F'$, it is clear that $X_0 \subseteq F'$. By Definition 5.12, there exists an initial state $x_0 \in I'$ s.t. $\xi'(x_0) \cap F' = X_0$. Since $\Pi' = \emptyset$, next we only need to show that $\text{Clk} \cap \xi'(x_0) = \text{Clk}' \cap \xi(\pi[0])$.

On one hand, for any $t \in \text{Clk} \cap \xi'(x_0)$, since $\xi'(x_0) \subseteq \text{Clk}'$, we have $t \in \text{Clk} \cap \xi'(x_0) \subseteq \text{Clk} \cap \text{Clk}' \subseteq F'$, so $t \in X_0 = \xi(\pi[0]) \cap \text{Clk}'$. On the other hand, for any $t \in \text{Clk}' \cap \xi(\pi[0])$, $t \in X_0 = \xi'(x_0) \cap F'$. Since $\xi(\pi[0]) \subseteq \text{Clk}$, we have $t \in \text{Clk}$, and therefore $t \in \text{Clk} \cap \xi'(x_0)$.

According to Definition 5.4, $\langle \pi[0], x_0 \rangle$ is an initial state of $\mathcal{M} \parallel \mathcal{M}'$.

(2): Let $X = F' \cap \xi(q')$, and since $X \subseteq F'$, in $\mathcal{M}'[F', C']$, by Definition 5.12, for any state $x \in Q'$, there exists a transition $\langle x, x' \rangle$ s.t. $\xi'(x') \cap F' = X$. Next we show that $\text{Clk} \cap \xi'(x') = \text{Clk}' \cap \xi(q')$.

On one hand, since $\xi'(x') \subseteq \text{Clk}'$, therefore $\text{Clk} \cap \xi'(x') = \text{Clk} \cap \text{Clk}' \cap \xi'(x') \subseteq F' \cap \xi'(x') = X$. Because $X = F' \cap \xi(q')$ and $\xi(q') \subseteq \text{Clk}$, therefore $X \subseteq \text{Clk}$. From $F' \cap \xi'(x') = X$, we have $X \subseteq \text{Clk} \cap \xi'(x')$. Thus, $\text{Clk} \cap \xi'(x') = X$. On the other hand, since $\xi(q') \subseteq \text{Clk}$, therefore $\text{Clk}' \cap \xi(q') = \text{Clk}' \cap \text{Clk} \cap \xi(q') \subseteq F' \cap \xi(q') = X$. But $F' \subseteq \text{Clk}'$, we immediately get $X \subseteq \text{Clk}' \cap \xi(q')$. Thus, $\text{Clk}' \cap \xi(q') = X$.

According to Definition 5.4, $\langle q', x' \rangle$ is a state of $\mathcal{M} \parallel \mathcal{M}'$, hence $\langle \langle q, x \rangle, \langle q', x' \rangle \rangle$ is a transition of $\mathcal{M} \parallel \mathcal{M}'$. $\square$

When composing two observers by parallel composition, we need to know what the composed CTSA, as an observer, is like, and what are its sets of free clocks and constrained clocks. This is important when we try to make the parallel composition of the CTSA's of several clock constraints.

For example, consider the two observers $\mathcal{M}_1[F_1, C_1]$ and $\mathcal{M}_2[F_2, C_2]$ of Figure 7 where we have seen that $\mathcal{M}_2 \text{ obst}_{F_2} \mathcal{M}_1$. It is easy to see that their parallel composition $(\mathcal{M}_1 \parallel \mathcal{M}_2)[F, C]$ (shown in Figure 7) is an observer with $F = F_1 \cup F_2 - (C_1 \cap F_2) = \{a, b\} \cup \{b, c\} - (\{c\} \cap \{b, c\}) = \{a, b\}$ and $C = C_1 \cup C_2 = \{c, d\}$.

We have the following proposition.

PROPOSITION 5.6. *Given two observers $\mathcal{M}_i[F_i, C_i] = \langle \Pi_i, \text{Clk}_i, Q_i, I_i, \theta_i, \xi_i, \mathcal{T}_i, \mathcal{E}_i \rangle$ ($i = 1, 2$), if $\mathcal{M}_2 \text{ obst}_{F_2} \mathcal{M}_1$, then the product $\mathcal{M}[F, C] = \mathcal{M}_1[F_1, C_1] \parallel \mathcal{M}_2[F_2, C_2]$ is an observer, with*

$$\begin{aligned} F &= (F_1 \cup F_2) - (C_1 \cap F_2) \\ &= (F_1 \cup F_2) - ((F_1 \cup F_2) \cap (C_1 \cup C_2)), \text{ and} \\ C &= C_1 \cup C_2. \end{aligned}$$

The proof of Proposition 5.6 is given in Appendix B. The main idea behind the proof is that for any clock behaviour $X \subseteq F$, we need to find two suitable clock behaviours (i.e., $\xi(x')$ and $\xi(y')$) in each of the observers $\mathcal{M}_1$ and $\mathcal{M}_2$, respectively, so that $\xi(\langle x', y' \rangle) = \xi(x') \cup \xi(y')$ is a suitable clock behaviour in $\mathcal{M}$ that observes X in the sense of Definition 5.12.

The result in Proposition 5.6 can be extended to the case for multiple observers of clock constraints, and thus we have the following proposition.

THEOREM 5.1. *Given a finite set of well-defined clock constraints Φ , we have that $\mathcal{M}[F, C] = \parallel_{Cn^+ \in \sim \Phi} \mathcal{M}^{Cn^+}[F_{Cn^+}, C_{Cn^+}]$ is an observer, with*

$$\begin{aligned} F &= \bigcup_{Cn^+ \in \sim \Phi} F_{Cn^+} - \left(\bigcup_{Cn^+ \in \sim \Phi} F_{Cn^+} \cap \bigcup_{Cn^+ \in \sim \Phi} C_{Cn^+} \right) \text{ and} \\ C &= \bigcup_{Cn^+ \in \sim \Phi} C_{Cn^+}. \end{aligned}$$

Let $A^{\sim \Phi}$ be the set of the CTSA's of all constraints in $\sim \Phi$. The proof can proceed by induction on the size of $A^{\sim \Phi}$. We omit it here, and one can refer to Appendix B for more details. The main idea behind the proof is that in the well-defined set $A^{\sim \Phi}$, we continuously compose the CTSA's

of constraints in a special order so that the composed CTSAs can be easily proved to have an observation relationship with the CTSAs we are about to compose.

Proposition 5.6 and Theorem 5.1 are necessary for proving Proposition 5.8 in Section 5.4.

5.4 Relation between Clock Constraints and Their CTSAs

After introducing the observation theory of CCSL in Section 5.3, we now turn to focus on analysis of the relation between clock constraints and their corresponding CTSAs. It forms the basis for the model checking theory proposed in Section 6.1. First we consider the case when no clock definitions are involved (Proposition 5.7), then we extend the result to the general case (Proposition 5.8).

PROPOSITION 5.7 (RELATION BETWEEN CLOCK RELATIONS AND THEIR CTSAs). *Given a clock schedule σ and a clock relation Cn ,*

$$\sigma \models Cn \text{ iff } \sigma|_{\mathcal{M}^{Cn}} \in \text{Sche}(\mathcal{M}^{Cn}) \text{ iff } \sigma|_{\mathcal{M}^{\sim Cn}} \notin \text{Sche}(\mathcal{M}^{\sim Cn}).$$

Proposition 5.7 is trivial according to the CTTA semantics of clock relations given in Table 3, by considering the CTSAs of all four types of clock relations ($c_1 \subseteq c_2$, $c_1 \# c_2$, $c_1 < c_2$ and $c_1 \leq c_2$) and their negations ($\sim(c_1 \subseteq c_2)$, $\sim(c_1 \# c_2)$, $\sim(c_1 < c_2)$ and $\sim(c_1 \leq c_2)$). We omit the details in this article.

When considering a satisfaction relation with a non-empty set of clock definitions as the condition (i.e., the satisfaction relation of the form $\sigma \models Cn, D$ (where $D \neq \emptyset$)), the situation is a bit complex, because the clocks in Cn may not be pure and may come from D . For any schedule σ that satisfies $\sigma \models Cn, D$, it not only relates to the schedules of $\mathcal{M}^{Cn}$ (and $\mathcal{M}^{\sim Cn}$) but also to those of $\|_{Cn \in D} \mathcal{M}^{Cn}$.

Based on the observation theory built in Section 5.3, first we introduce the notion of observation relation between schedules.

Definition 5.15 (Observation Relation between Schedules). *Given a schedule $\sigma = B_0 B_1 \dots B_i \dots$ of an observer $\mathcal{M}[F, C]$, for any schedule $\sigma' = B'_0 B'_1 \dots B'_i \dots$, σ observes σ' in $\mathcal{M}[F, C]$, abbreviated as $\sigma \text{ obs}_{\mathcal{M}[F, C]} \sigma'$ (or simply $\sigma \text{ obs } \sigma'$), is defined s.t.*

$$\text{for any } i \geq 0, B_i \cap F = B'_i \cap F.$$

Definition 5.15 says that at each time, the part of a clock behaviour of the schedule σ' that belongs to the set of free clocks of the observer $\mathcal{M}$ (i.e., $B'_i \cap F$) is observed by a clock behaviour of the schedule σ (i.e., B_i) in the sense of Definition 5.12.

For example, in the observer $\mathcal{M}_2$ of Figure 7, $\sigma = \emptyset \emptyset \{d, b\} \{d, b, c\} \{d, b, c\} \{d, b, c\} \dots$ is a schedule of $\mathcal{M}_2$. It is easy to see that the schedule σ observes the schedule $\sigma' = \{a\} \emptyset \{b\} \{c, a, b\} \{c, a, b\} \{c, a, b\} \dots$ of $\mathcal{M}_1$ (i.e., $\sigma \text{ obs}_{\mathcal{M}_2} \sigma'$). We have, for example, $\sigma[0] \cap F_2 = \emptyset \cap \{b, c\} = \emptyset = \{a\} \cap \{b, c\} = \sigma'[0] \cap F_2$, and $\sigma[3] \cap F_2 = \{d, b, c\} \cap \{b, c\} = \{b, c\} = \{c, a, b\} \cap \{b, c\} = \sigma'[3] \cap F_2$.

Given a clock behaviour B , we use $\text{pure}_{\Phi}(B)$ to denote the set of pure clocks of B that appear in the set of clock constraints Φ . When Φ is clear in the context, we often omit it and simply write $\text{pure}(B)$. We lift the function pure and the union operator $\cup$ to sequences of clock behaviours: for any schedule $\sigma_1 = B_1 B_2 \dots B_i \dots$ and $\sigma_2 = B'_1 B'_2 \dots B'_i \dots$, we define

$$\begin{aligned} \text{pure}(\sigma_1) &= \text{pure}(B_1) \text{pure}(B_2) \dots \text{pure}(B_i) \dots, \\ \sigma_1 \cup \sigma_2 &= (B_1 \cup B'_1)(B_2 \cup B'_2) \dots (B_i \cup B'_i) \dots \end{aligned}$$

PROPOSITION 5.8 (RELATION BETWEEN Cn AND $\mathcal{M}^{Cn}$). *Given a clock schedule σ , a clock relation Cn and a set of clock definitions D , Φ is a set of constraints s.t. $Cn \in \Phi$ and $\Phi^d = D$, then there exists a schedule $\sigma' \in \text{Sche}(\|_{Cn \in D} \mathcal{M}^{Cn})$ that satisfies $\sigma' \text{ obs } \sigma$, and moreover,*

$$\sigma \models Cn, \mathbf{D} \text{ iff } \sigma' \cup \text{pure}_{\Phi}(\sigma) \models Cn.$$

In Proposition 5.8, $\text{pure}_{\Phi}(\sigma)$ must be considered because there might exist some clocks in Φ but not in $\mathbf{D}$ so that they might not be included in σ' .

For example, let $\mathbf{D} = \{c \triangleq a * b, d \triangleq b + c\}$, $Cn = (c \leq e)$, let e be a pure clock in Φ , and let $\sigma = \{a, b\}\{a, e\}\{a, b\}\{a, e\}\{a, b\}\{a, e\} \dots$ be a schedule that satisfies $\sigma \models Cn, \mathbf{D}$. Then the schedule σ' in the observer $\mathcal{M}^{c \triangleq a * b} \parallel \mathcal{M}^{d \triangleq b + c}$ (which is exactly the observer $\mathcal{M}_1 \parallel \mathcal{M}_2$ shown in Figure 7) that satisfies $\sigma' \text{ obs } \sigma$ and $\sigma' \cup \text{pure}_{\Phi}(\sigma) \models Cn$ is $\sigma' = \{d, c, a, b\}\{a\}\{d, c, a, b\}\{a\}\{d, c, a, b\}\{a\} \dots$ e is the clock that is pure in σ but not in σ' .

The proof of Proposition 5.8 mainly relies on Lemma B.4 given in Appendix B.

PROOF OF PROPOSITION 5.8. Let $\sigma = B_0 B_1 \dots B_k \dots$, $\mathcal{M}[F, C] = \parallel_{Cn \in \mathbf{D}} \mathcal{M}^{Cn}$, and starting from the initial state of $\mathcal{M}$, by Lemma B.4 we can build a schedule $\sigma' = B'_0 B'_1 \dots B'_k \dots$ s.t. any B'_k ($k \geq 0$) satisfies $B'_k \cap F = B_k \cap F$, and for any non-pure clock d in $\mathbf{D}$, $B_k \models d, \mathbf{D}$ iff $d \in B'_k$. By Definition 5.15, we have $\sigma' \text{ obs } \sigma$. To see $\sigma' \cup \text{pure}(\sigma) \models Cn$, notice that for any clock c in Cn and $\mathbf{D}$, no matter whether it is a pure clock or not, there will always be that for any $k \geq 0$, $B_k \models c, \mathbf{D}$ iff $c \in B'_k \cup \text{pure}(B_k)$. Intuitively, this means that the schedule $\sigma' \cup \text{pure}(\sigma)$ exactly contains all clocks satisfied by σ at each instant. $\square$

As a corollary of Proposition 5.8, Theorem 5.2 forms the basis for model checking CCSL constraints in LTL_c (as seen in Section 6.1).

In the rest of the article, we use $F(\sim \Phi)$ and $C(\sim \Phi)$ to express the set of free clocks and the set of constrained clocks of all observers of the clock constraints in $\sim \Phi$ shown in Proposition 5.3, respectively.

THEOREM 5.2. *Given a CTS $\mathcal{M} = \langle \Pi, Clk, Q, I, \theta, \xi, \mathcal{T} \rangle$, a clock relation Cn and a set of clock constraints $\mathbf{D}$ with $F(\mathbf{D}) \subseteq Clk$ and $C(\mathbf{D}) \cap Clk = \emptyset$, let $\mathcal{M}' = \mathcal{M} \parallel (\parallel_{Cn \in \mathbf{D}} \mathcal{M}^{Cn})$, then for any schedule $\sigma \in \text{Sche}(\mathcal{M})$, there exists a schedule $\sigma' \in \text{Sche}(\mathcal{M}')$ s.t. $\sigma'|_{\mathcal{M}} = \sigma$. Moreover,*

$$\sigma \models Cn, \mathbf{D} \text{ iff } \sigma' \models Cn.$$

PROOF. Let $N[F, C] = \parallel_{Cn \in \mathbf{D}} \mathcal{M}^{Cn}$, let $\sigma = B_0 B_1 \dots B_k \dots$, and by Proposition 5.8, there exists a schedule $\sigma'' = B''_0 B''_1 \dots B''_k \dots$ s.t. $\sigma'' \in \text{Sche}(N)$ and $\sigma'' \text{ obs } \sigma$. Let $\sigma' = B'_0 B'_1 \dots B'_k \dots$, where $B'_k = B_k \cup B''_k$ for all $k \geq 0$. Clearly, $\sigma'|_{\mathcal{M}} = \sigma$. First we show that $\sigma' \in \text{Sche}(\mathcal{M}')$.

Since $\Pi_N = \emptyset$, by Definition 5.4, we only need to prove that $B_k \cap Clk_N = B'_k \cap Clk$ for any $k \geq 0$. On one hand, for any $t \in B_k \cap Clk_N$, because $C(\mathbf{D}) \cap Clk = C \cap Clk = \emptyset$, therefore $t \in F$. From the fact that $B_k \cap F = B''_k \cap F$ (since $\sigma'' \text{ obs } \sigma$), we have $t \in B''_k$. Thus, $t \in B'_k \cap B_k \subseteq B'_k \cap Clk$. On the other hand, for any $t \in B'_k \cap Clk$, since $t \in Clk$, therefore $t \in F$ due to the fact that $t \in Clk_N$, $F = F(\mathbf{D}) \subseteq Clk$ and $C \cap Clk = \emptyset$. Thus, $t \in B''_k \cap F = B_k \cap F \subseteq B_k$. Therefore, $t \in B_k \cap B''_k \subseteq B_k \cap Clk_N$.

To see $\sigma' \models Cn$ iff $\sigma \models Cn, \mathbf{D}$, we prove that in fact $\sigma' = \sigma'' \cup \text{pure}(\sigma)$, where $\sigma'' \cup \text{pure}(\sigma) \models Cn$ iff $\sigma \models Cn, \mathbf{D}$ by Proposition 5.8. It is enough to show $B_k = \text{pure}(B_k)$, which is obvious because $C \cap Clk = \emptyset$ —that is, any non-pure clocks of $\mathbf{D}$ are not in Clk . $\square$

6 Verification of LTL_c /CCSL Specifications

In this section, we propose a model checking approach for verification of LTL_c /CCSL specifications. Similar to the model checking technique for LTL [10], the main idea behind our method is to transform both LTL_c formulae and CCSL specifications into a type of transition system, then make reachability analysis on the product of these transition systems and a system model. Since not all CCSL specifications are bounded (see Section 5.2), it is not always possible to verify LTL_c /CCSL

specifications in a full model checking procedure. Thus, in this work, we only consider the verification of a subset of $LTL_c/CCSL$ where the set of CCSL constraints of an LTL_c formula is bounded.

The goal of this section is to prove the main theorem (Theorem 6.2), which states that how a system model satisfies a $LTL_c/CCSL$ specification can be related to the satisfaction problem of the composition of their CTSA. Based on the observation relation between a system model and the CTSA of a set of clock constraints as analyzed in Sections 5.1, 5.2, 5.3 and 5.4, in Section 6.1 we prove Theorem 6.2. Based on Theorem 6.2, we propose a model checking algorithm—Algorithm 2—for verifying $LTL_c/CCSL$ specifications. Because the expressiveness of LTL_c stays in LTL, the transformation from LTL_c formulae into CTSA can be realized by the transformation from LTL formulae into Büchi automata, while the transformation from CCSL constraints into CTSA is straightforward. Finally, we make an analysis of the complexity of our model checking algorithm in Section 6.2.

6.1 Model Checking for Bounded $LTL_c/CCSL$ Specifications

We discuss the model checking of $LTL_c/CCSL$ specifications based on Section 5. The main idea behind it is that given a CTS and an $LTL_c/CCSL$ specification, we transform the $LTL_c/CCSL$ specification into CTSA, then make reachability analysis on the parallel composition of the CTS and these CTSA. The transformation from LTL_c formulae to CTSA can be obtained through the transformation from LTL formulae to Büchi automata, since LTL_c formulae in fact can be expressed by LTL formulae according to Section 2.2. The transformation from CCSL specifications to CTSA is straightforward according to the CTSA semantics of CCSL constraints (Tables 3 and 4) and the composition between CTSA (Definition 5.4). As shown in Section 6.1.1, since CTSA/CTSA can be seen as OTS by taking clocks as special atomic propositions, the model checking of $LTL_c/CCSL$ specifications can actually be realized by reachability analysis on OTS.

In Section 6.1.1, we first discuss the transformation from LTL_c to CTSA. Then, in Section 6.1.2, we discuss the automata-based model checking for $LTL_c/CCSL$.

6.1.1 Transformation from LTL_c Formulae to CTSA. As indicated in Section 2.2, the expressiveness of LTL_c actually lies in that of LTL since the occurrences of each CCSL clock c can be expressed as a special atomic proposition named P_c . Similarly, a CTS and a CTSA can be taken as an OTS and a Büchi automaton (introduced in Appendix A), respectively, by encoding the occurrences of each clock c at each state as an atomic proposition P_c .

As indicated in Definition 5.7, CTSA are a special case of CTSA; therefore, in the following, we only define the relation between CTSA and Büchi automata. The relation between CTSA and OTS can be defined in a similar way.

Definition 6.1 (CTSA as a Büchi Automaton). For any CTSA $\mathcal{M} = \langle \Pi, Clk, Q, I, \theta, \xi, \mathcal{T}, \mathcal{E} \rangle$, its corresponding Büchi automaton, denoted as $\mathcal{M}^\triangleright$, is defined as

$$\mathcal{M}^\triangleright = \langle \Pi^\triangleright, Q, I, \theta^\triangleright, \mathcal{T}, \mathcal{E} \rangle,$$

where $\Pi^\triangleright = \Pi \cup \{P_c \mid c \in Clk\}$, P_c is the atomic proposition indicating the occurrences of clock c in $\mathcal{M}$, which is distinguished from the propositions in Π . $\theta^\triangleright$ is defined as

$$\text{for any } q \in Q, \theta^\triangleright(q) = \theta(q) \cup \{P_c \mid c \in \xi(q)\}.$$

From Definition 6.1, it is easy to see that each CTSA corresponds to exactly one Büchi automaton since they share the same set of states and transitions. Hence, if a Büchi automaton $\mathcal{A}$ has a CTSA $\mathcal{M}$ s.t. $\mathcal{M}^\triangleright = \mathcal{A}$, we also denote $\mathcal{M}$ as $\mathcal{A}^{-\triangleright}$ (i.e., $\mathcal{M} = \mathcal{A}^{-\triangleright}$).

Similarly, a CTS $\mathcal{M}$ exactly corresponds to an OTS, denoted as $\mathcal{M}^\triangleright$. We omit its formal definition here, which is similar to Definition 6.1. If an OTS $\mathcal{A}$ has a CTS $\mathcal{M}$ s.t. $\mathcal{M}^\triangleright = \mathcal{A}$, we also denote $\mathcal{M}$ as $\mathcal{A}^{-\triangleright}$ (i.e., $\mathcal{M} = \mathcal{A}^{-\triangleright}$).

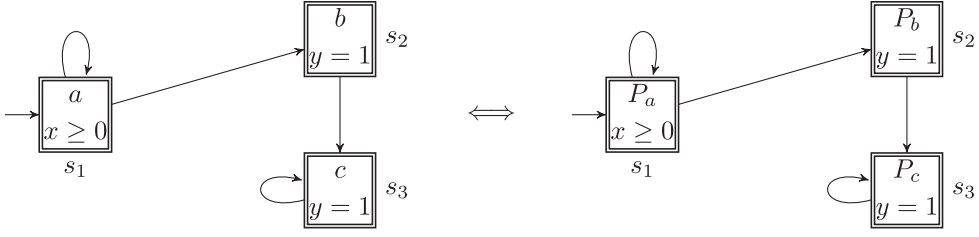


Fig. 9. An example of a CTSA and its corresponding Büchi automaton.

Obviously, a CTSA $\mathcal{M}$ and its corresponding Büchi automaton $\mathcal{M}^\triangleright$ share the same set of traces (i.e., $\text{Trace}(\mathcal{M}) = \text{Trace}(\mathcal{M}^\triangleright)$). The behaviour of $\mathcal{M}$ is equivalent to the behaviour of $\mathcal{M}^\triangleright$ in the sense that any clock behaviour in the CTSA is taken as a set of atomic propositions as defined in Definition 6.1.

As an example, Figure 9 shows the CTS of Figure 6, $\mathcal{M} = \langle \Pi, \text{Clk}, Q, I, \theta, \xi, \mathcal{T}, \mathcal{E} \rangle$ (on the left), and its corresponding Büchi automaton, $\mathcal{M}^\triangleright = \langle \Pi^\triangleright, Q, I, \theta^\triangleright, \mathcal{T}, \mathcal{E} \rangle$ (on the right). Different from Figure 6, here the CTS is taken as a special CTSA with all states as accepting states: $\mathcal{E} = \{s_1, s_2, s_3\}$. Two automata $\mathcal{M}$ and $\mathcal{M}^\triangleright$ share the same set of states, the same set of initial states, the same set of transitions and the same set of accepting states. The set of atomic propositions $\Pi^\triangleright = \{x \geq 0, y = 1, P_a, P_b, P_c\}$. The function $\theta^\triangleright$ is defined s.t. $\theta^\triangleright(s_1) = \{x \geq 0, P_a\}$, $\theta^\triangleright(s_2) = \{y = 1, P_b\}$, $\theta^\triangleright(s_3) = \{y = 1, P_c\}$.

Comparing Definition 2.6 and Definition 6.1, we see that for any trace π of $\mathcal{M} = \langle \Pi, \text{Clk}, Q, I, \theta, \xi, \mathcal{T} \rangle$ with $\langle \pi, \theta, \xi \rangle$, the configuration of π , $\theta^\xi = \theta^\triangleright$ holds. Recall that in Section 2.2, we have already shown that an LTL_c formula ϕ can be encoded as an LTL formula $\phi^\triangleright$ in Definition 2.5, and their relation is given in Theorem 2.1. Based on Theorem 2.1, we now discuss the relation between LTL_c formulae and their encoded LTL formulae in terms of the satisfaction relations w.r.t. transition systems.

PROPOSITION 6.1 (RELATION BETWEEN LTL_c FORMULAE AND LTL FORMULAE). *Given a CTSA $\mathcal{M}$ and an LTL_c formula ϕ , for any trace $\pi \in \text{Trace}(\mathcal{M}) = \text{Trace}(\mathcal{M}^\triangleright)$,*

$$\pi \models_{\mathcal{M}} \phi \text{ iff } \pi \models_{\mathcal{M}^\triangleright} \phi^\triangleright.$$

The relation $\pi \models_{\mathcal{N}} \psi$ is given in Appendix A for an OTS $\mathcal{N}$ and an LTL formula ψ .

Proposition 6.1 can be easily proved from Theorem 2.1.

THE PROOF OF PROPOSITION 6.1. Let $\mathcal{M} = \langle \Pi, Q, I, \theta, \xi, \mathcal{T} \rangle$, then $\mathcal{M}^\triangleright = \langle \Pi^\triangleright, Q, I, \theta^\triangleright, \mathcal{T} \rangle$ according to Definition 6.1. By Theorem 2.1 and the fact that $\theta^\triangleright = \theta^\xi$ analyzed previously, we have $\pi \models_{\mathcal{M}} \phi \text{ iff } \pi \models_{\theta, \xi} \phi \text{ iff } \pi \models_{\theta^\xi} \phi^\triangleright \text{ iff } \pi \models_{\theta^\triangleright} \phi^\triangleright \text{ iff } \pi \models_{\mathcal{M}^\triangleright} \phi^\triangleright$. $\square$

With the support of Definition 2.5, Definition 6.1 and Proposition 6.1, we now show that the transformation from an LTL_c formula into a CTSA can be performed through the transformation from its corresponding LTL formula into a Büchi automaton [10]. The next theorem states the relation between an LTL_c formula and its corresponding CTSA.

THEOREM 6.1. *Given an LTL_c formula ϕ , let $\mathfrak{B}(\phi^\triangleright)$ denote the Büchi automaton of the LTL formula $\phi^\triangleright$, then $\mathcal{M}(\mathcal{E}) = \mathfrak{B}(\phi^\triangleright)^{-\triangleright}$ is the CTSA of ϕ —that is, for any trace π , it satisfies that*

$$\pi \models_{\mathcal{M}(\mathcal{E})} \phi \text{ iff } \pi \in \text{Trace}(\mathcal{M}(\mathcal{E})).$$

ALGORITHM 1: Transformation from an LTL_c Formula to CTSA

```

1 function  $LTL_c2CTSA(\phi)$ ;
   Input : an  $LTL_c$  formula  $\phi$ 
   Output: the corresponding CTSA  $\mathcal{M}(\mathcal{E}) = \langle \Pi, Clk, Q, I, \theta, \xi, \mathcal{T}, \mathcal{E} \rangle$  of  $\phi$ 
2 encode  $\phi$  into its corresponding LTL formula  $\phi^\nabla$ ;
3  $A := \text{LTL2Büchi}(\phi^\nabla)$ ; /* run the standard procedure of transforming an LTL formula into a
   Büchi automaton */
4  $\mathcal{M}(\mathcal{E}) := A^{-\nabla}$ ; /* return the corresponding CBA */

```

PROOF. Let $\mathcal{B}(\phi^\nabla) = \langle \Pi, Q, I, \theta, \mathcal{T}, \mathcal{E} \rangle$. Because for any $c \in C(\phi)$, P_c is a subformula of ϕ^∇ , according to the standard procedure of the transformation from LTL formula into Büchi automaton [10], $A = \{P_c \mid c \in C(\phi)\}$ is the set of all atomic propositions that correspond to clocks in Π . According to Definition 6.1, the corresponding CTSA of $\mathcal{B}(\phi^\nabla)$ is

$$\mathcal{M}(\mathcal{E}) = \langle \Pi^{-\nabla}, Clk, Q, I, \theta^{-\nabla}, \xi, \mathcal{T}, \mathcal{E} \rangle,$$

where $\Pi^{-\nabla} = \Pi - A$, $Clk = \{c \mid P_c \in A\} = C(\phi)$, $\theta^{-\nabla}(q) = \theta(q) - \{P_c \mid P_c \in \theta(q)\}$ for any $q \in Q$, and $\xi(q) = \{c \mid P_c \in \theta(q)\}$ for any $q \in Q$.

For any trace π , by Proposition 6.1, we get that $\pi \models_{\mathcal{M}} \phi$ iff $\pi \models_{\mathcal{B}(\phi^\nabla)} \phi^\nabla$. By the relation between an LTL formula and its corresponding Büchi automaton [10], there is $\pi \models_{\mathcal{B}(\phi^\nabla)} \phi^\nabla$ iff $\pi \in \text{Trace}(\mathcal{B}(\phi^\nabla))$. Since $\text{Trace}(\mathcal{B}(\phi^\nabla)) = \text{Trace}(\mathcal{M}(\mathcal{E}))$, we have $\pi \in \text{Trace}(\mathcal{B}(\phi^\nabla))$ iff $\pi \in \text{Trace}(\mathcal{M}(\mathcal{E}))$. $\square$

Based on Theorem 6.1, we build a main procedure for the transformation from an LTL_c formula into a CTSA, presented as Algorithm 1. In Algorithm 1, we assume the standard transformation procedure LTL2Büchi from LTL formulae to Büchi automaton in the model checking theory of LTL, whose implementation is outside the scope of this work. One can refer to the work of Baier and Katoen [10] for more details.

6.1.2 Automata-Based Model Checking for LTL_c /CCSL Specifications. The verification problem for LTL_c /CCSL specifications asks that given a CTS model $\mathcal{M}$ and an LTL_c /CCSL specification $\langle \phi, \Phi \rangle$, whether all traces starting from the initial states of $\mathcal{M}$ satisfy $\langle \phi, \Phi \rangle$. The model checking technique mainly follows the idea behind the model checking theory of LTL [10]. We propose transforming the negation of the LTL_c formula ϕ into a CTSA, named $\mathcal{M}^{-\phi}$ (Section 6.1.1), encoding the negations of all clock relations Φ^r as a CTSA, named $\mathcal{M}^{\sim \Phi^r}$, and encoding the set of all clock definitions Φ^d as a CTSA, named $\mathcal{M}^{\Phi^d}$. The validity of the specification $\langle \phi, \Phi \rangle$ w.r.t. the CTS $\mathcal{M}$ can then be verified by checking whether there exists a trace starting from the initial states of $\mathcal{M}$ and accepted by the parallel composition of the CTS $\mathcal{M}$, the CTSA $\mathcal{M}^{-\phi}$, $\mathcal{M}^{\sim \Phi^r}$ and $\mathcal{M}^{\Phi^d}$ s.t. it is reachable to some accepting state of $\mathcal{M}^{-\phi}$ and $\mathcal{M}^{\sim \Phi^r}$.

Before introducing the main theorem—Theorem 6.2—of the model checking theory, we first show that there always exists an observation relation between a system model $\mathcal{M}$ and the CTSA $\mathcal{M}^{-\phi}$ of the negation of an LTL_c formula ϕ .

As stated at the beginning of Section 5.3, in the model checking theory of LTL, there always exists an observation relation between a system model and the Büchi automaton of the negation of an LTL formula that specifies the system model. This is guaranteed by the transformation from LTL formulae into Büchi automata [10]. Formally, given an OTS $\mathcal{N} = \langle \Pi, Q, I, \theta, \mathcal{T} \rangle$ and an LTL formula ϕ , we have

$$\mathcal{B}(\neg \phi) \text{ obs } \mathcal{N},$$

where according to Theorem 6.1, $\mathcal{B}(\neg\phi)$ denotes the Büchi automaton of the formula $\neg\phi$.

From the relation between CTSAs and Büchi automata described in Definition 6.1 and the fact that the transformation from LTL_c formulae into CTSAs is performed through the transformation procedure from LTL formulae into Büchi automata, it is not hard to prove that there exists an observation relation between a CTS and the CTSA of the negation of an LTL_c formula.

In the rest of the article, we use $\Pi(\phi)$ to represent the set of all atomic propositions of a formula ϕ .

PROPOSITION 6.2. *Given a CTS $\mathcal{M} = \langle \Pi, Clk, Q, I, \theta, \xi, \mathcal{T} \rangle$ and an LTL_c formula ϕ with $\Pi(\phi) \subseteq \Pi$, let $\mathcal{M}^{-\phi} = \mathcal{B}((\neg\phi)^\mathcal{D})^{-\mathcal{D}}$. We have*

$$\mathcal{M}^{-\phi} \text{ obs } \mathcal{M}.$$

The proof of Proposition 6.2 is given in Appendix B.

THEOREM 6.2. *Given a CTS $\mathcal{M} = \langle \Pi, Clk, Q, I, \theta, \xi, \mathcal{T} \rangle$ and an $LTL_c/CCSL$ specification $\langle \phi, \Phi \rangle$ with $\Pi(\phi) \subseteq \Pi$, $C(\phi) \subseteq Clk$, $F(\sim\Phi) \subseteq Clk$ and $C(\sim\Phi) \cap Clk = \emptyset$, let $\mathcal{M}' = \mathcal{M} \parallel \mathcal{M}^{-\phi} \parallel \mathcal{M}^\Phi$, where $\mathcal{M}^{-\phi} = \mathcal{B}((\neg\phi)^\mathcal{D})^{-\mathcal{D}}$, $\mathcal{M}^\Phi = \mathcal{M}^{\Phi^d} \parallel \mathcal{M}^{\sim\Phi^r}$, $\mathcal{M}^{\Phi^d} = \parallel_{Cn \in \Phi^d} \mathcal{M}^{Cn}$, and $\mathcal{M}^{\sim\Phi^r} = \parallel_{Cn^+ \in \sim\Phi^r} \mathcal{M}^{Cn^+}$, then for any $q \in I$,*

$$q \models_{\mathcal{M}} \langle \phi, \Phi \rangle \text{ iff}$$

$$\text{there is no trace } \pi \in \text{Trace}(\mathcal{M}') \text{ s.t. } \pi[0]_{\mathcal{M}} = q \text{ and } \pi|_{\mathcal{M}^{-\phi} \parallel \mathcal{M}^{\sim\Phi^r}} \in \text{Trace}(\mathcal{M}^{-\phi} \parallel \mathcal{M}^{\sim\Phi^r}).$$

Theorem 6.2 says that to check whether a CTS $\mathcal{M}$ satisfies an $LTL_c/CCSL$ specification $\langle \phi, \Phi \rangle$, we make the product $\mathcal{M}'$ of the CTS $\mathcal{M}$ and the CTSAs $\mathcal{M}^{-\phi}$ and $\mathcal{M}^\Phi$. $\mathcal{M}^{-\phi}$ is the CTSA of the negation of the formula ϕ , and $\mathcal{M}^\Phi$ is the product of the CTSAs of the negations of all clock relations in Φ and the CTSAs of all clock definitions in Φ . After making this product, we check that there exists no trace π of $\mathcal{M}'$ s.t. (1) it starts from an initial state of $\mathcal{M}$, and (2) its projection on $\mathcal{M}^{-\phi} \parallel \mathcal{M}^{\sim\Phi^r}$ is accepted by $\mathcal{M}^{-\phi} \parallel \mathcal{M}^{\sim\Phi^r}$, which means that π passes one of the accepting states of $\mathcal{M}^{-\phi}$ or any $\mathcal{M}^{Cn^+}$ (where $Cn^+ \in (\sim\Phi^r)$) for infinitely many times.

The conditions $\Pi(\phi) \subseteq \Pi$ and $C(\phi) \subseteq Clk$ say that the formula ϕ can only faithfully capture the property in the system model $\mathcal{M}$, and cannot contain any atomic propositions or clocks not in $\mathcal{M}$. The condition $F(\sim\Phi) \subseteq Clk$ says that all clock constraints in Φ can only observe the clock behaviour in $\mathcal{M}$ and should not contain any free clocks not in $\mathcal{M}$. The condition $C(\sim\Phi) \cap Clk = \emptyset$ means that all clocks defined in Φ must be new clocks not appearing in $\mathcal{M}$.

The proof of Theorem 6.2 is given next, mainly based on the observation theory of CCSL and the observation relation between a system model and the composition of the CTSAs of its $LTL_c/CCSL$ specification. Although a process of reasoning looks tedious, the idea of the proof is simple, just by contradiction on both directions. The dependence relations between different lemmas, propositions and theorems are given in Figure 5.

THE PROOF OF THEOREM 6.2. Due to the associativity of the parallel composition (Proposition 5.1), we can write $\mathcal{M}'$ as

$$\mathcal{M}' = \mathcal{M} \parallel \mathcal{M}^{-\phi} \parallel (\mathcal{M}^{\Phi^d} \parallel \mathcal{M}^{\sim\Phi^r}) = (\mathcal{M} \parallel \mathcal{M}^{\Phi^d}) \parallel (\mathcal{M}^{-\phi} \parallel \mathcal{M}^{\sim\Phi^r}).$$

Let $\mathcal{M}'' = \mathcal{M} \parallel \mathcal{M}^{\Phi^d}$, $\mathcal{M}^{(3)} = \mathcal{M}^{-\phi} \parallel \mathcal{M}^{\sim\Phi^r}$, and we proceed the proof by contradictions on both directions.

($\Rightarrow$): Suppose that there exists a trace $\pi_0 \in \text{Trace}(\mathcal{M}')$ s.t. $\pi_0|_{\mathcal{M}^{(3)}} \in \text{Trace}(\mathcal{M}^{(3)})$ and $\pi_0[0]_{\mathcal{M}} = q$, we now prove that it contradicts $q \models_{\mathcal{M}} \langle \phi, \Phi \rangle$. Since $\pi_0|_{\mathcal{M}^{(3)}} \in \text{Trace}(\mathcal{M}^{(3)})$, by the definition of CTSA (Definition 5.7) and the product of CTSA defined in Section 5.1, we know that trace $\pi_0|_{\mathcal{M}^{(3)}}$ must pass one of accepting states of $\mathcal{M}^{-\phi}$ or any $\mathcal{M}^{Cn^+}$ (where $Cn^+ \in \sim\Phi^r$) for infinite times. Thus, we need to consider two situations:

- (1) there exists a constraint $Cn_0^+ \in \sim \Phi^r$ s.t. $\pi_0|_{\mathcal{M}^{Cn_0^+}} \in \text{Trace}(\mathcal{M}^{Cn_0^+})$;
- (2) $\pi_0|_{\mathcal{M}^{-\phi}} \in \text{Trace}(\mathcal{M}^{-\phi})$.

For situation (1), we first prove that for any $Cn^+ \in \sim \Phi^r$, $\sigma^{\pi_0|_{\mathcal{M}''}}|_{\mathcal{M}^{Cn^+}} = \sigma^{\pi_0|_{\mathcal{M}^{Cn^+}}}$ holds. Considering any state u of trace π_0 , we see that $u|_{\mathcal{M}''}$ is a state of trace $\pi_0|_{\mathcal{M}''}$, and $u|_{\mathcal{M}^{Cn^+}}$ is a state of trace $\pi_0|_{\mathcal{M}^{Cn^+}}$. Since $C(Cn^+) \subseteq F(\sim \Phi^r) \subseteq F(\sim \Phi) \subseteq \text{Clk} \subseteq \text{Clk}_{\mathcal{M}''}$, $\text{Clk}_{\mathcal{M}^{Cn^+}} = C(Cn^+) \subseteq \text{Clk}_{\mathcal{M}''}$ holds. For any CTS $N = N_1 \parallel N_2$ and any state v of N , since $\text{Clk}_{N_2} \cap \xi_{N_1}(v|_{N_1}) = \text{Clk}_{N_1} \cap \xi_{N_2}(v|_{N_2})$ (by Definition 5.4), we can have

$$\begin{aligned}
 \xi_{N_1}(v|_{N_1}) &= \xi_{N_1}(v|_{N_1}) \cup (\xi_{N_1}(v|_{N_1}) \cap \text{Clk}_{N_2}) \\
 &= \xi_{N_1}(v|_{N_1}) \cup (\xi_{N_2}(v|_{N_2}) \cap \text{Clk}_{N_1}) \\
 &= (\xi_{N_1}(v|_{N_1}) \cap \text{Clk}_{N_1}) \cup (\xi_{N_2}(v|_{N_2}) \cap \text{Clk}_{N_1}) \\
 &= (\xi_{N_1}(v|_{N_1}) \cup \xi_{N_2}(v|_{N_2})) \cap \text{Clk}_{N_1} \\
 &= \xi_N(v) \cap \text{Clk}_{N_1}.
 \end{aligned} \tag{1}$$

Therefore, we have $\xi_{\mathcal{M}''}(u|_{\mathcal{M}''}) = \xi_{\mathcal{M}'}(u) \cap \text{Clk}_{\mathcal{M}''}$ and $\xi_{\mathcal{M}^{Cn^+}}(u|_{\mathcal{M}^{Cn^+}}) = \xi_{\mathcal{M}'}(u) \cap \text{Clk}_{\mathcal{M}^{Cn^+}}$. Thus, $\xi_{\mathcal{M}''}(u|_{\mathcal{M}''}) \cap \text{Clk}_{\mathcal{M}^{Cn^+}} = \xi_{\mathcal{M}'}(u) \cap \text{Clk}_{\mathcal{M}''} \cap \text{Clk}_{\mathcal{M}^{Cn^+}} = \xi_{\mathcal{M}'}(u) \cap \text{Clk}_{\mathcal{M}^{Cn^+}} = \xi_{\mathcal{M}^{Cn^+}}(u|_{\mathcal{M}^{Cn^+}})$. Hence, $\sigma^{\pi_0|_{\mathcal{M}''}}|_{\mathcal{M}^{Cn^+}} = \sigma^{\pi_0|_{\mathcal{M}^{Cn^+}}}$.

Since $\pi_0|_{\mathcal{M}^{Cn_0^+}} \in \text{Trace}(\mathcal{M}^{Cn_0^+})$, $\sigma^{\pi_0|_{\mathcal{M}^{Cn_0^+}}} \in \text{Sche}(\mathcal{M}^{Cn_0^+})$. From the preceding, we know that $\sigma^{\pi_0|_{\mathcal{M}''}}|_{\mathcal{M}^{Cn_0^+}} = \sigma^{\pi_0|_{\mathcal{M}^{Cn_0^+}}} \in \text{Sche}(\mathcal{M}^{Cn_0^+})$. Let $Cn_0^+ = \sim \text{Rel}_0$, where $\text{Rel}_0 \in \Phi^r$, and by Proposition 5.7, we know that $\sigma^{\pi_0|_{\mathcal{M}''}} \not\models \text{Rel}_0$. Let $\pi = \pi_0|_{\mathcal{M}''}|_{\mathcal{M}}$, for any state w of the trace $\pi_0|_{\mathcal{M}''}$, and by Equation (1), we have $\xi_{\mathcal{M}''}(w) \cap \text{Clk} = \xi(w|_{\mathcal{M}})$, hence we get $\sigma^{\pi_0|_{\mathcal{M}''}}|_{\mathcal{M}} = \sigma^\pi$. Because $\sigma^{\pi_0|_{\mathcal{M}''}} \not\models \text{Rel}_0$, by Theorem 5.2 we obtain $\sigma^\pi \not\models \text{Rel}_0, \Phi^d$. Notice that $\pi[0] = q$; this contradicts the assumption that $q \models_{\mathcal{M}} \langle \phi, \Phi \rangle$.

For situation (2), by Theorem 6.1, we have $\pi_0|_{\mathcal{M}^{-\phi}} \models_{\mathcal{M}^{-\phi}} \neg\phi$, which is $\pi_0|_{\mathcal{M}^{-\phi}} \not\models_{\mathcal{M}^{-\phi}} \phi$. Let x be any state of the trace π_0 , and we show that

- (A) for any $p \in \Pi(\phi)$, $p \in \theta(x|_{\mathcal{M}})$ iff $p \in \theta_{\mathcal{M}^{-\phi}}(x|_{\mathcal{M}^{-\phi}})$;
- (B) for any $c \in C(\neg\phi)$, $c \in \xi(x|_{\mathcal{M}})$ iff $c \in \xi_{\mathcal{M}^{-\phi}}(x|_{\mathcal{M}^{-\phi}})$.

For (A), from $\theta_{\mathcal{M}^{-\phi}}(x|_{\mathcal{M}^{-\phi}}) \cap \Pi = \theta(x|_{\mathcal{M}}) \cap \Pi_{\mathcal{M}^{-\phi}}$, $\Pi(\phi) \subseteq \Pi$ and the fact that $\Pi(\phi) = \Pi_{\mathcal{M}^{-\phi}}$, we have $p \in \theta_{\mathcal{M}^{-\phi}}(x|_{\mathcal{M}^{-\phi}})$ iff $p \in \theta_{\mathcal{M}^{-\phi}}(x|_{\mathcal{M}^{-\phi}}) \cap \Pi$ iff $p \in \theta(x|_{\mathcal{M}}) \cap \Pi_{\mathcal{M}^{-\phi}}$ iff $p \in \theta(x|_{\mathcal{M}})$. For (B), from $\xi_{\mathcal{M}^{-\phi}}(x|_{\mathcal{M}^{-\phi}}) \cap \text{Clk} = \xi(x|_{\mathcal{M}}) \cap \text{Clk}_{\mathcal{M}^{-\phi}}$, $C(\neg\phi) = C(\phi) \subseteq \text{Clk}$ and the fact that $C(\phi) = \text{Clk}_{\mathcal{M}^{-\phi}}$, we have $c \in \xi_{\mathcal{M}^{-\phi}}(x|_{\mathcal{M}^{-\phi}})$ iff $c \in \xi_{\mathcal{M}^{-\phi}}(x|_{\mathcal{M}^{-\phi}}) \cap \text{Clk}$ iff $c \in \xi(x|_{\mathcal{M}}) \cap \text{Clk}_{\mathcal{M}^{-\phi}}$ iff $c \in \xi(x|_{\mathcal{M}})$.

By (A) and (B), it is not hard to see that $\pi_0|_{\mathcal{M}} \not\models_{\mathcal{M}} \phi$ because $\pi_0|_{\mathcal{M}^{-\phi}} \not\models_{\mathcal{M}^{-\phi}} \phi$. This contradicts the assumption that $q \models_{\mathcal{M}} \langle \phi, \Phi \rangle$.

($\Leftarrow$): Suppose $q \not\models_{\mathcal{M}} \langle \phi, \Phi \rangle$, then there exists a trace $\pi_0 \in \text{Trace}(q)$ s.t.

- (i) $\sigma^{\pi_0} \not\models \Phi$, or
- (ii) $\pi_0 \not\models_{\mathcal{M}} \phi$.

For situation (i), we know there exists a clock relation $\text{Rel}_0 \in \Phi^r$ s.t. $\sigma^{\pi_0} \not\models \text{Rel}_0, \Phi^d$. Let $Cn_0^+ = \sim \text{Rel}_0$, so $\sigma^{\pi_0} \models Cn_0^+, \Phi^d$. Since $C(\sim \Phi) \cap \text{Clk} = \emptyset$, $C(\Phi^d) \cap \text{Clk} = \emptyset$ hold, thus we have $\text{Clk}_{\mathcal{M}^{\Phi^d}} \cap \text{Clk} = (F(\Phi^d) \cup C(\Phi^d)) \cap \text{Clk} = F(\Phi^d) \cap \text{Clk} \subseteq F(\Phi^d)$. Therefore, $\mathcal{M}^{\Phi^d} \text{ obst } \mathcal{M}$, and we have $\mathcal{M}^{\Phi^d} \text{ obs } \mathcal{M}$ by Proposition 5.5. Thus, there is a trace $\pi_1 \in \text{Trace}(\mathcal{M}'')$ s.t. $\pi_1|_{\mathcal{M}} = \pi_0$. Let w be a state of $\mathcal{M}''$, and by Equation (1), we have that $\xi_{\mathcal{M}''}(w) \cap \text{Clk} = \xi(w|_{\mathcal{M}})$. Therefore, $\sigma^{\pi_1}|_{\mathcal{M}} = \sigma^{\pi_0}$. According to Theorem 5.2, we have $\sigma^{\pi_1} \not\models \text{Rel}_0$, so by Proposition 5.7, there is $\sigma^{\pi_1}|_{\mathcal{M}^{Cn_0^+}} \in \text{Sche}(\mathcal{M}^{Cn_0^+})$.

ALGORITHM 2: Model Checking of $LTL_c/CCSL$ Specifications

```

1 function MC_LTLc-CCSL ( $\mathcal{M}, \langle \phi, \Phi \rangle$ );           /* where  $\Pi(\phi) \subseteq \Pi$ ,  $C(\phi) \subseteq Clk$ ,  $F(\Phi) \subseteq Clk$  and
    $C(\Phi) \cap Clk = \emptyset$ ;  $\Phi$  is bounded */
   Input : a CTS  $\mathcal{M}$ , an  $LTL_c/CCSL$  specification  $\langle \phi, \Phi \rangle$ 
   Output: a Boolean value  $Res$  ( $Res = true$  iff  $\mathcal{M}$  satisfies  $\langle \phi, \Phi \rangle$ )
2  $\mathcal{M}^{\neg\phi} := \underline{LTL_c2CTSA}(\neg\phi)$ ;                /* encoding formula  $\neg\phi$  into a CTSA */
3  $\mathcal{M}^\Phi := (\|_{Cn \in \Phi^d} \mathcal{M}^{Cn}) \parallel (\|_{Cn^+ \in \sim\Phi^r} \mathcal{M}^{Cn^+})$ ; /* encoding clock-constraint set  $\Phi$  into a CTSA
   */
4  $\mathcal{M}'(\mathcal{E}') := \mathcal{M} \parallel \mathcal{M}^{\neg\phi} \parallel \mathcal{M}^\Phi$ ;
5  $Res := \underline{RB\_Alys}(I_{\mathcal{M}'}, \mathcal{M}', \mathcal{E}')$ ; /* run the standard procedure for reachability analysis on
   the product  $\mathcal{M}'$  */

```

Now we show that $\mathcal{M}^{\neg\phi} \text{ obs } \mathcal{M}''$. Since $\mathcal{M}^{\neg\phi} \text{ obs } \mathcal{M}$ (by Proposition 6.2) and the fact that $\Pi(\mathcal{M}^{\Phi^d}) = \emptyset$, it is enough to only consider their clock behaviours and prove that for any trace $\pi \in \text{Trace}(\mathcal{M}'')$,

- (I) for $\pi[0]$, there exists an initial state x_0 of $\mathcal{M}^{\neg\phi}$ s.t. $\xi_{\mathcal{M}''}(\pi[0]) \cap Clk_{\mathcal{M}^{\neg\phi}} = \xi_{\mathcal{M}^{\neg\phi}}(x_0) \cap Clk_{\mathcal{M}''}$;
- (II) for any transition $\langle q, q' \rangle$ of the trace π and any state x of $\mathcal{M}^{\neg\phi}$ s.t. $\langle q, x \rangle$ is a state of $\mathcal{M}'' \parallel \mathcal{M}^{\neg\phi}$, there exists a state $\langle q', x' \rangle$ of $\mathcal{M}'' \parallel \mathcal{M}^{\neg\phi}$ s.t. $\xi_{\mathcal{M}''}(q') \cap Clk_{\mathcal{M}^{\neg\phi}} = \xi_{\mathcal{M}^{\neg\phi}}(x') \cap Clk_{\mathcal{M}''}$.

We only prove (I), as the proof of (II) is similar. Let $\pi[0] = \langle y_1, y_2 \rangle \in \mathcal{M} \parallel \mathcal{M}^{\Phi^d}$, and since $\mathcal{M}^{\neg\phi} \text{ obs } \mathcal{M}$, there exists an initial state x_0 of $\mathcal{M}^{\neg\phi}$ s.t. $\xi(y_1) \cap Clk_{\mathcal{M}^{\neg\phi}} = \xi_{\mathcal{M}^{\neg\phi}}(x_0) \cap Clk$. Now we prove that $\xi_{\mathcal{M}''}(\pi[0]) \cap Clk_{\mathcal{M}^{\neg\phi}} = \xi_{\mathcal{M}^{\neg\phi}}(x_0) \cap Clk_{\mathcal{M}''}$. On one hand, for any $t \in \xi_{\mathcal{M}''}(\pi[0]) \cap Clk_{\mathcal{M}^{\neg\phi}}$, $t \in (\xi(y_1) \cup \xi_{\mathcal{M}^{\Phi^d}}(y_2)) \cap Clk_{\mathcal{M}^{\neg\phi}}$. If $t \in \xi_{\mathcal{M}^{\Phi^d}}(y_2)$, since $Clk_{\mathcal{M}^{\neg\phi}} = C(\phi) \subseteq Clk$, $t \in \xi_{\mathcal{M}^{\Phi^d}}(y_2) \cap Clk$ holds. By $\mathcal{M}^{\Phi^d} \text{ obs } \mathcal{M}$ proved earlier, we have $t \in \xi_{\mathcal{M}^{\Phi^d}}(y_2) \cap Clk = \xi(y_1) \cap Clk_{\mathcal{M}^{\Phi^d}}$. Thus, $t \in \xi(y_1)$. Therefore, $t \in \xi(y_1) \cap Clk_{\mathcal{M}^{\neg\phi}} = \xi_{\mathcal{M}^{\neg\phi}}(x_0) \cap Clk \subseteq \xi_{\mathcal{M}^{\neg\phi}}(x_0) \cap Clk_{\mathcal{M}''}$. On the other hand, for any $t \in \xi_{\mathcal{M}^{\neg\phi}}(x_0) \cap Clk_{\mathcal{M}''}$, $t \in \xi_{\mathcal{M}^{\neg\phi}}(x_0) \subseteq Clk_{\mathcal{M}^{\neg\phi}} = C(\phi) \subseteq Clk$. Thus, we have $t \in \xi_{\mathcal{M}^{\neg\phi}}(x_0) \cap Clk = \xi(y_1) \cap Clk_{\mathcal{M}^{\neg\phi}} \subseteq (\xi(y_1) \cup \xi_{\mathcal{M}^{\Phi^d}}(y_2)) \cap Clk_{\mathcal{M}^{\neg\phi}} = \xi_{\mathcal{M}''}(\pi[0]) \cap Clk_{\mathcal{M}^{\neg\phi}}$.

By $\mathcal{M}^{\neg\phi} \text{ obs } \mathcal{M}''$, there exists a trace $\pi_2 \in \text{Trace}(\mathcal{M}'' \parallel \mathcal{M}^{\neg\phi})$ s.t. $\pi_2|_{\mathcal{M}''} = \pi_1$. Since $Clk_{\mathcal{M}^{\neg\Phi^r}} \cap Clk_{\mathcal{M}'' \parallel \mathcal{M}^{\neg\phi}} = F(\sim\Phi^r) \cap Clk_{\mathcal{M}'' \parallel \mathcal{M}^{\neg\phi}} \subseteq F(\sim\Phi^r)$, by Definition 5.14 we have $\mathcal{M}^{\neg\Phi^r} \text{ obst } \mathcal{M}'' \parallel \mathcal{M}^{\neg\phi}$, and therefore $\mathcal{M}^{\neg\Phi^r} \text{ obs } \mathcal{M}'' \parallel \mathcal{M}^{\neg\phi}$ by Proposition 5.5. Hence, there is a trace $\pi_3 \in \text{Trace}(\mathcal{M}')$ s.t. $\pi_3|_{\mathcal{M}'' \parallel \mathcal{M}^{\neg\phi}} = \pi_2$. But π_3 contradicts the assumption.

For situation (ii), we immediately obtain $\pi_0 \models_{\mathcal{M}} \neg\phi$, so $\pi_0 \in \text{Trace}(\mathcal{M}^{\neg\phi})$ by Theorem 6.1. Since $\Pi(\phi) \subseteq Clk$, we have $\pi_0 \in \text{Trace}(\mathcal{M} \parallel \mathcal{M}^{\neg\phi})$. From $C(\sim\Phi) \subseteq Clk = \emptyset$ and $C(\phi) \subseteq Clk$, we have $Clk_{\mathcal{M}^{\Phi^d} \parallel \mathcal{M}^{\neg\Phi^r}} \cap Clk_{\mathcal{M} \parallel \mathcal{M}^{\neg\phi}} = (F(\sim\Phi) \cup C(\sim\Phi)) \cap (Clk \cup C(\phi)) = (F(\sim\Phi) \cup C(\sim\Phi)) \cap Clk \subseteq F(\sim\Phi)$. By Definition 5.14, we get that $\mathcal{M}^{\Phi^d} \parallel \mathcal{M}^{\neg\Phi^r} \text{ obst } \mathcal{M} \parallel \mathcal{M}^{\neg\phi}$, so $\mathcal{M}^{\Phi^d} \parallel \mathcal{M}^{\neg\Phi^r} \text{ obs } \mathcal{M} \parallel \mathcal{M}^{\neg\phi}$ by Proposition 5.5. Therefore, there exists a trace $\pi_1 \in \text{Trace}(\mathcal{M}')$ s.t. $\pi_1|_{\mathcal{M} \parallel \mathcal{M}^{\neg\phi}} = \pi_0$, which violates the assumption. $\square$

Based on Theorem 6.2, we propose a main procedure—Algorithm 2—for model checking a subset of $LTL_c/CCSL$ specifications in which the set of CCSL constraints of an LTL_c formula is bounded. Algorithm 2 mainly follows the idea behind the algorithm for model checking LTL formulae [10]. The transformation from a clock definition Cn to its CTS $\mathcal{M}^{Cn}$, or from the negation of a clock relation Cn to its CTSA $\mathcal{M}^{\neg Cn}$, is straightforward according to Table 4. The reachability analysis is realized by procedure $\underline{RB_Alys}(I_{\mathcal{M}'}, \mathcal{M}', \mathcal{E}')$, which checks whether there exists a trace of $\mathcal{M}'$ starting from one of the states in $I_{\mathcal{M}'}$ (i.e., the initial states) s.t. it passing one of states in $\mathcal{E}'$ for infinitely many times. For each state in $\mathcal{E}'$ that is reachable to a state in $I_{\mathcal{M}'}$, procedure $\underline{RB_Alys}$ searches whether there exists a cycle that contains the state.

We omit the details of procedure RB_Alys in this article. According to Definition 6.1, CTSs/CTSAs and OTS/Büchi automata share the same set of states and transitions; therefore, procedure RB_Alys is similar to the standard procedure for reachability analysis on the product of an OTS and a Büchi automaton of an LTL formula, which can be found in relevant works, such as that of Baier and Katoen [10].

6.2 Model Checking Complexity

The worst-case time complexity of Algorithm 2 is

$$O(2^{O(|\phi|)} \cdot |\mathcal{M}| \cdot N(\Phi)),$$

where ϕ , $\mathcal{M}$ and Φ are given in Algorithm 2. The length of the formula ϕ — $|\phi|$ —is defined as the number of logical operators that appear in ϕ . We use $|\mathcal{A}| = |\mathcal{Q}_{\mathcal{A}}| + |\mathcal{T}_{\mathcal{A}}|$ to express the number of states and transitions of any CTS/CTSA $\mathcal{A}$. $N(\Phi) = |\mathcal{M}^\Phi|$, where $\mathcal{M}^\Phi$, given in Algorithm 2, represents the product of the CTSA of all constraints in $\sim \Phi$. It is easy to see that $N(\Phi)$ not only depends on the size of Φ but also on the clock expressions of Φ themselves. For example, the CTSA of constraints $\Phi_1 = \{c_1 < c, c \triangleq c_1 \$1, c < c_2, c_2 < c_1\}$ has several states and transitions that are much smaller than that of the CTSA of constraints $\Phi_2 = \{c_1 < c, c \triangleq c_1 \$1000, c < c_2, c_2 < c_1\}$, even though $|\Phi_1| = |\Phi_2|$. The cause of the difference is from the clock definitions $c \triangleq c_1 \$1$ and $c \triangleq c_1 \$1000$.

The analysis of the complexity of Algorithm 2 is given as follows, where the symbols ϕ , $\mathcal{M}$, $\mathcal{M}'$ and Φ are given in Algorithm 2.

In procedure LTL_c2CTSA of Algorithm 1, according to Baier and Katoen [10], the complexity of procedure LTL2Büchi for transforming LTL formulae into Büchi automata is $2^{O(|\phi^\diamond|)} \cdot 2^{O(|\phi^\diamond|)}$ is also the upper bound of the number of states and transitions of the Büchi automaton of $\phi^\diamond$: $\mathfrak{B}(\phi^\diamond)$. Since $\mathfrak{B}(\phi^\diamond)$ and the CTSA of ϕ (i.e., $\mathfrak{B}(\phi^\diamond)^{-\diamond}$) share the same set of states and transitions according to Definition 6.1, $2^{O(|\phi^\diamond|)}$ is also the upper bound of the number of states and transitions of $\mathfrak{B}(\phi^\diamond)^{-\diamond}$. However, it is not hard to see that $O(|\phi^\diamond|) = O(|\phi|)$ according to the definition of the transformation $\diamond$ in Definition 2.5. Therefore, $|\mathcal{M}^{-\phi}|$ lies in $2^{O(|\phi|)}$. Hence, the upper bound of $|\mathcal{M}'|$ is $O(2^{O(|\phi|)} \cdot |\mathcal{M}| \cdot N(\Phi))$. According to the model checking theory of LTL [10], the worst-case time and space complexity of procedure RB_Alys(I, TS, S) is linear in $|TS|$ (i.e., $O(|TS|)$). Thus, in Algorithm 2, the complexity of procedure RB_Alys($I_{\mathcal{M}'}, \mathcal{M}', \mathcal{E}_0$) is $O(|\mathcal{M}'|) = O(2^{O(|\phi|)} \cdot |\mathcal{M}| \cdot N(\Phi))$, which obviously is also the complexity of Algorithm 2.

7 Related Work and Further Discussion

7.1 Temporal Logics for Real-Time Systems

Variations of temporal logics have been proposed for specifying real-time systems. Here we choose some of the most representative temporal logics among them which are closely related to our proposed LTL_c, and we make comparisons between them.

MTL [6] is a logic proposed in early years for specifying properties that involve explicit time. It extends propositional temporal logic with bounded temporal operators like $\diamond_{<t}$ to express a temporal property with a real-time constraint. For example, formula $\diamond_{<10}p$ means that p must be true within 10 time units. In MTL, each state is mapped to a non-negative integer indicating an explicit real time. The time model of MTL is thus a discrete one, described as a sequence of non-negative integers. The expression bound to a temporal operator (e.g., $\diamond_{<t}$) in fact plays the role of a filter that extracts the states whose associated time values are less than t . The bound operator (e.g., $\square_c$) in LTL_c is inspired by this idea. Compared to MTL, LTL_c only involves logical time instants indicated by clocks rather than explicit time values. LTL_c/CCSL specifications only

focus on properties related to logical clocks, not properties related to time values. Our LTL_c can naturally express a partial-ordered time model indicated by multiple clocks defined as primitives in LTL_c , whereas the time model of MTL is totally ordered.

MITL [5] is based on a continuous time model where each state corresponds to an interval of time (e.g., (1.2, 2.4)) during which the system is in that state. Behaviour of a system is thus modelled as a sequence of states whose corresponding timed intervals are joined end to end. Any temporal operator in MITL is bound to a timed interval (e.g., $\Diamond_{(1.2, 2.4]}$), confining the operators to only working on the states during this period. For example, formula $\Diamond_{(1.2, 2.4]} \phi$ means that ϕ will hold from time 1.2 to time 2.4 in the future. Different from MTL, the time domain of MITL is real. Compared to MITL, the time model of LTL_c is discrete, which is more suitable for specifying discrete event systems which are based on logical clocks.

Signal temporal logic (STL) [34] is for specifying continuous-timed behaviours, such as the trajectories of hybrid systems or the behaviour of physical plants (often modelled as differential equations). The syntax of STL is the same as a subset of MITL called *bounded MITL* but defined in a different semantics. In STL, it is considered whether a continuous real-valued signal satisfies a STL formula rather than a sequence of timed states as in MITL. For example, for a continuous real-valued signal $x(t) = \sin(\omega t)$, consider checking whether x satisfies a formula $\Diamond_{[3, 5]}(x > 0.7)$. Due to the undecidability of dealing with the continuity, STL is mainly used for monitoring systems. Recently, some variations of STL have been proposed for specifying different types of real-time properties. For example, Donzé et al. [18] present a time-frequency logic for specifying time-frequency properties; Brim et al. [14] propose an extension of STL for expressing various dynamic aspects of oscillations. Like MITL, STL mainly focus on specifying continuous-timed behaviours, whereas LTL_c is specially designed for specifying properties related to logical clocks.

The logics discussed previously, possessing either discrete or continuous time models, are all based on a total-order time model. Next we introduce some logics that can describe a partial-order time model as LTL_c . Compared to them, in LTL_c , we use CCSL to further specify the logical constraints between clocks and combine LTL_c and CCSL for specification purpose.

Liu and Orgun [32] proposed a **temporal logic with clocks (TLC)** for specifying reactive systems with multiple granularities of time (i.e., multiple clocks). TLC extends the traditional LTL by associating each formula to a clock name, like $(\Box \phi)_c$, indicating that the formula $\Box \phi$ is defined at the projected trace, whose states are exactly those where clock c ticks, of a global trace. By projection, it means that for those states not in the projected trace, formula $\Box \phi$ is not defined. This is different from LTL_c , where a formula, say $\Box_c \phi$, is defined on a global state. Clock c just plays the role of a “filter”, that when talking about satisfaction excludes those states where c does not tick on a global trace. That is why in LTL_c , we only need to combine clocks to atomic propositions and temporal operators. In LTL_c , we want a formula to describe a trace filtered by different clocks w.r.t. to a global trace, rather than projecting the global trace into a subtrace with a clock, which can be further projected by other clocks. For example, in formula $\Box_{c_1}(\phi_1 \rightarrow \Diamond_{c_2} \phi_2)$ describing Property (2) in Figure 2, c_2 does not have to be a subclock of clock c_1 (i.e., in the projective trace by clock c_1). c_1 and c_2 just perform different filtering on the global trace. Instead of a model checking approach as we develop for LTL_c , TLC was mainly developed for verification in a theorem-proving approach [33].

Eisner et al. [19] made a natural extension on LTL with a clock operator @ and defined a logic called LTL° . Similar to TLC, in LTL° , each LTL formula ϕ can be bound to a clock c by $\phi@c$, expressing that formula ϕ holds along a trace where clock c ticks. Clocks only play the role of path projections in LTL° . This is a bit different from LTL_c , where a formula also relates to the current global state. For example, $X_c \phi$ means that at the next state where c ticks w.r.t. the current global state, ϕ holds. It is equivalent to an LTL° formula $X(\phi@c)$, which is related to the next global state.

However, formula $(X\phi)@c$ means that at the second state where clock c ticks, ϕ holds, which is equivalent to the LTL_c formula $(tt_c \wedge X_c\phi) \vee (\neg tt_c \wedge X_cX_c\phi)$. Despite this different design of clock operators, LTL_c and LTL° have the same expressiveness, since they are both as expressive as LTL (cf. the work of Eisner et al. [19] for the encoding from LTL° to LTL). The advantages of LTL° compared to LTL_c are that in LTL° , one can directly express the projection of a finite clock and can easily express normal LTL properties without clocks by simply not using the binding '@'. Although LTL_c can do the same things in an ugly way, one of motivations to design LTL_c , however, is that an LTL_c property should be always bound to some local clock(s), because in a multi-clock system, all temporal properties are restricted by local times. And as far as reactive systems are concerned, clocks are always infinite, and for us there is currently no need to consider finite clocks.

7.2 Languages for Modelling/Specifying Multi-Clock Systems

The notion of multi-clock systems arises from the concepts of logical clocks and synchronous relations that first appeared in early synchronous programming languages such as Signal [11], Lustre [15] and Esterel [12] developed in the 1980s. It was also implied in the later proposed computation model, the “tagged signal model” [30], which is the foundation of many modelling/specification languages such as Ptolemy [39] and its variants (like Forsyde or ModHel’X) for systems of heterogeneous MoCs. In the following, we introduce some modelling/specifying languages for multi-clock systems from different aspects and at different abstraction levels. We mainly discuss two languages—MARTE/CCSL and Signal—which are closely related to the design of our proposed $LTL_c/CCSL$. We make comparisons between them and $LTL_c/CCSL$.

MARTE [38] is a profile of UML for modelling and analyzing real-time and embedded systems. It extends the time model of UML, which only considers physical time, to a multiform time model that considers both physical and logical time [8]. In MARTE, the clock is the basic element to model different forms of time. A clock can be referred to different time bases. For example, an ideal clock is referred to a dense time domain to model continuous physical time; a logical clock is referred to a discrete time domain to model a sequence of events. MARTE supports modelling heterogeneous systems, where different notions of time of different components can be referred to different clocks. These clocks can be coordinated by CCSL, a companion language of MARTE which was invented to facilitate expressing complex logical constraints between clocks. In our work, we use CCSL to specify the logical constraints between clocks in LTL_c formulae. Compared to MARTE/CCSL, $LTL_c/CCSL$ is not intended to be a modelling language. It rather focuses on high-level specification and only supports modelling logical time.

Signal is a synchronous programming language that was designed for high-level specification of reactive systems [11]. It is a dataflow language whose semantics is based on equations of vectors. Each vector represents an infinite sequence of data and is associated to a clock that captures its occurrence at each instant. An equation of vectors captures a constraint (or say a relation) between the clocks and the values of the vectors. In Signal, a system is made of a set of equations, where different clocks (of vectors) are constrained through the equations in a loose way. Unlike other synchronous languages such as Lustre [15] and Esterel [12] which require the schedule of clocks to be unique, in Signal, the behaviour of a clock at each instant is non-deterministic and thus clocks form a partial-ordered time model. Signal has been widely used in academic and industry communities with the support of a powerful tool [2]. Compared to Signal, the semantics of $LTL_c/CCSL$ is based on automata rather than a set of mathematical equations. $LTL_c/CCSL$ separates the concerns of the clocks and values of data, whereas in Signal, clock constraints are indicated by the equations themselves. It is interesting to compare their expressiveness, which could be one of our future works.

Other dataflow languages such as Ptolemy and Giotto also reflect the multi-clock feature. Ptolemy [39] is a modelling language that supports heterogeneous modelling/specification of embedded systems. It is based on an actor model—a dataflow-style model that supports different types of MoCs which model both continuous and discrete dynamics. Ptolemy supports a type of dataflow models called *synchronous-reactive models* [31]. Like synchronous languages such as Signal, it supports the synchronous execution mechanism in which the executions of events take no time and several events can occur simultaneously. Another type of dataflow models in Ptolemy called *dataflow models of computation* [31] focus on the dependency between the flows of data but ignore discussing the relations between different flows. Giotto [23] is similar to the synchronous-reactive model of Ptolemy, but the executions of events take time instead of being instantaneous and simultaneous. Each event is triggered at the ticks of some clock but is also associated with a logical execution time. Giotto mainly focuses on the real-time constraints of embedded systems rather than the logical time constraints considered in Signal.

Comparing all of the preceding languages with $LTL_c/CCSL$, none of them stresses the multi-clock feature in a model of temporal logics.

8 Conclusion and Future Work

In this article, we mainly proposed a specification language $LTL_c/CCSL$ for specifying and verifying multi-clock systems, in which different components are based on local clocks that forms a partially ordered time model. We first built the syntax and semantics of LTL_c and introduced an adaptive semantics of $CCSL$ independently, then combined them together to obtain $LTL_c/CCSL$. After defining $LTL_c/CCSL$, we proposed a model checking approach to verify $LTL_c/CCSL$ specifications based on the model checking theory of LTL and the observation theory of CCSL. Additionally, we illustrated how $LTL_c/CCSL$ can be used to specify local temporal properties of a multi-clock system through an example.

We mainly consider our future work in the following three aspects:

- (1) The next step of our work is to apply $LTL_c/CCSL$ in specifying and verifying multi-clock systems in practice, such as Signal programs, Ptolemy models, and MARTE models. Relevant verification prototype tools can be developed for $LTL_c/CCSL$ according to the specific application domains. We also consider implementing $LTL_c/CCSL$ in famous model checking tools like Spin [24] or Maude [16], or consider developing a translator from $LTL_c/CCSL$ to the languages accepted by these tools.
- (2) In this work, we only consider logical clocks. We also consider to extend LTL_c with the *ideal clock* in CCSL, to also capture physical time in a multi-clock system. The ideal clock in CCSL models the global continuous physical time. Based on the ideal clock, we can define a discrete clock, whose ticks are tagged with physical time values, using an operator called *DiscretizedBy* [35]. For example, the expression $c \triangleq \text{idealClock DiscretizedBy } 1s$ means to define a clock c that ticks every 1second. Since *idealClock* is continuous, CTS/CTSA is no longer enough for describing the semantics of the extended LTL_c . A timed version of CTS/CTSA would be required, which is based on timed automata [4].
- (3) Another future work would focus on the runtime verification of $LTL_c/CCSL$ specifications. To this end, we shall propose a finite-trace semantics for $LTL_c/CCSL$ and an $LTL_c/CCSL$ monitoring algorithm. A prototype tool can also be developed.

A Preliminaries of LTL and OTS

The syntax of LTL is given as follows:

$$\phi ::= p \mid \neg\phi \mid \phi \wedge \phi \mid X\phi \mid \phi U\phi,$$

where X , U are two temporal operators called *next* and *until*, respectively. Intuitively, $X\phi$ means that along a trace, the next state satisfies ϕ , whereas $\phi_1 U \phi_2$ means that along a trace, there exists a state satisfying ϕ_2 , and before that state all states satisfy ϕ_1 .

An ordinary configuration is a pair $\langle \pi, \theta \rangle$, where π is a trace defined in Definition 2.3 and $\theta : S(\pi) \rightarrow 2^\Pi$ is a mapping from each state of the trace to a set of atomic propositions just defined in Definition 2.3.

Given an LTL formula ϕ , for any configuration $\langle \pi, \theta \rangle$, the trace semantics of ϕ , denoted by $\pi \models_\theta \phi$, is defined as follows:

- (1) $\pi \models_\theta p$ iff $p \in \theta(\pi[0])$.
- (2) $\pi \models_\theta \neg\phi$ iff $\pi \not\models_\theta \phi$.
- (3) $\pi \models_\theta \phi_1 \wedge \phi_2$ iff $\pi \models_\theta \phi_1$ and $\pi \models_\theta \phi_2$.
- (4) $\pi \models_\theta X\phi$ iff $\pi[1, \infty] \models_\theta \phi$.
- (5) $\pi \models_\theta \phi_1 U \phi_2$ iff there exists an $i \geq 0$ s.t. $\pi[i, \infty] \models_\theta \phi_2$, and for all $0 \leq j < i$, $\pi[j, \infty] \models_\theta \phi_1$.

An OTS is a 5-tuple $\mathcal{N} = \langle \Pi, Q, I, \theta, \mathcal{T} \rangle$, where all components Π , Q , I , θ and $\mathcal{T}$ are defined just as in Definition 5.1. $\text{Trace}(\mathcal{N})$, $\pi[i]$, $\pi[0, i]$, $\pi[i, \infty]$, $\pi[e]$ and $\text{Trace}(q)$ are defined just as their counterparts in the CTS in Section 5.1.

A Büchi automaton is an OTS with a set of accepting states. It can be defined as a 6-tuple $\mathcal{N}(\mathcal{E}) = \langle \Pi, Q, I, \theta, \mathcal{T}, \mathcal{E} \rangle$, where $\mathcal{E} \subseteq Q$. A trace π accepted by $\mathcal{N}(\mathcal{E})$ is defined similarly as in the CTSA in Definition 5.7.

OTSs can be seen as a special type of Büchi automata where all states are accepting states.

Given an OTS $\mathcal{N} = \langle \Pi, Q, I, \theta, \mathcal{T} \rangle$ and an LTL formula ϕ , for any trace $\pi \in \text{Trace}(\mathcal{N})$, the satisfaction relation between $\mathcal{N}$ and ϕ , denoted by $\pi \models_{\mathcal{N}} \phi$, can be defined as

$$\pi \models_{\mathcal{N}} \phi \text{ iff } \pi \models_\theta \phi.$$

For any state $q \in Q$, the semantics of the LTL formula ϕ : $q \models_{\mathcal{N}} \phi$, is defined s.t.

$$\text{for any } \pi \in \text{Trace}(q), \pi \models_{\mathcal{N}} \phi.$$

B Proofs of some Lemmas, Propositions and Theorems

Figure 5 shows the dependence relations between the main lemmas, propositions and theorems in this article.

THE PROOF OF PROPOSITION 5.1. Let $N_1 = (\mathcal{M}_1 \parallel \mathcal{M}_2) \parallel \mathcal{M}_3 = \langle \Pi^1, Clk^1, Q^1, I^1, \theta^1, \xi^1, \mathcal{T}^1 \rangle$ and $N_2 = \mathcal{M}_1 \parallel (\mathcal{M}_2 \parallel \mathcal{M}_3) = \langle \Pi^2, Clk^2, Q^2, I^2, \theta^2, \xi^2, \mathcal{T}^2 \rangle$; obviously, $I^1 = I^2$, $Clk^1 = Clk^2$, $I^1 = I^2$, $\theta^1 = \theta^2$ and $\xi^1 = \xi^2$ can be easily proved provided that $Q^1 = Q^2$. Therefore, it is sufficient to only show that $Q^1 = Q^2$ and $\mathcal{T}^1 = \mathcal{T}^2$, which are equivalent to prove that for any state q ,

- (1) q is an initial state of N_1 iff q is an initial state of N_2 ;
- (2) if q is in both N_1 and N_2 , then $\langle q, q' \rangle$ is a transition of N_1 iff $\langle q, q' \rangle$ is a transition of N_2 .

In the following proof of (1) and (2), we only consider the condition involving clock behaviours and ignore the corresponding condition involving atomic propositions since it is similar.

(1): For any initial state u of N_1 , let $u = \langle g, u_3 \rangle$ with g being a state of $\mathcal{M}_1 \parallel \mathcal{M}_2$ and u_3 being a state of $\mathcal{M}_3$. By Definition 5.4, we have

$$Clk_{\mathcal{M}_1 \parallel \mathcal{M}_2} \cap \xi_{\mathcal{M}_3}(u_3) = Clk_{\mathcal{M}_3} \cap \xi_{\mathcal{M}_1 \parallel \mathcal{M}_2}(g). \quad (2)$$

Since $g \in \mathcal{M}_1 \parallel \mathcal{M}_2$, let $g = \langle u_1, u_2 \rangle$ with u_1 being a state of $\mathcal{M}_1$ and u_2 being a state of $\mathcal{M}_2$. Again, by Definition 5.4, we have

$$Clk_{\mathcal{M}_1} \cap \xi_{\mathcal{M}_2}(u_2) = Clk_{\mathcal{M}_2} \cap \xi_{\mathcal{M}_1}(u_1). \quad (3)$$

By Equations (2) and (3), now we show that

- (a) $Clk_{\mathcal{M}_3} \cap \xi_{\mathcal{M}_2}(u_2) = Clk_{\mathcal{M}_2} \cap \xi_{\mathcal{M}_3}(u_3)$ and
- (b) $Clk_{\mathcal{M}_2 \parallel \mathcal{M}_3} \cap \xi_{\mathcal{M}_1}(u_1) = Clk_{\mathcal{M}_1} \cap \xi_{\mathcal{M}_2 \parallel \mathcal{M}_3}(\langle u_2, u_3 \rangle)$.

For (a), on one hand, for any $t \in Clk_{\mathcal{M}_3} \cap \xi_{\mathcal{M}_2}(u_2)$, we have $t \in \xi_{\mathcal{M}_2}(u_2) \subseteq \xi_{\mathcal{M}_1}(u_1) \cup \xi_{\mathcal{M}_2}(u_2) = \xi_{\mathcal{M}_1 \parallel \mathcal{M}_2}(g)$, hence $t \in Clk_{\mathcal{M}_3} \cap \xi_{\mathcal{M}_1 \parallel \mathcal{M}_2}(g)$. By Equation (2), we know that $t \in \xi_{\mathcal{M}_3}(u_3)$ and $t \in Clk_{\mathcal{M}_1 \parallel \mathcal{M}_2} = Clk_{\mathcal{M}_1} \cup Clk_{\mathcal{M}_2}$. If $t \in Clk_{\mathcal{M}_2}$, then we immediately get $t \in Clk_{\mathcal{M}_2} \cap \xi_{\mathcal{M}_3}(u_3)$. If $t \in Clk_{\mathcal{M}_1}$, then $t \in Clk_{\mathcal{M}_1} \cap \xi_{\mathcal{M}_2}(u_2)$, and by Equation (3), we still have $t \in Clk_{\mathcal{M}_2}$. On the other hand, for any $t \in Clk_{\mathcal{M}_2} \cap \xi_{\mathcal{M}_3}(u_3)$, we have $t \in Clk_{\mathcal{M}_1 \parallel \mathcal{M}_2} \cap \xi_{\mathcal{M}_3}(u_3)$. By Equation (2), we can get $t \in Clk_{\mathcal{M}_3} \cap \xi_{\mathcal{M}_1 \parallel \mathcal{M}_2}(g)$. Thus, $t \in \xi_{\mathcal{M}_1}(u_1) \cup \xi_{\mathcal{M}_2}(u_2)$. If $t \in \xi_{\mathcal{M}_2}(u_2)$, we immediately get $t \in Clk_{\mathcal{M}_3} \cap \xi_{\mathcal{M}_2}(u_2)$. If $t \in \xi_{\mathcal{M}_1}(u_1)$, by Equation (3), we also have $t \in \xi_{\mathcal{M}_2}$.

For (b), on one hand, for any $t \in Clk_{\mathcal{M}_2 \parallel \mathcal{M}_3} \cap \xi_{\mathcal{M}_1}(u_1)$, if $t \in Clk_{\mathcal{M}_2}$, by Equation (3) we have $t \in Clk_{\mathcal{M}_1} \cap \xi_{\mathcal{M}_2}(u_2)$, and therefore $t \in Clk_{\mathcal{M}_1} \cap \xi_{\mathcal{M}_2 \parallel \mathcal{M}_3}(\langle u_2, u_3 \rangle)$. If $t \in Clk_{\mathcal{M}_3}$, since $t \in \xi_{\mathcal{M}_1}(u_1)$, $t \in \xi_{\mathcal{M}_1 \parallel \mathcal{M}_2}(g)$, so by Equation (2), there is $t \in Clk_{\mathcal{M}_1 \parallel \mathcal{M}_2} \cap \xi_{\mathcal{M}_3}(u_3)$. From Equation (3) and the fact that $t \in \xi_{\mathcal{M}_1}(u_1)$, we can see that $t \in Clk_{\mathcal{M}_1}$ must hold. Hence, $t \in Clk_{\mathcal{M}_1} \cap \xi_{\mathcal{M}_2 \parallel \mathcal{M}_3}(\langle u_2, u_3 \rangle)$. On the other hand, for any $t \in Clk_{\mathcal{M}_1} \cap \xi_{\mathcal{M}_2 \parallel \mathcal{M}_3}(\langle u_2, u_3 \rangle)$, if $t \in \xi_{\mathcal{M}_2}(u_2)$, then by Equation (3), we know $t \in Clk_{\mathcal{M}_2} \cap \xi_{\mathcal{M}_1}(u_1)$, hence $t \in Clk_{\mathcal{M}_2 \parallel \mathcal{M}_3} \cap \xi_{\mathcal{M}_1}(u_1)$. If $t \in \xi_{\mathcal{M}_3}(u_3)$, by Equation (2), there is $t \in Clk_{\mathcal{M}_3} \cap \xi_{\mathcal{M}_1 \parallel \mathcal{M}_2}(g)$. Thus, $t \in \xi_{\mathcal{M}_1 \parallel \mathcal{M}_2}(g)$. From Equation (3) and the fact that $t \in Clk_{\mathcal{M}_1}$, we can see that $t \in \xi_{\mathcal{M}_1}(u_1)$ must hold. Therefore, $t \in Clk_{\mathcal{M}_2 \parallel \mathcal{M}_3} \cap \xi_{\mathcal{M}_1}(u_1)$.

According to Definition 5.4, from (a) we know that $\langle u_2, u_3 \rangle$ is an initial state of $\mathcal{M}_2 \parallel \mathcal{M}_3$, and from (b) we know that q is an initial state of N_2 .

The other direction can be proved in a similar way, and we omit it here.

(2): For any state q of both N_1 and N_2 and any transition $\langle q, q' \rangle$ of N_1 , let $q' = \langle g', u'_3 \rangle$. By Definition 5.4, we have

$$Clk_{\mathcal{M}_1 \parallel \mathcal{M}_2} \cap \xi_{\mathcal{M}_3}(u'_3) = Clk_{\mathcal{M}_3} \cap \xi_{\mathcal{M}_1 \parallel \mathcal{M}_2}(g'). \quad (4)$$

Let $g' = \langle u'_1, u'_2 \rangle$, and again by Definition 5.4, we have

$$Clk_{\mathcal{M}_1} \cap \xi_{\mathcal{M}_2}(u'_2) = Clk_{\mathcal{M}_2} \cap \xi_{\mathcal{M}_1}(u'_1). \quad (5)$$

Following a similar proof procedure as the proof of (a) and (b) shown earlier in the proof of (1) (by replacing u_1, u_2, u_3, g with u'_1, u'_2, u'_3, g' everywhere), we can prove that

- (a') $Clk_{\mathcal{M}_3} \cap \xi_{\mathcal{M}_2}(u'_2) = Clk_{\mathcal{M}_2} \cap \xi_{\mathcal{M}_3}(u'_3)$ and
- (b') $Clk_{\mathcal{M}_2 \parallel \mathcal{M}_3} \cap \xi_{\mathcal{M}_1}(u'_1) = Clk_{\mathcal{M}_1} \cap \xi_{\mathcal{M}_2 \parallel \mathcal{M}_3}(\langle u'_2, u'_3 \rangle)$.

According to Definition 5.4, from (a') we know that $\langle u'_2, u'_3 \rangle$ is an initial state of $\mathcal{M}_2 \parallel \mathcal{M}_3$, so $\langle q |_{\mathcal{M}_2 \parallel \mathcal{M}_3}, \langle u'_2, u'_3 \rangle \rangle$ is a transition of $\mathcal{M}_2 \parallel \mathcal{M}_3$, and from (b') we know that q' is a state of N_2 , so $\langle q, q' \rangle$ is a transition of N_2 . $\square$

THE PROOF OF PROPOSITION 5.6. First, since $\mathcal{M}_2 \text{ obst}_{F_2} \mathcal{M}_1$, we have $Clk_1 \cap Clk_2 \subseteq F_2$. Therefore, $C_2 \cap F_1 = \emptyset$. Thus, we have

$$\begin{aligned} F &= (F_1 \cup F_2) - ((F_1 \cup F_2) \cap (C_1 \cup C_2)) \\ &= (F_1 \cup F_2) - ((F_1 \cap C_1) \cup (F_1 \cap C_2) \cup (F_2 \cap C_1) \cup (F_2 \cap C_2)) \\ &= (F_1 \cup F_2) - (\emptyset \cup \emptyset \cup (F_2 \cap C_1) \cup \emptyset) \\ &= (F_1 \cup F_2) - (F_2 \cap C_1). \end{aligned}$$

Now we show that F and C are sets of free clocks and constrained clocks of the observer $\mathcal{M}$, respectively.

Let $\mathcal{M} = \langle \Pi, Clk, Q, I, \theta, \xi, \mathcal{T}, \mathcal{E} \rangle$, and for any clock behaviour $X \subseteq F$, we need to prove that

- (1) there exists an initial state q_0 of $\mathcal{M}$ s.t. $\xi(q_0) \cap F = X$;
- (2) for any state q of $\mathcal{M}$, there exists a transition $\langle q, q' \rangle$ s.t. $\xi(q') \cap F = X$.

(1): Since $X \cap F_1 \subseteq F_1$, by Definition 5.12, there exists an initial state x_0 of $\mathcal{M}_1$ s.t. $\xi_1(x_0) \cap F_1 = X \cap F_1$. Let $K = (X \cap F_2) \cup (\xi_1(x_0) \cap F_2) \subseteq F_2$, and by Definition 5.12, we know there is an initial state y_0 of $\mathcal{M}_2$ s.t. $\xi_2(y_0) \cap F_2 = K$. Now we prove that $\text{Clk}_2 \cap \xi_1(x_0) = \text{Clk}_1 \cap \xi_2(y_0)$.

On one hand, for any $t \in \text{Clk}_2 \cap \xi_1(x_0)$, since $t \in \xi_1(x_0) \subseteq \text{Clk}_1$, thus $t \in \text{Clk}_1 \cap \text{Clk}_2$. By $\mathcal{M}_2 \text{ obst}_{F_2} \mathcal{M}_1$, we know $t \in \text{Clk}_1 \cap \text{Clk}_2 \subseteq F_2$. Therefore, $t \in \xi_1(x_0) \cap F_2 \subseteq K$. Since $\xi_2(y_0) \cap F_2 = K$, $t \in \xi_2(y_0)$. Thus, $t \in \text{Clk}_1 \cap \xi_2(y_0)$. On the other hand, for any $t \in \text{Clk}_1 \cap \xi_2(y_0)$, we get $t \in \xi_2(y_0) \subseteq \text{Clk}_2$. Since $\text{Clk}_1 \cap \text{Clk}_2 \subseteq F_2$, hence $t \in \text{Clk}_1 \cap \xi_2(y_0) \subseteq \text{Clk}_1 \cap \text{Clk}_2 \subseteq F_2$. Because $t \in \xi_2(y_0)$ and $t \in F_2$, therefore $t \in K$, and thus $t \in (X \cap F_2) \cup (\xi_1(x_0) \cap F_2)$. Consider t for two cases. As a first case, if $t \in X \cap F_2$, then it is easy to see that $t \in F_1$ from the fact that $t \notin C_1$ but $t \in \text{Clk}_1$. Therefore, $t \in X \cap F_1 = \xi_1(x_0) \cap F_1 \subseteq \xi_1(x_0)$. Hence, $t \in \text{Clk}_2 \cap \xi_1(x_0)$. In a second case, if $t \in \xi_1(x_0) \cap F_2$, we immediately obtain $t \in \xi_1(x_0)$. Thus, $t \in \text{Clk}_2 \cap \xi_1(x_0)$.

According to Definition 5.4, we know that $\langle x_0, y_0 \rangle$ is an initial state of $\mathcal{M}$. Next we show that $\xi(\langle x_0, y_0 \rangle) \cap F = (\xi_1(x_0) \cap \xi_2(y_0)) \cap F = X$.

On one hand, for any $t \in X$, $t \in X \subseteq F$, therefore $t \in F_1 \cup F_2$. If $t \in F_1$, then $t \in X \cap F_1$, so $t \in \xi_1(x_0) \cap F_1$, similar for the case $t \in F_2$. Hence, $t \in \xi_1(x_0) \cup \xi_2(y_0)$. On the other hand, for any $t \in (\xi_1(x_0) \cup \xi_2(y_0)) \cap F$, consider two situations. In a first situation, if $t \in \xi_1(x_0)$, then $t \in \xi_1(x_0) \subseteq \text{Clk}_1$. Because $t \in F$, therefore $t \in F_1 \cup F_2$ and $t \notin C_1 \cap F_2$. It is easy to see that $t \in F_1$; otherwise, if $t \notin F_1$, then $t \in F_2$. Since $t \notin C_1 \cap F_2$, therefore $t \notin C_1$. But $t \in \text{Clk}_1$, so it must be $t \in F_1$, which causes the contradiction. From $t \in \xi_1(x_0)$ and $t \in F_1$, we obtain $t \in \xi_1(x_0) \cap F_1 = X \cap F_1$, thus $t \in X$. In a second situation, if $t \in \xi_2(y_0) \subseteq \text{Clk}_2$, it is easy to see that $t \in F_2$. If not so, since $t \in F = (F_1 \cup F_2) - (C_1 \cap F_2)$, we know that $t \in F_1$. Thus, there is $t \in \xi_2(y_0) \cap F_1 \subseteq \text{Clk}_2 \cap \text{Clk}_1$. But $\text{Clk}_2 \cap \text{Clk}_1 \subseteq F_2$ due to $\mathcal{M}_2 \text{ obst}_{F_2} \mathcal{M}_1$, which contradicts the assumption that $t \notin F_2$. From $t \in \xi_2(y_0)$ and $t \in F_2$, we have $t \in \xi_2(y_0) \cap F_2 = K = (X \cap F_2) \cup (\xi_1(x_0) \cap F_2)$. Since $t \notin \xi_1(x_0)$, we have $t \in X \cap F_2$, hence $t \in X$.

(2): For any state $\langle x, y \rangle$ of $\mathcal{M}$, since $X \cap F_1 \subseteq F_1$, by Definition 5.12, there exists a transition $\langle x, x' \rangle$ in $\mathcal{M}_1$ s.t. $\xi_1(x') \cap F_1 = X \cap F_1$. Let $K = (X \cap F_2) \cup (\xi_1(x') \cap F_2) \subseteq F_2$, and by Definition 5.12, we know there is a transition $\langle y, y' \rangle$ in $\mathcal{M}_2$ s.t. $\xi_2(y') \cap F_2 = K$. By following a similar procedure of the proof of $\text{Clk}_2 \cap \xi_1(x_0) = \text{Clk}_1 \cap \xi_2(y_0)$ in (1) shown previously (explicitly, by simply replacing x_0, y_0 with x', y' , respectively), we can prove $\text{Clk}_2 \cap \xi_1(x') = \text{Clk}_1 \cap \xi_2(y')$. Therefore, $\langle x', y' \rangle$ is a state of $\mathcal{M}$, and according to Definition 5.4, $\langle \langle x, y \rangle, \langle x', y' \rangle \rangle$ is a transition of $\mathcal{M}$. Again, following a similar procedure of the proof of $\xi(\langle x_0, y_0 \rangle) \cap F = X$ in (1) shown previously (by simply replacing x_0, y_0 with x', y' , respectively), we can prove $\xi(\langle x', y' \rangle) \cap F = X$.

For C , we have $C = \text{Clk} - F = ((\text{Clk}_1 \cup \text{Clk}_2) - (F_1 \cup F_2)) \cup (C_1 \cap F_2) = (C_1 \cup C_2) \cup (C_1 \cap F_2) = C_1 \cup C_2$. $\square$

In the proofs of the following lemmas, use $\mathcal{M}_1 \leftrightarrow \mathcal{M}_2$ to represent both $\mathcal{M}_1 \rightsquigarrow^+ \mathcal{M}_2$ and $\mathcal{M}_2 \rightsquigarrow^+ \mathcal{M}_1$, and use $\mathcal{M}_1 \uparrow \mathcal{M}_2$ to express that $\mathcal{M}_1$ and $\mathcal{M}_2$ are independent from each other.

$\mathcal{M}_1$ only depends on $\mathcal{M}_2$, denoted by $\mathcal{M}_1 \rightsquigarrow^o \mathcal{M}_2$, is defined such that $\mathcal{M}_1 \rightsquigarrow \mathcal{M}_2$ and there is no other $\mathcal{M}$ s.t. $\mathcal{M}_1 \rightsquigarrow^+ \mathcal{M}$ and $\mathcal{M} \rightsquigarrow^+ \mathcal{M}_2$.

Lemma B.1 shows a relationship between the observation relation and the dependence relation.

LEMMA B.1. Given two observers $\mathcal{M}_1$ and $\mathcal{M}_2$, $\mathcal{M}_1 \text{ obst } \mathcal{M}_2$ iff

- (a) $\mathcal{M}_1 \rightsquigarrow \mathcal{M}_2$ and $\mathcal{M}_2 \not\rightsquigarrow \mathcal{M}_1$, or
- (b) $\mathcal{M}_1, \mathcal{M}_2$ are directly independent, i.e., $\mathcal{M}_1 \not\rightsquigarrow \mathcal{M}_2$ and $\mathcal{M}_2 \not\rightsquigarrow \mathcal{M}_1$.

PROOF. Let $\mathcal{M}_i[F_i, C_i] = \langle \Pi_i, \text{Clk}_i, Q_i, I_i, \theta_i, \xi_i, \mathcal{T}_i, \mathcal{E}_i \rangle$ ($i = 1, 2$). It is not hard to see that

$$\text{Clk}_1 \cap \text{Clk}_2 \subseteq F_1 \text{ iff } C_1 \cap \text{Clk}_2 = \emptyset \text{ iff } C_1 \cap \text{Clk}_2 = \emptyset \text{ and } C_2 \cap \text{Clk}_1 \supseteq \emptyset.$$

If $C_1 \cap \text{Clk}_2 = \emptyset$ and $C_2 \cap \text{Clk}_1 \supset \emptyset$, then (a) holds. If $C_1 \cap \text{Clk}_2 = \emptyset$ and $C_2 \cap \text{Clk}_1 = \emptyset$, then (b) holds. $\square$

The proof of Theorem 5.1 relies on Lemma B.1, Lemma B.2 and Lemma B.3.

LEMMA B.2. *Given a finite set of observers A ($|A| \geq 2$), if A is well defined, then there exist $M_1, M_2 \in \Phi$ s.t. $M_1 \rightsquigarrow^o M_2$ or $M_1 \Uparrow M_2$.*

PROOF. We show that in A , if there exists a relation $M_1 \rightsquigarrow N_1$ between observers M_1 and N_1 , then there must exist a relation $O_1 \rightsquigarrow^o O_2$ for some observers O_1 and O_2 .

Define two properties:

- $P_1(A, M)$: A is a set of observers in which each observer O satisfies $O = M$ or $O \rightsquigarrow^+ M$.
- $P_2(A, M)$: A is a set of observers in which each observer O satisfies $O = M$ or $M \rightsquigarrow^+ O$.

Set $D_1 = \{M_1\}$, $E_1 = \{N_1\}$; clearly, $P_1(D_1, M_1)$ and $P_2(E_1, N_1)$ hold.

For the initial step (step 1), suppose $M_1 \not\rightsquigarrow^o N_1$, then by Definition 5.13, there is an observer N_2 different from all elements in D_1 and E_1 s.t. $M_1 \rightsquigarrow^+ N_2 \rightsquigarrow^+ N_1$. Without loss of generality, we can assume there is an observer M_2 s.t. $M_1 \rightsquigarrow^+ M_2 \rightsquigarrow N_2$, where M_2 may equal to M_1 . Let $D_2 = D_1 \cup \{M_2\}$ and $E_2 = E_1 \cup \{N_2\}$; it is easy to see that $P_1(D_2, M_2)$ and $P_2(E_2, N_2)$ hold.

Now suppose that at any step n ($n \geq 2$), there are $M_n \rightsquigarrow N_n$, D_n and E_n s.t. $P_1(D_n, M_n)$ and $P_2(E_n, N_n)$ hold. Suppose $M_n \not\rightsquigarrow^o N_n$, then by Definition 5.13 we know that there is an observer N_{n+1} different from M_n and N_n s.t. $M_n \rightsquigarrow^+ N_{n+1} \rightsquigarrow^+ N_n$. It is not hard to see that N_{n+1} must not be equal to other elements in both D_n and E_n , as otherwise a contradiction will occur. For example, if $N_{n+1} \in D_n$, then by $P_1(D_n, M_n)$ we have $N_{n+1} \rightsquigarrow^+ M_n$, which contradicts the fact that A is well defined. Similar for $N_{n+1} \in E_n$. Therefore, $N_{n+1} \notin D_n \cup E_n$. Without loss of generality, we can assume there is an observer M_{n+1} s.t. $M_n \rightsquigarrow^+ M_{n+1} \rightsquigarrow N_{n+1}$, where M_{n+1} may be equal to M_n . Let $D_{n+1} = D_n \cup \{M_{n+1}\}$ and $E_{n+1} = E_n \cup \{N_{n+1}\}$; it is easy to see that $P_1(D_{n+1}, M_{n+1})$ and $P_2(E_{n+1}, N_{n+1})$ hold.

Continuing the steps, we can see that if for every $M_n \rightsquigarrow N_n$ ($n \geq 1$), $M_n \not\rightsquigarrow^o N_n$, then the set $D_n \cup E_n$ will grow to infinitely large, which violates the assumption that A is finite. Thus, there must be $M_i \rightsquigarrow^o N_i$ for some $i \geq 1$. $\square$

LEMMA B.3. *Given a set of observers A and any two observers $M_i[F_i, C_i] = \langle \Pi_i, Clk_i, Q_i, I_i, \theta_i, \xi_i, \mathcal{T}_i, \mathcal{E}_i \rangle \in A$ ($i = 1, 2$) s.t. either $M_1 \rightsquigarrow^o M_2$ or $M_1 \Uparrow M_2$ holds, if A is well defined, then $A' = (A \cup \{M_1 \parallel M_2\}) - \{M_1, M_2\}$ is also well defined.*

PROOF. Let $M = M_1 \parallel M_2$. Suppose A' is not well defined, then there must exist a relation $M \rightsquigarrow M''$ for some observer $M'' \in A'$. Because if the interdependence relation happens between two observers N_1 and N_2 that is different from M , there must be a path $N_1 \rightsquigarrow^+ M \rightsquigarrow^+ N_2$ or $N_2 \rightsquigarrow^+ M \rightsquigarrow^+ N_1$ that has something to do with M , as otherwise N_1 and N_2 are dependent on each other in the old set A , which contradicts the assumption that A is well defined. Assume $N_1 \rightsquigarrow^+ M \rightsquigarrow^+ N_2$, then we have $M \rightsquigarrow^+ N_2$ and $N_2 \rightsquigarrow^+ N_1 \rightsquigarrow^+ M$, hence $M \rightsquigarrow N_2$.

Suppose $M \rightsquigarrow M''$. Without loss of generality, we can assume there are two observers $M_i[F_i, C_i] = \langle \Pi_i, Clk_i, Q_i, I_i, \theta_i, \xi_i, \mathcal{T}_i, \mathcal{E}_i \rangle \in A$ ($i = 3, 4$) satisfying $M \rightsquigarrow M_3 \rightsquigarrow^+ M'' \rightsquigarrow^+ M_4 \rightsquigarrow M$. With $M \rightsquigarrow M_3$ and $M_4 \rightsquigarrow M$, by Definition 5.13 and Proposition 5.6, we have that

$$C_3 \cap (Clk_1 \cup Clk_2) \neq \emptyset \text{ and } (C_1 \cup C_2) \cap Clk_4 \neq \emptyset$$

hold. If $C_3 \cap Clk_1 \neq \emptyset$ and $C_1 \cap Clk_4 \neq \emptyset$ hold (similar for the case $C_3 \cap Clk_2 \neq \emptyset$ and $C_2 \cap Clk_4 \neq \emptyset$), by Definition 5.13 we know that $M_1 \rightsquigarrow M_3 \rightsquigarrow^+ M'' \rightsquigarrow^+ M_4 \rightsquigarrow M_1$, which contradicts the fact that A is well defined. If $C_3 \cap Clk_1 \neq \emptyset$ and $C_2 \cap Clk_4 \neq \emptyset$ hold, then by Definition 5.13 there is $M_1 \rightsquigarrow M_3 \rightsquigarrow^+ M'' \rightsquigarrow^+ M_4 \rightsquigarrow M_2$, which contradicts the fact that $M_1 \rightsquigarrow^o M_2$. If $C_3 \cap Clk_2 \neq \emptyset$ and $C_1 \cap Clk_4 \neq \emptyset$ hold, then by Definition 5.13 $M_2 \rightsquigarrow M_3 \rightsquigarrow^+ M'' \rightsquigarrow^+ M_4 \rightsquigarrow M_1$ hold, which contradicts the fact that A is well defined.

In summary, A' must be well defined. $\square$

The proof of Theorem 5.1 relies on Lemmas B.1, B.2 and B.3.

THE PROOF OF THEOREM 5.1. We proceed by induction on the size of $A^{\sim\Phi}$. When $|A^{\sim\Phi}| = 2$, let $A^{\sim\Phi} = \{\mathcal{M}_1[F_1, C_1], \mathcal{M}_2[F_2, C_2]\}$, and by Lemma B.1 we know that either $\mathcal{M}_1 \text{ obst } \mathcal{M}_2$ or $\mathcal{M}_2 \text{ obst } \mathcal{M}_1$ holds. Assume $\mathcal{M}_2 \text{ obst } \mathcal{M}_1$, and by Proposition 5.6 we obtain $\mathcal{M}[F, C] = \mathcal{M}_1 \parallel \mathcal{M}_2$, where $F = (F_1 \cup F_2) - (C_1 \cap F_2)$. Since $\mathcal{M}_2 \text{ obst } \mathcal{M}_1$, it is easy to see that $C_2 \cap F_1 = \emptyset$. Therefore, $F = (F_1 \cup F_2) - ((C_1 \cap F_2) \cup (C_2 \cap F_1) \cup (C_1 \cap F_1) \cup (C_2 \cap F_2)) = (F_1 \cup F_2) - ((F_1 \cup F_2) \cap (C_1 \cup C_2))$, $C = \text{Clk}_{\mathcal{M}} - F = C_1 \cup C_2$. Now we show that this proposition holds for any $|A^{\sim\Phi}| > 2$, under the assumption that it holds for any B with $|B| < |A^{\sim\Phi}|$.

According to the associativity of operator $\parallel$ (Definition 5.1), by Lemma B.2 and B.3 we know that by gradually making product between two observers that have a direct dependent relationship, and we can obtain a product of $|A^{\sim\Phi}| - 1$ observers: $\mathcal{M}[F, C] = \parallel_{Cn^+ \in \sim\Phi'} \mathcal{M}^{Cn^+}$, where $\sim\Phi'$ is the set of the constraints of the composed observers, and it satisfies that $\sim\Phi' \subseteq (\sim\Phi)$ and $|\sim\Phi'| = |\sim\Phi| - 1 = |A^{\sim\Phi}| - 1$. For example, in $A^{\sim\Phi}$, first we choose two observers $\mathcal{M}_1, \mathcal{M}_2$ s.t. $\mathcal{M}_1 \rightsquigarrow^o \mathcal{M}_2$ or $\mathcal{M}_1 \uparrow\uparrow \mathcal{M}_2$, and we compute $\mathcal{M}_1 \parallel \mathcal{M}_2$ and obtain a new well-defined set $A_1 = A^{\sim\Phi} \cup \{\mathcal{M}_1 \parallel \mathcal{M}_2\} - \{\mathcal{M}_1, \mathcal{M}_2\}$. We continue this process in A_1 and obtain the sets $A_2, A_3, \dots$, until the size of the set, named A_k ($k \geq 1$), is reduced to 2. By hypothesis, F, C satisfy

$$F = \bigcup_{Cn^+ \in \sim\Phi'} F_{Cn^+} - \left(\bigcup_{Cn^+ \in \sim\Phi'} F_{Cn^+} \cap \bigcup_{Cn^+ \in \sim\Phi'} C_{Cn^+} \right) \text{ and } C = \bigcup_{Cn^+ \in \sim\Phi'} C_{Cn^+}.$$

Let $A_k = \{\mathcal{M}[F, C], \mathcal{M}'[F', C']\}$, and from the building procedure of $\mathcal{M}$ we know that A_k is well defined. Let $\mathcal{M}''[F'', C''] = \mathcal{M} \parallel \mathcal{M}'$, and by Definition 5.6, no matter whether $\mathcal{M} \text{ obst } \mathcal{M}'$ or $\mathcal{M}' \text{ obst } \mathcal{M}$, we have

$$F'' = (F \cup F') - ((F \cup F') \cap (C \cup C')) \text{ and } C'' = C \cup C'.$$

Obviously, $C'' = \bigcup_{Cn^+ \in \sim\Phi} C_{Cn^+}$. Let $U = \bigcup_{Cn^+ \in \sim\Phi} F_{Cn^+} \cup \bigcup_{Cn^+ \in \sim\Phi} C_{Cn^+}$ be the set of all clocks in the context, $\bar{X} = \{x \mid x \in U \text{ and } x \notin X\}$ be the complementary set of X , then $X - Y = X \cap \bar{Y}$. F'' can be computed as follows:

$$\begin{aligned} F'' &= X_3 - (X_3 \cap X_4) = X_3 \cap \overline{(X_3 \cap X_4)} = X_3 \cap (\overline{X_3} \cup \overline{X_4}) = X_3 \cap \bar{X}_4 \\ &= ((X_1 \cap \overline{(X_1 \cap C)}) \cup F') \cap \bar{X}_4 = (X_1 \cup F') \cap \overline{((X_1 \cap C) \cup F')} \cap \bar{X}_4 \\ &= (X_1 \cup F') \cap \overline{\{((X_1 \cap C) \cap \bar{F}') \cup X_4\}} \\ &= (X_1 \cup F') \cap \overline{\{((X_1 \cap \bar{F}') \cup X_4) \cap (C \cup X_4)\}} \\ &= (X_1 \cup F') \cap \overline{\{((X_1 \cap \bar{F}') \cup X_4) \cap X_4\}} \\ &= (X_1 \cup F') \cap \bar{X}_4 = (X_1 \cup F') - ((X_1 \cup F') \cap X_4) \\ &= \bigcup_{Cn^+ \in \sim\Phi} F_{Cn^+} - \left(\bigcup_{Cn^+ \in \sim\Phi} F_{Cn^+} \cap \bigcup_{Cn^+ \in \sim\Phi} C_{Cn^+} \right), \end{aligned}$$

where $X_1 = \bigcup_{Cn^+ \in \sim\Phi'} F_{Cn^+}$, $X_2 = X_1 - (X_1 \cap C)$, $X_3 = X_2 \cup F'$ and $X_4 = C \cup C'$. $\square$

LEMMA B.4. For a clock behaviour B and a set of well-defined clock definitions $\mathbf{D}$, we can construct a state x in the observer $\mathcal{M}[F, C] = \parallel_{Cn \in \mathbf{D}} \mathcal{M}^{Cn}$ which satisfies one of the following two cases:

- (a) either x is an initial state of $\mathcal{M}$ s.t. $\xi_{\mathcal{M}}(x) \cap F = B \cap F$ or
- (b) $\langle q, x \rangle$ is a transition starting from a state q s.t. $\xi_{\mathcal{M}}(x) \cap F = B \cap F$.

Moreover, it satisfies that

$$\text{for any clock } d \text{ in } \mathbf{D}, \text{ if } d \text{ is not pure, then } B \models d, \mathbf{D} \text{ iff } d \in \xi_{\mathcal{M}}(x).$$

PROOF. We proceed by induction on the steps of a procedure for constructing such a state x . We begin by Step 1.

Step 1. Let $A = \{\mathcal{M}^{Cn}\}_{Cn \in D}$, and since D is well defined, we can find a clock definition $d_0 \triangleq e_0$ s.t. for all other $\mathcal{N} \in A - \{\mathcal{M}^{d_0 \triangleq e_0}\}$, $\mathcal{N} \rightsquigarrow^+ \mathcal{M}^{d_0 \triangleq e_0}$ or $\mathcal{N} \Uparrow \mathcal{M}^{d_0 \triangleq e_0}$ holds. Let $B_0 = B - C$ —that is, B_0 is obtained by removing all constrained clocks in B . In the observer $\mathcal{M}^{d_0 \triangleq e_0}[F_0, C_0]$ (where $F_0 = C(e_0)$, $C_0 = \{d_0\}$ according to Proposition 5.3), for the clock behaviour $B_0 \cap F_0$, by Definition 5.12, there is a state s_0 satisfying the case (a) or (b) (let ξ_0 be the clock labelling function of $\mathcal{M}^{d_0 \triangleq e_0}$):

case (a): there exists an initial state s_0 s.t. $\xi_0(s_0) \cap F_0 = B_0 \cap F_0$;

case (b): from the state $q|_{\mathcal{M}^{d_0 \triangleq e_0}}$, there exists a transition $\langle q|_{\mathcal{M}^{d_0 \triangleq e_0}}, s_0 \rangle$ s.t. s_0 s.t. $\xi_0(s_0) \cap F_0 = B_0 \cap F_0$.

In both preceding cases, we have $\xi_0(s_0) \cap F_0 = B_0 \cap F_0$.

Next we show that the following two propositions hold:

- (1) $\xi_0(s_0) \cap F_0 = B \cap F_0$.
- (2) For any $d \in C_0$, $B \models d, D$ iff $d \in \xi_0(s_0)$.

For (1), it obviously holds because $C \cap F_0 = \emptyset$.

As for (2), since $C_0 = \{d_0\}$, we only need to prove $B \models d_0, D$ iff $d_0 \in \xi_0(s_0)$. Without loss of generality, we assume that the clock expression e_0 is a Union expression—that is, $e_0 = c_1 + c_2$ for some clocks c_1 and c_2 . Other cases can be proved in a similar way. If $B \models d_0, D$, then by the semantics of $d_0 \triangleq e_0$ in Table 2, we have $B \models c_1, D$ or $B \models c_2, D$. Assume $B \models c_1, D$, and since $\mathcal{M}^{d_0 \triangleq e_0}$ does not depend on any observers, c_1, c_2 must be pure clocks. So according to the semantics of $d_0 \triangleq c_1 + c_2$ in Table 2, we have $c_1 \in B$. Since c_1 is a pure clock, it cannot be defined in other clock definitions, so we have $c_1 \notin C$, hence $c_1 \in B - C = B_0$. Because $c_1 \in B_0 \cap F_0$, therefore $c_1 \in \xi_0(s_0) \cap F_0 \subseteq \xi_0(s_0)$. According to the observer $\mathcal{M}^{d_0 \triangleq c_1 + c_2}$ in Table 4, it is easy to see that $d_0 \in \xi_0(s_0)$. However, if $d_0 \in \xi_0(s_0)$, by the observer $\mathcal{M}^{d_0 \triangleq c_1 + c_2}$ in Table 4, we know that either $c_1 \in \xi_0(s_0)$ or $c_2 \in \xi_0(s_0)$ holds. Assume $c_1 \in \xi_0(s_0)$, and because $c_1 \in F_0$, we have $c_1 \in \xi_0(s_0) \cap F_0 = B_0 \cap F_0 \subseteq B$. Since c_1 is pure, we have $B \models c_1, D$, and therefore $B \models d_0, D$ by the semantics of $d_0 \triangleq c_1 + c_2$ in Table 2.

Let $B_1 = B_0 \cup \xi_0(s_0)$, $D_1 = D - \{d_0 \triangleq e_0\}$, $\mathcal{M}_0 = \mathcal{M}^{d_0 \triangleq e_0}$ and $A_1 = (A \cup \mathcal{M}_0) - \{\mathcal{M}^{d_0 \triangleq e_0}\}$. Obviously, D_1 and A_1 are well defined.

[the end of Step 1]

Now suppose that after Step n ($n \geq 1$), we obtain a sequence of clock definitions $d_0 \triangleq e_0, d_1 \triangleq e_1, \dots, d_{n-1} \triangleq e_{n-1}$, a sequence of sets of clock definitions $D_1, \dots, D_n$ where $D_k = D - \{d_0 \triangleq e_0, d_1 \triangleq e_1, \dots, d_{k-1} \triangleq e_{k-1}\}$ ($1 \leq k \leq n$), a sequence of observers $\mathcal{M}_0, \dots, \mathcal{M}_{n-1}$ where $\mathcal{M}_k[F_k, C_k] = \bigsqcup_{i=0}^k \mathcal{M}^{d_i \triangleq e_i}$ ($0 \leq k \leq n-1$), a sequence of states $s_0, \dots, s_{n-1}$, a sequence of clock behaviours $B_1, \dots, B_n$ where $B_k = B_0 \cup \xi_{k-1}(s_{k-1})$ (where ξ_{k-1} is the labelling function of the observer $\mathcal{M}_{k-1}$, $1 \leq k \leq n$), and a sequence of sets of observers $A_1, \dots, A_n$ where $A_k = (A \cup \mathcal{M}_{k-1}) - \bigsqcup_{i=0}^{k-1} \{\mathcal{M}^{d_i \triangleq e_i}\}$ ($1 \leq k \leq n$). They satisfy that

- (A) D_k and A_k are well defined ($1 \leq k \leq n$);
- (B) $F_k = \bigcup_{i=0}^k F_i - (\bigcup_{i=0}^k F_i \cap \bigcup_{i=0}^k C_i)$ and $C_k = \bigcup_{i=0}^k C_i$, where $0 \leq k \leq n-1$, F_i, C_i are the sets of free clocks and constrained clocks of the observer $\mathcal{M}_i$, respectively, obtained at Step $i+1$;
- (C) for any $\mathcal{N} \in A_k - \{\mathcal{M}_{k-1}\}$, $\mathcal{N} \rightsquigarrow^+ \mathcal{M}_{k-1}$ or $\mathcal{N} \Uparrow \mathcal{M}_{k-1}$ holds ($1 \leq k \leq n$).

In addition, it satisfies that

- (I) s_k is a state of $\mathcal{M}_k$ s.t. $\xi_{\mathcal{M}_k}(s_k) \cap F_k = B \cap F_k$ holds, where $0 \leq k \leq n-1$.
- (II) For any $d \in C_k$, $B \models d, D$ iff $d \in \xi_{\mathcal{M}_k}(s_k)$, where $0 \leq k \leq n-1$.

Clearly, in Step 1, all conditions (A), (B), (C), (I), (II) hold, and now we construct Step $n + 1$ from the assumption of Step n .

Step $n + 1$. Since $\mathbf{D}_n$ is well defined, we can select a clock definition $d_n \triangleq e_n$ in $\mathbf{D}_n$ s.t. for any $\mathcal{N} \in A_n - \{\mathcal{M}^{d_n \triangleq e_n}, \mathcal{M}_{n-1}\}$, $\mathcal{N} \rightsquigarrow^+ \mathcal{M}^{d_n \triangleq e_n}$ or $\mathcal{N} \Uparrow \mathcal{M}^{d_n \triangleq e_n}$ holds. From the preceding condition (C), it is easy to see that in the set A_n , $\mathcal{M}^{d_n \triangleq e_n} \rightsquigarrow^o \mathcal{M}_{n-1}$ or $\mathcal{M}^{d_n \triangleq e_n} \Uparrow \mathcal{M}_{n-1}$ holds. This is because if there exists a $\mathcal{N}'$ s.t. $\mathcal{M}^{d_n \triangleq e_n} \rightsquigarrow^+ \mathcal{N}' \rightsquigarrow^+ \mathcal{M}_{n-1}$, then we immediately get that $\mathcal{N}'$ violates the way of how $d_n \triangleq e_n$ is selected. By Lemma B.1, we know $\mathcal{M}^{d_n \triangleq e_n} \text{ obst } \mathcal{M}_{n-1}$. In the observer $\mathcal{M}^{d_n \triangleq e_n}[F', C']$ (where $F' = C(e_n)$, $C' = \{d_n\}$), similar to Step 1, for the behaviour $B_n \cap F'$, by Definition 5.12, there is a state s' satisfying case (a) or (b), and no matter whether it satisfies case (a) or (b), we both have

$$\xi'(s') \cap F' = B_n \cap F'$$

(let ξ' be the labelling function of $\mathcal{M}^{d_n \triangleq e_n}$).

Let $\mathcal{M}_n[F_n, C_n] = \mathcal{M}^{d_n \triangleq e_n} \parallel \mathcal{M}_{n-1}$, and by Proposition 5.6, we see that $F_n = (F' \cup F_{n-1}) - (C' \cup C_{n-1})$ and $C_n = C' \cup C_{n-1}$. Let $s_n = \langle s', s_{n-1} \rangle$. We now prove that conditions (I) and (II) hold for $k = n$ —that is, the following two propositions hold:

- (1) s_n is a state of $\mathcal{M}_n$ s.t. $\xi_{\mathcal{M}_n}(s_n) \cap F_n = B \cap F_n$ holds.
- (2) For any $d \in C_n$, $B \models d, \mathbf{D}$ iff $d \in \xi_{\mathcal{M}_n}(s_n)$.

For (1), we first show that s_n is a state of $\mathcal{M}_n$. Let Clk' , Clk_{n-1} be the set of clocks of the observers $\mathcal{M}^{d_n \triangleq e_n}$ and $\mathcal{M}_{n-1}$, respectively. By Definition 5.4, we need to show $\xi_{n-1}(s_{n-1}) \cap \text{Clk}' = \xi'(s') \cap \text{Clk}_{n-1}$. On one hand, for any $t \in \xi_{n-1}(s_{n-1}) \cap \text{Clk}'$, since $\mathcal{M}^{d_n \triangleq e_n} \text{ obst } \mathcal{M}_{n-1}$, we have $t \in \xi_{n-1}(s_{n-1}) \cap \text{Clk}' \subseteq \text{Clk}_{n-1} \cap \text{Clk}' \subseteq F'$. From $t \in \xi_{n-1}(s_{n-1})$, we also have $t \in B_n$ because $B_n = B_0 \cup \xi_{n-1}(s_{n-1})$. Thus, $t \in B_n \cap F' = \xi'(s') \cap F'$. Therefore, $t \in \xi'(s')$. From $t \in \xi_{n-1}(s_{n-1}) \subseteq \text{Clk}_{n-1}$, we obtain $t \in \xi'(s') \cap \text{Clk}_{n-1}$. On the other hand, for any $t \in \xi'(s') \cap \text{Clk}_{n-1}$, since $t \in \xi'(s') \cap \text{Clk}_{n-1} \subseteq \text{Clk}' \cap \text{Clk}_{n-1} \subseteq F'$, we get $t \in \xi'(s') \cap F' = B_n \cap F'$. Thus, $t \in B_n = B_0 \cup \xi_{n-1}(s_{n-1})$. If $t \in B_0$, then $t \notin C$, and from $t \in \text{Clk}_{n-1}$ we get that $t \in F_{n-1}$. From the fact that $B_0 \subseteq B$, we get $t \in \xi_{n-1}(s_{n-1})$ because $\xi_{n-1}(s_{n-1}) \cap F_{n-1} = B \cap F_{n-1}$. From $t \in \xi'(s') \subseteq \text{Clk}'$, we obtain $t \in \xi_{n-1}(s_{n-1}) \cap \text{Clk}'$.

Now we show that $\xi_{\mathcal{M}_n}(s_n) \cap F_n = B \cap F_n$. On one hand, for any $t \in \xi_{\mathcal{M}_n}(s_n) \cap F_n = (\xi_{n-1}(s_{n-1}) \cup \xi'(s')) \cap F_n$, we consider two situations: (a) if $t \in \xi_{n-1}(s_{n-1})$, from $t \in F_n$, $t \in \xi_{n-1}(s_{n-1}) \subseteq \text{Clk}_{n-1}$ and the fact that $F_n = (F' \cup F_{n-1}) - (C' \cup C_{n-1})$, we can get $t \in F_{n-1}$, thus, $t \in \xi_{n-1}(s_{n-1}) \cap F_{n-1} = B \cap F_{n-1}$, hence $t \in B$; (b) if $t \in \xi'(s')$, then $t \in \xi'(s') \subseteq \text{Clk}'$. Again from $t \in F_n$, we have $t \in F'$. Therefore, $t \in \xi'(s') \cap F' = B_n \cap F'$. Thus, $t \in B_n = B_0 \cup \xi_{n-1}(s_{n-1})$. Since $t \notin \xi_{n-1}(s_{n-1})$, we get that $t \in B_0$. From $B_0 = B - C$, we immediately obtain that $t \in B$. On the other hand, for any $t \in B \cap F_n$, from $t \in F_n$ and the fact that for any $\mathcal{N} \in A_n - \{\mathcal{M}^{d_n \triangleq e_n}, \mathcal{M}_{n-1}\}$, $\mathcal{M}^{d_n \triangleq e_n} \not\rightsquigarrow \mathcal{N}$ and $\mathcal{M}_{n-1} \not\rightsquigarrow \mathcal{N}$ hold, we can have $t \notin C$. First, since $t \in F_n$, $t \notin C' \cup C_{n-1}$. If $t \in C_s$ for some observer $\mathcal{N}_s[F_s, C_s] \in A_n - \{\mathcal{M}^{d_n \triangleq e_n}, \mathcal{M}_{n-1}\}$, we have $t \in C_s \cap F' \subseteq C_s \cap \text{Clk}'$ or $t \in C_s \cap F_{n-1} \subseteq C_s \cap \text{Clk}_{n-1}$, which violates the fact that $\mathcal{M}^{d_n \triangleq e_n} \not\rightsquigarrow \mathcal{N}_s$ or $\mathcal{M}_{n-1} \not\rightsquigarrow \mathcal{N}_s$. From $B_0 = B - C$, we get $t \in B_0 \subseteq B_n$. We consider two situations: (c) if $t \in F'$, then $t \in B_n \cap F'$, thus $t \in \xi'(s') \subseteq \xi_{\mathcal{M}_n}(s_n)$; (d) if $t \in F_{n-1}$, then $t \in B \cap F_{n-1}$, therefore $t \in \xi_{n-1}(s_{n-1}) \subseteq \xi_{\mathcal{M}_n}(s_n)$.

For (2), we only need to prove that for any $d \in C'$ (which is d_n), $B \models d, \mathbf{D}$ iff $d \in \xi_{\mathcal{M}_n}(s_n)$ since for any $d' \in C_{n-1}$, by the condition (II) in the assumption of Step n ,

we already have $B \models d', \mathbf{D}$ iff $d' \in \xi_{n-1}(s_{n-1})$. Since $\xi_{n-1}(s_{n-1}) \subseteq \xi_{n-1}(s_{n-1}) \cup \xi'(s') = \xi_{\mathcal{M}_n}(s_n)$, we have $d' \in \xi_{\mathcal{M}_n}(s_n)$.

Without loss of generality, we assume that the clock expression e_n is a Union expression—that is, $e_n = c_1 + c_2$ for some clocks c_1, c_2 . Other cases are similar and are omitted here. On one hand, if $B \models d_n, \mathbf{D}$, then by the expression e_n , we have $B \models c_1, \mathbf{D}$ or $B \models c_2, \mathbf{D}$ holds. Assume $B \models c_1, \mathbf{D}$. We consider two cases when c_1 is a pure clock or not. In a first case, if c_1 is pure, by the semantics of clock definitions in Table 2, $c_1 \in B$. Since c_1 cannot be defined in other clock definitions, we have $c_1 \notin C$. Therefore, $c_1 \in B - C$, hence $c_1 \in B_0 \subseteq B_n$. Since $c_1 \in F'$, we have $c_1 \in B_n \cap F' = \xi'(s') \cap F' \subseteq \xi'(s')$. According to the observer $\mathcal{M}^{d_n \triangleq c_1 + c_2}$ in Table 4, we get $d_n \in \xi'(s') \subseteq \xi_{\mathcal{M}_n}(s_n)$. In a second case, if c_1 is not pure, it must be defined in some clock definition(s). Thus, $c_1 \in C$. It is not hard to see that it must be $c_1 \in C_{n-1}$ because except for $\mathcal{M}_{n-1}$, $\mathcal{M}^{d_n \triangleq e_n} \not\rightsquigarrow N$ for any other $N \in A_n$. Because $B \models c_1, \mathbf{D}$, therefore by condition (II) in the assumption of the preceding Step n , we have $c_1 \in \xi_{n-1}(s_{n-1})$. Since $\xi'(s') \cap \text{Clk}_{n-1} = \xi_{n-1}(s_{n-1}) \cap \text{Clk}'$ as proved earlier, we know that $c_1 \in \xi'(s')$. Again by the observer $\mathcal{M}^{d_n \triangleq c_1 + c_2}$ in Table 4, we have $d_n \in \xi'(s') \subseteq \xi_{\mathcal{M}_n}(s_n)$.

On the other hand, suppose $d_n \in \xi_{\mathcal{M}_n}(s_n)$, we have $d_n \in \xi'(s') \cup \xi_{n-1}(s_{n-1})$. It is not hard to see that $d_n \in \xi'(s')$ because $\mathcal{M}_{n-1} \not\rightsquigarrow \mathcal{M}^{d_n \triangleq e_n}$. By the observer $\mathcal{M}^{d_n \triangleq c_1 + c_2}$ in Table 4, we know that $c_1 \in \xi'(s')$ or $c_2 \in \xi'(s')$ holds. Assume $c_1 \in \xi'(s')$; since $c_1 \in \xi'(s') \cap F' = B_n \cap F'$, we have $c_1 \in B_n$. As earlier, we consider two cases when c_1 is a pure clock or not. In a first case, if c_1 is pure, $c_1 \notin C$. Since $c_1 \in B_n$, $c_1 \in B_0$ or $c_1 \in \xi_{n-1}(s_{n-1})$ holds. If $c_1 \in B_0$, we immediately have $c_1 \in B$. If $c_1 \in \xi_{n-1}(s_{n-1})$, it must be that $c_1 \in F_{n-1}$. Thus, $c_1 \in \xi_{n-1}(s_{n-1}) \cap F_{n-1} = B \cap F_{n-1}$, hence $c_1 \in B$. Anyhow, we have $c_1 \in B$. Since c_1 is pure, $B \models c_1, \mathbf{D}$. Thus, $B \models d_n, \mathbf{D}$ according to the semantics of $d_n \triangleq c_1 + c_2$ in Table 2. In a second case, if c_1 is not pure, for the same reason stated previously, it must be that $c_1 \in C_{n-1}$. Because $c_1 \in B_n$, $c_1 \in \xi_{n-1}(s_{n-1})$ holds since $B_0 \cap C = \emptyset$. By condition (II) of Step n , we immediately have $B \models c_1, \mathbf{D}$. Again by the semantics of $d_n \triangleq c_1 + c_2$ in Table 2, we get $B \models d_n, \mathbf{D}$.

By now we have proved the preceding conditions (1) and (2).

Let $B_{n+1} = B_n \cup \xi_n(s_n)$, and since $\xi_n(s_n) \supseteq \xi_{n-1}(s_{n-1})$, we have $B_{n+1} = B_0 \cup \xi_{n-1}(s_{n-1}) \cup \xi_n(s_n) = B_0 \cup \xi_n(s_n)$. Let $\mathbf{D}_{n+1} = \mathbf{D}_n - \{d_n \triangleq e_n\} = \mathbf{D} - \{d_0 \triangleq e_0, d_1 \triangleq e_1, \dots, d_n \triangleq e_n\}$, $A_{n+1} = (A_n \cup \{\mathcal{M}_n\}) - \{\mathcal{M}^{d_n \triangleq e_n}, \mathcal{M}_{n-1}\} = (A \cup \{\mathcal{M}_n\}) - \bigcup_{i=0}^n \{\mathcal{M}^{d_i \triangleq e_i}\}$.

We now show that conditions (A), (B), (C) shown earlier in the assumption of Step n also hold for Step $n + 1$. Obviously, $\mathbf{D}_{n+1}$ is well defined. By Lemma B.3, A_{n+1} is well defined. By Proposition 5.6 and Theorem 5.1, it is easy to see that $F_n = \bigcup_{i=0}^{n-1} F_i - (\bigcup_{i=0}^{n-1} F_i \cap \bigcup_{i=0}^k C_i)$ and $C_n = \bigcup_{i=0}^{n-1} C_i$. To show that condition (C) also holds for Step $n + 1$, we observe that for any $N \in A_{n+1} - \{\mathcal{M}_n\}$, there is $\mathcal{M}_n \not\rightsquigarrow^+ N$. If not, suppose $\mathcal{M}_n \rightsquigarrow N_0$ for some N_0 , then $C_{N_0} \cap \text{Clk}_{\mathcal{M}_n} \neq \emptyset$, hence $C_{N_0} \cap (\text{Clk}' \cup \text{Clk}_{n-1}) \neq \emptyset$. Then we have $\mathcal{M}^{d_n \triangleq e_n} \rightsquigarrow N_0$ or $\mathcal{M}_{n-1} \rightsquigarrow N_0$, which contradicts the fact that $\mathcal{M}^{d_n \triangleq e_n} \not\rightsquigarrow N_0$ and $\mathcal{M}_{n-1} \not\rightsquigarrow N_0$.

[the end of Step $n + 1$]

Following the procedure of constructing the state x shown earlier, suppose $|\mathbf{D}| = l$, then after l steps, we can get $\mathcal{M}[F, C] = \mathcal{M}_l[F_l, C_l] = \bigcup_{i=0}^{l-1} \mathcal{M}^{d_i \triangleq e_i}$ and we can construct a unique state $x = s_l = \langle s_0, s_1, \dots, s_{l-1} \rangle$ of $\mathcal{M}$ with $\xi_{\mathcal{M}}(x) = \xi_0(s_0) \cup \dots \cup \xi_{l-1}(s_{l-1})$, which satisfies case (a) or (b). $C = C_l = \bigcup_{i=0}^{l-1} C_{\mathcal{M}^{d_i \triangleq e_i}}$ is exactly the set of clocks that are not pure in $\mathbf{D}$. For any $d \in C$, we have $B \models d, \mathbf{D}$ iff $d \in \xi_{\mathcal{M}}(x)$. $\square$

THE PROOF OF PROPOSITION 6.2. By Definition 6.1, the corresponding OTS of $\mathcal{M}$ is $\mathcal{M}^\triangleright = \langle \Pi^\triangleright, Q, I, \theta^\triangleright, \mathcal{T} \rangle$, where $\Pi^\triangleright = \Pi \cup \{P_c \mid c \in Clk\}$ and for any $q \in Q$, $\theta^\triangleright(q) = \theta(q) \cup \{P_c \mid c \in \xi(q)\}$. Let $\mathcal{M}^{-\phi} = \mathfrak{B}((\neg\phi)^\triangleright)^{-\triangleright} = \langle \Pi_1, Clk_1, Q_1, I_1, \theta_1, \xi_1, \mathcal{T}_1, \mathcal{E}_1 \rangle$, and by Definition 6.1, we know that its corresponding Büchi automaton is $\mathfrak{B}((\neg\phi)^\triangleright) = \langle \Pi_1^\triangleright, Q_1, I_1, \theta_1^\triangleright, \mathcal{T}_1, \mathcal{E}_1 \rangle$, where $\Pi_1^\triangleright = \Pi_1 \cup \{P_c \mid c \in Clk_1\}$ and for any $q \in Q$, $\theta_1^\triangleright(q) = \theta_1 \cup \{P_c \mid c \in \xi_1(q)\}$. According to the observation relation between an OTS and the Büchi automaton of an LTL formula, we have $\mathfrak{B}((\neg\phi)^\triangleright) \text{ obs } \mathcal{M}^\triangleright$. Since $(\neg\phi)^\triangleright = \neg\phi^\triangleright$, $\mathfrak{B}(\neg\phi^\triangleright) \text{ obs } \mathcal{M}^\triangleright$.

To prove $\mathcal{M}^{-\phi} \text{ obs } \mathcal{M}$, we only need to show that for any trace $\pi \in \text{Trace}(\mathcal{M})$,

- (1) for $\pi[0]$, there exists an initial state x_0 of $\mathcal{M}^{-\phi}$ s.t. $\langle \pi[0], x_0 \rangle$ is an initial state of $\mathcal{M} \parallel \mathcal{M}^{-\phi}$;
- (2) for any transition $\langle q, q' \rangle$ of the trace π and any state x of $\mathcal{M}^{-\phi}$ s.t. $\langle q, x \rangle$ is a state of $\mathcal{M} \parallel \mathcal{M}^{-\phi}$, there exists a transition $\langle \langle q, x \rangle, s \rangle$ of $\mathcal{M} \parallel \mathcal{M}^{-\phi}$.

(1): By $\mathfrak{B}((\neg\phi)^\triangleright) \text{ obs } \mathcal{M}^\triangleright$, we know that there is an initial state x_0 of $\mathfrak{B}((\neg\phi)^\triangleright)$ s.t. $\theta^\triangleright(\pi[0]) \cap \Pi_1^\triangleright = \theta_1^\triangleright(x_0) \cap \Pi^\triangleright$ holds. Let $C_1 = \{P_c \mid c \in \xi(\pi[0])\}$, $C_2 = \{P_c \mid c \in Clk_1\}$, $C_3 = \{P_c \mid c \in \xi_1(x_0)\}$ and $C_4 = \{P_c \mid c \in Clk\}$. By the fact that each P_c ($c \in Clk \cup Clk_1$) is a special atomic proposition and is different from all other general atomic propositions in Π and Π_1 , we have

$$\begin{aligned} \theta^\triangleright(\pi[0]) \cap \Pi_1^\triangleright &= (\theta(\pi[0]) \cup C_1) \cap (\Pi_1(x_0) \cup C_2) \\ &= (\theta(\pi[0]) \cap \Pi_1) \cup (C_1 \cap C_2), \\ \theta_1^\triangleright(x_0) \cap \Pi^\triangleright &= (\theta_1(x_0) \cup C_3) \cap (\Pi \cup C_4) \\ &= (\theta_1(x_0) \cap \Pi) \cup (C_3 \cap C_4). \end{aligned}$$

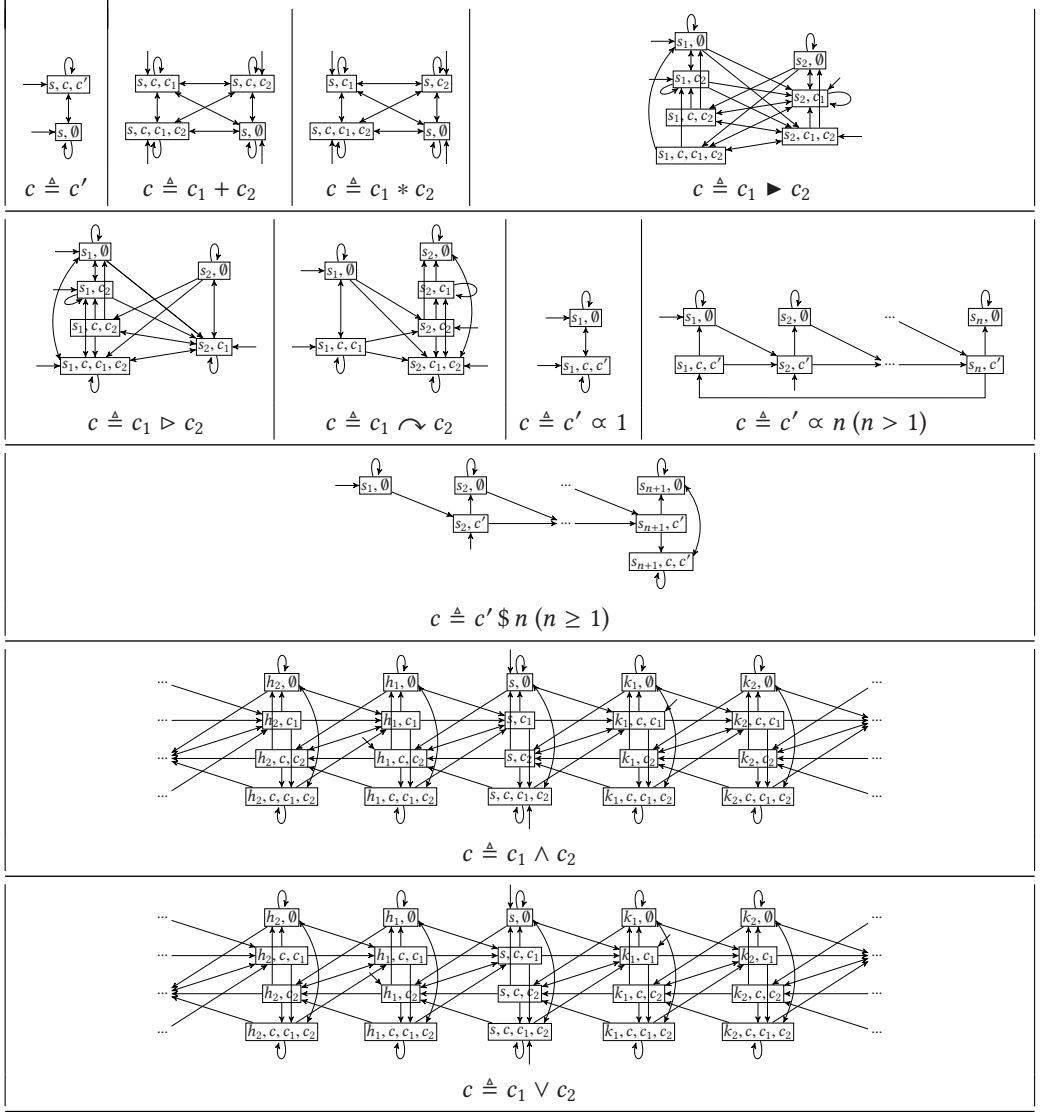
Therefore, we get $(\theta(\pi[0]) \cap \Pi_1) = (\theta_1(x_0) \cap \Pi)$ and $(C_1 \cap C_2) = (C_3 \cap C_4)$ separately. By $(C_1 \cap C_2) = (C_3 \cap C_4)$ and the one-to-one correspondence between a clock c and its corresponding atomic proposition P_c , there is $\xi(\pi[0]) \cap Clk_1 = \xi_1(x_0) \cap Clk$. Hence, $\langle \pi[0], x_0 \rangle$ is an initial state of $\mathcal{M} \parallel \mathcal{M}^{-\phi}$.

(2): From $\mathfrak{B}((\neg\phi)^\triangleright) \text{ obs } \mathcal{M}^\triangleright$, we know that there is a state $\langle q', x' \rangle$ of $\mathfrak{B}((\neg\phi)^\triangleright) \parallel \mathcal{M}^\triangleright$ s.t. $\langle \langle q, x \rangle, \langle q', x' \rangle \rangle$ is a transition of $\mathfrak{B}((\neg\phi)^\triangleright) \parallel \mathcal{M}^\triangleright$. Therefore, $\theta^\triangleright(x') \cap \Pi_1^\triangleright = \theta_1^\triangleright(q') \cap \Pi^\triangleright$. Let $C_1 = \{P_c \mid c \in \xi(s_2)\}$, $C_2 = \{P_c \mid c \in Clk_1\}$, $C_3 = \{P_c \mid c \in \xi_1(q')\}$ and $C_4 = \{P_c \mid c \in Clk\}$. Similar to earlier in (1), we can prove that both $\theta(x') \cap \Pi_1 = \theta_1(q') \cap \Pi$ and $\xi(x') \cap Clk_1 = \xi_1(q') \cap Clk$ hold. Therefore, $\langle \langle q, x \rangle, \langle q', x' \rangle \rangle$ is a transition of $\mathcal{M} \parallel \mathcal{M}^{-\phi}$. $\square$

C CTSA Semantics of Clock Definitions

Table 4 shows the semantics of clock definitions in CCSL.

Table 4. CTSA Semantics of Clock Definitions



D Acknowledgements

We thank Professor Yixiang Chen and Huibiao Zhu for their comments and advisement on this work.

References

- [1] MathWorks. n.d. Simulink. Retrieved June 10, 2024 from <https://www.mathworks.com/products/simulink.html>

- [2] Polychrony. n.d. The Polychrony Toolset. Retrieved June 10, 2024 from <http://www.irisa.fr/espresso/Polychrony>
- [3] IEEE. 2010. *IEEE Standard for Property Specification Language (PSL) - Redline*. IEEE Std 1850-2010 (Revision of IEEE Std 1850-2005) - Redline. IEEE.
- [4] Rajeev Alur and David L. Dill. 1994. A theory of timed automata. *Theoretical Computer Science* 126, 2 (1994), 183–235.
- [5] Rajeev Alur, Tomás Feder, and Thomas A. Henzinger. 1996. The benefits of relaxing punctuality. *Journal of the ACM* 43, 1 (Jan. 1996), 116–146.
- [6] R. Alur and T. A. Henzinger. 1993. Real-time logics: Complexity and expressiveness. *Information and Computation* 104, 1 (1993), 35–77.
- [7] Charles André. 2009. *Syntax and Semantics of the Clock Constraint Specification Language (CCSL)*. Research Report RR-6925. INRIA.
- [8] Charles André, Frédéric Mallet, and Robert de Simone. 2007. Modeling time(s). In *Model Driven Engineering Languages and Systems*, Gregor Engels, Bill Opdyke, Douglas C. Schmidt, and Frank Weil (Eds.). Springer, Berlin, Germany, 559–573.
- [9] K. Ashton. 2009. That “Internet of Things” thing. *RFID Journal* 22, 7 (July 2009), 97–114.
- [10] Christel Baier and Joost P. Katoen. 2008. *Principles of Model Checking*. MIT Press.
- [11] Albert Benveniste, Paul Le Guernic, and Christian Jacquemot. 1991. Synchronous programming with events and relations: The SIGNAL language and its semantics. *Science of Computer Programming* 16, 2 (1991), 103–149.
- [12] Gérard Berry and Georges Gonthier. 1992. The Esterel synchronous programming language: Design, semantics, implementation. *Science of Computer Programming* 19, 2 (1992), 87–152.
- [13] Frédéric Boulanger, Ayman Dogui, Cécile Hardebolle, Christophe Jacquet, Dominique Marcadet, and Iuliana Prodan. 2012. Semantic adaptation using CCSL clock constraints. In *Models in Software Engineering*, Jörg Kienzle (Ed.). Springer, Berlin, Germany, 104–118.
- [14] L. Brim, P. Dluhoš, D. Šafránek, and T. Vejnostek. 2014. STL*: Extending signal temporal logic with signal-value freezing operator. *Information and Computation* 236 (2014), 52–67.
- [15] P. Caspi, D. Pilaud, N. Halbwachs, and J. A. Plaice. 1987. LUSTRE: A declarative language for real-time programming. In *Proceedings of the 14th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages (POPL’87)*. ACM, New York, NY, USA, 178–188.
- [16] Manuel Clavel, Francisco Durán, Steven Eker, Patrick Lincoln, Narciso Martí-Oliet, Jose Meseguer, and Carolyn Talcott. 2007. *All About Maude—A High-Performance Logical Framework: How to Specify, Program, and Verify Systems in Rewriting Logic*. Lecture Notes in Computer Science, Vol. 4350. Springer.
- [17] Jean-Louis Colaço, Bruno Pagano, and Marc Pouzet. 2017. SCADE 6: A formal language for embedded critical software development (invited paper). In *Proceedings of the 2017 11th International Symposium on Theoretical Aspects of Software Engineering (TASE’17)*. IEEE, 1–11. <https://doi.org/10.1109/TASE.2017.8285623>
- [18] Alexandre Donzé, Oded Maler, Ezio Bartocci, Dejan Nickovic, Radu Grosu, and Scott Smolka. 2012. On temporal logic and signal processing. In *Automated Technology for Verification and Analysis*, Supratik Chakraborty and Madhavan Mukund (Eds.). Springer, Berlin, Germany, 92–106.
- [19] Cindy Eisner, Dana Fisman, John Havlicek, Anthony McIsaac, and David Van Campenhout. 2003. The definition of a temporal clock operator. In *Automata, Languages and Programming*, Jos C. M. Baeten, Jan Karel Lenstra, Joachim Parrow, and Gerhard J. Woeginger (Eds.). Springer, Berlin, Germany, 857–870.
- [20] Abdoulaye Gamatié, Thierry Gautier, Paul Le Guernic, and Jean-Pierre Talpin. 2007. Polychronous design of embedded real-time applications. *ACM Transactions on Software Engineering and Methodology* 16, 2 (2007), 9–es.
- [21] R. Gascon, F. Mallet, and J. Deantoni. 2011. Logical time and temporal logics: Comparing UML MARTE/CCSL and PSL. In *Proceedings of the 2011 18th International Symposium on Temporal Representation and Reasoning*. 141–148.
- [22] Calin Glitia, Julien Deantoni, Frédéric Mallet, Jean-Vivien Millo, Pierre Boulet, and Abdoulaye Gamatié. 2012. Progressive and explicit refinement of scheduling for multidimensional data-flow applications using UML MARTE. *Design Automation for Embedded Systems* 16, 2 (2012), 137–169. <https://doi.org/10.1007/s10617-012-9093-y>
- [23] Thomas A. Henzinger, Benjamin Horowitz, and Christoph Meyer Kirsch. 2001. Giotto: A time-triggered language for embedded programming. In *Embedded Software*, Thomas A. Henzinger and Christoph M. Kirsch (Eds.). Springer, Berlin, Germany, 166–184.
- [24] Gerard J. Holzmann. 1997. The model checker SPIN. *IEEE Transactions on Software Engineering* 23, 5 (May 1997), 279–295.
- [25] Ming Hu, Tongquan Wei, Min Zhang, Frédéric Mallet, and Mingsong Chen. 2019. Sample-guided automated synthesis for CCSL specifications. In *Proceedings of the 56th Annual Design Automation Conference 2019 (DAC’19)*. ACM, New York, NY, USA, Article 98, 6 pages. <https://doi.org/10.1145/3316781.3317904>
- [26] Leslie Lamport. 1978. Time, clocks, and the ordering of events in a distributed system. *Communications of the ACM* 21, 7 (July 1978), 558–565.

- [27] Paul Le Guernic, Jean-Pierre Talpin, and Jean-Christophe Le Lann. 2003. POLYCHRONY for system design. *Journal of Circuits, Systems and Computers* 12, 3 (2003), 261–303.
- [28] E. A. Lee. 2008. Cyber physical systems: Design Challenges. In *Proceedings of the 2008 11th IEEE International Symposium on Object and Component-Oriented Real-Time Distributed Computing (ISORC'08)*. 363–369.
- [29] Edward A. Lee and Marten Lohstroh. 2021. Time for all programs, not just real-time programs. In *Leveraging Applications of Formal Methods, Verification and Validation*, Tiziana Margaria and Bernhard Steffen (Eds.). Springer International Publishing, Cham, Switzerland, 213–232.
- [30] Edward A. Lee and Alberto L. Sangiovanni-Vincentelli. 1998. A framework for comparing models of computation. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 17, 12 (1998), 1217–1229.
- [31] Edward A. Lee and Sanjit A. Seshia. 2011. *Introduction to Embedded Systems: A Cyber-Physical Systems Approach*. MIT Press.
- [32] Chuchang Liu and Mehmet A. Orgun. 1996. Dealing with multiple granularity of time in temporal logic programming. *Journal of Symbolic Computation* 22, 5-6 (1996), 699–720.
- [33] Chuchang Liu and Mehmet A. Orgun. 1999. Verification of reactive systems using temporal logic with clocks. *Theoretical Computer Science* 220, 2 (June 1999), 377–408.
- [34] Oded Maler and Dejan Nickovic. 2004. Monitoring temporal properties of continuous signals. In *Formal Techniques, Modelling and Analysis of Timed and Fault-Tolerant Systems*, Yassine Lakhnech and Sergio Yovine (Eds.). Springer, Berlin, Germany, 152–166.
- [35] Frédéric Mallet. 2008. Clock constraint specification language: Specifying clock constraints with UML/MARTE. *Innovations in Systems and Software Engineering* 4, 3 (2008), 309–314.
- [36] Frédéric Mallet and Robert de Simone. 2015. Correctness issues on MARTE/CCSL constraints. *Science of Computer Programming* 106 (2015), 78–92.
- [37] Frédéric Mallet, Jean-Vivien Millo, and Robert de Simone. 2013. Safe CCSL specifications and marked graphs. In *Proceedings of the 2013 11th ACM/IEEE International Conference on Formal Methods and Models for Codesign (MEM-CODE'13)*. IEEE, 157–166.
- [38] OMG. 2011. *UML Profile for MARTE: Modeling and Analysis of Real-Time Embedded Systems*. Technical Report. OMG.
- [39] Claudius Ptolemaeus (Ed.). 2014. *System Design, Modeling, and Simulation Using Ptolemy II*. Ptolemy.org.
- [40] Min Zhang and Yunhui Ying. 2017. Towards SMT-based LTL model checking of clock constraint specification language for real-time and embedded systems. In *Proceedings of the 18th ACM SIGPLAN/SIGBED Conference on Languages, Compilers, and Tools for Embedded Systems (LCTES'17)*. ACM, New York, NY, USA, 61–70. <https://doi.org/10.1145/3078633.3081035>
- [41] Yuanrui Zhang, Hengyang Wu, Yixiang Chen, and Frédéric Mallet. 2021. A clock-based dynamic logic for the verification of CCSL specifications in synchronous systems. *Science of Computer Programming* 203 (2021), 102591. <https://doi.org/10.1016/j.scico.2020.102591>

Received 24 July 2022; revised 8 December 2023; accepted 23 May 2024